```
27
28      // 转换成非齐次坐标
29      for (int i = 0; i < pts_4d.cols; i++) {
30          Mat x = pts_4d.col(i);
31          x /= x.at<float>(3, 0); // 归一化
32          Point3d p(
33              x.at<float>(0, 0),
34              x.at<float>(1, 0),
35              x.at<float>(2, 0)
36          );
37          points.push_back(p);
38      }
39  }
```

同时，在 main 函数中加入三角测量部分，然后画出各点的深度示意图。读者可以自行运行此程序查看三角化结果。

7.6.2　讨论

关于三角测量，还有一个必须注意的地方。

三角测量是由**平移**得到的，有平移才会有对极几何中的三角形，才谈得上三角测量。因此，纯旋转是无法使用三角测量的，因为在平移为零时，对极约束一直为零。当然，实际数据往往不会完全等于零。在平移存在的情况下，我们还要关心三角测量的不确定性，这会引出一个**三角测量的矛盾**。

如图 7-12 所示，当平移很小时，像素上的不确定性将导致较大的深度不确定性。也就是说，如果特征点运动一个像素 δx，使得视线角变化了一个角度 $\delta\theta$，那么将测量到深度值有 δd 的变化。从几何关系可以看到，当 $\boldsymbol{t}$ 较大时，δd 将明显变小，这说明平移较大时，在同样的相机分辨率下，三角化测量将更精确。对该过程的定量分析可以使用正弦定理得到。

因此，要提高三角化的精度，一种方式是提高特征点的提取精度，也就是提高图像分辨率——但这会导致图像变大，增加计算成本。另一种方式是使平移量增大。但是，这会导致图像的**外观**发生明显的变化，比如箱子原先被挡住的侧面显示出来，或者物体的光照发生变化，等等。外观变化会使得特征提取与匹配变得困难。总而言之，增大平移，可能导致匹配失效；而平移太小，则三角化精度不够——这就是三角化的矛盾。我们把这个问题称为“视差”（parallax）。

在单目视觉中，由于单目图像没有深度信息，我们要等待特征点被追踪几帧之后，产生了足够的视角，再用三角化来确定新增特征点的深度值。这有时也被称为延迟三角化[55]。但是，如果相机发生了原地旋转，导致视差很小，就不好估计新观测到的特征点的深度。这种情况在机器人场合下更常见，因为原地旋转往往是一个机器人常见的指令。在这种情况下，单目视觉就可能出现追踪失败、尺度不正确等情况。

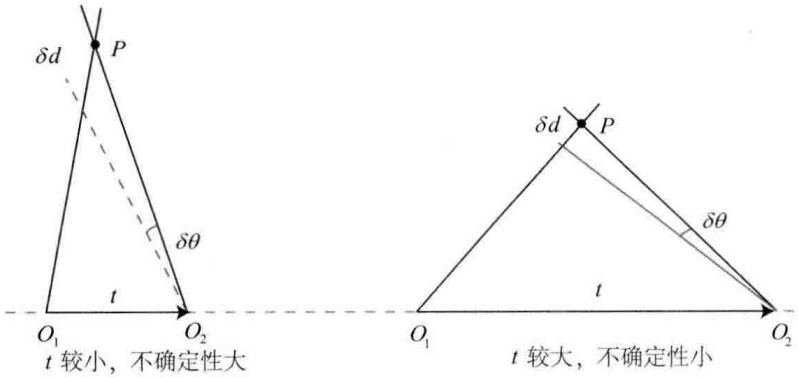


图 7-12 三角测量的矛盾

虽然本节只介绍了三角化的深度估计，但只要我们愿意，也能够定量地计算每个特征点的**位置及不确定性**。所以，如果假设特征点服从高斯分布，并且不断地对它进行观测，在信息正确的情况下，我们就能够期望**它的方差不断减小乃至收敛**。这就得到了一个**滤波器**，称为**深度滤波器（Depth Filter）**。不过，由于它的原理较复杂，我们将留到后面章节详细讨论。下面，我们讨论如何通过 3D-2D 的匹配点估计相机运动，以及 3D-3D 的估计方法。

7.7 3D-2D：PnP

PnP（Perspective-n-Point）是求解 3D 到 2D 点对运动的方法。它描述了当知道 n 个 3D 空间点及其投影位置时，如何估计相机的位姿。前面说到，2D-2D 的对极几何方法需要 8 个或 8 个以上的点对（以八点法为例），且存在着初始化、纯旋转和尺度的问题。然而，如果两张图像中的一张特征点的 3D 位置已知，那么最少只需 3 个点对（以及至少一个额外点验证结果）就可以估计相机运动。特征点的 3D 位置可以由三角化或者 RGB-D 相机的深度图确定。因此，在双目或 RGB-D 的视觉里程计中，我们可以直接使用 PnP 估计相机运动。而在单目视觉里程计中，必须先进行初始化，才能使用 PnP。3D-2D 方法不需要使用对极约束，又可以在很少的匹配点中获得较好的运动估计，是一种最重要的姿态估计方法。

PnP 问题有很多种求解方法，例如，用 3 对点估计位姿的 P3P[56]、直接线性变换（DLT）、EPnP（Efficient PnP）[57]、UPnP[58]，等等。此外，还能用**非线性优化**的方式，构建最小二乘问题并迭代求解，也就是万金油式的光束法平差（Bundle Adjustment，BA）。我们先来介绍 DLT，再讲解 BA。

7.7.1 直接线性变换

我们考虑这样一个问题：已知一组 3D 点的位置，以及它们在某个相机中的投影位置，求该相机的位姿。这个问题也可以用于求解给定地图和图像时的相机状态问题。如果把 3D 点看成在

另一个相机坐标系中的点的话，则也可以用来求解两个相机的相对运动问题。我们从简单的问题出发。

考虑某个空间点 P，它的齐次坐标为 $\boldsymbol{P} = (X, Y, Z, 1)^{\mathrm{T}}$。在图像 I_1 中，投影到特征点 $\boldsymbol{x}_1 = (u_1, v_1, 1)^{\mathrm{T}}$（以归一化平面齐次坐标表示）。此时，相机的位姿 $\boldsymbol{R}, \boldsymbol{t}$ 是未知的。与单应矩阵的求解类似，我们定义增广矩阵 $[\boldsymbol{R}|\boldsymbol{t}]$ 为一个 3×4 的矩阵，包含了旋转与平移信息[①]。我们将其展开形式列写如下：

$$s \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = \begin{pmatrix} t_1 & t_2 & t_3 & t_4 \\ t_5 & t_6 & t_7 & t_8 \\ t_9 & t_{10} & t_{11} & t_{12} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}. \tag{7.26}$$

用最后一行把 s 消去，得到两个约束：

$$u_1 = \frac{t_1 X + t_2 Y + t_3 Z + t_4}{t_9 X + t_{10} Y + t_{11} Z + t_{12}}, \quad v_1 = \frac{t_5 X + t_6 Y + t_7 Z + t_8}{t_9 X + t_{10} Y + t_{11} Z + t_{12}}.$$

为了简化表示，定义 $\boldsymbol{T}$ 的行向量：

$$\boldsymbol{t}_1 = (t_1, t_2, t_3, t_4)^{\mathrm{T}}, \boldsymbol{t}_2 = (t_5, t_6, t_7, t_8)^{\mathrm{T}}, \boldsymbol{t}_3 = (t_9, t_{10}, t_{11}, t_{12})^{\mathrm{T}},$$

于是有

$$\boldsymbol{t}_1^{\mathrm{T}} \boldsymbol{P} - \boldsymbol{t}_3^{\mathrm{T}} \boldsymbol{P} u_1 = 0,$$

和

$$\boldsymbol{t}_2^{\mathrm{T}} \boldsymbol{P} - \boldsymbol{t}_3^{\mathrm{T}} \boldsymbol{P} v_1 = 0.$$

请注意，$\boldsymbol{t}$ 是待求的变量，可以看到，每个特征点提供了两个关于 $\boldsymbol{t}$ 的线性约束。假设一共有 N 个特征点，则可以列出如下线性方程组：

$$\begin{pmatrix} \boldsymbol{P}_1^{\mathrm{T}} & 0 & -u_1 \boldsymbol{P}_1^{\mathrm{T}} \\ 0 & \boldsymbol{P}_1^{\mathrm{T}} & -v_1 \boldsymbol{P}_1^{\mathrm{T}} \\ \vdots & \vdots & \vdots \\ \boldsymbol{P}_N^{\mathrm{T}} & 0 & -u_N \boldsymbol{P}_N^{\mathrm{T}} \\ 0 & \boldsymbol{P}_N^{\mathrm{T}} & -v_N \boldsymbol{P}_N^{\mathrm{T}} \end{pmatrix} \begin{pmatrix} \boldsymbol{t}_1 \\ \boldsymbol{t}_2 \\ \boldsymbol{t}_3 \end{pmatrix} = 0. \tag{7.27}$$

①请注意，这和 SE(3) 中的变换矩阵 $\boldsymbol{T}$ 是不同的。

$\boldsymbol{t}$ 一共有 12 维，因此最少通过 6 对匹配点即可实现矩阵 $\boldsymbol{T}$ 的线性求解，这种方法称为 DLT。当匹配点大于 6 对时，也可以使用 SVD 等方法对超定方程求最小二乘解。

在 DLT 求解中，我们直接将 $\boldsymbol{T}$ 矩阵看成了 12 个未知数，忽略了它们之间的联系。因为旋转矩阵 $\boldsymbol{R} \in \mathrm{SO}(3)$，用 DLT 求出的解不一定满足该约束，它是一个一般矩阵。平移向量比较好办，它属于向量空间。对于旋转矩阵 $\boldsymbol{R}$，我们必须针对 DLT 估计的 $\boldsymbol{T}$ 左边 3×3 的矩阵块，寻找一个最好的旋转矩阵对它进行近似。这可以由 QR 分解完成[3, 59]，也可以像这样来计算[6, 60]：

$$\boldsymbol{R} \leftarrow \left(\boldsymbol{R}\boldsymbol{R}^{\mathrm{T}}\right)^{-\frac{1}{2}}\boldsymbol{R}. \tag{7.28}$$

这相当于把结果从矩阵空间重新投影到 SE(3) 流形上，转换成旋转和平移两部分。

需要解释的是，我们这里的 $\boldsymbol{x}_1$ 使用了归一化平面坐标，去掉了内参矩阵 $\boldsymbol{K}$ 的影响——这是因为内参 $\boldsymbol{K}$ 在 SLAM 中通常假设为已知。即使内参未知，也能用 PnP 去估计 $\boldsymbol{K}, \boldsymbol{R}, \boldsymbol{t}$ 三个量。然而由于未知量增多，效果会差一些。

7.7.2 P3P

下面讲的 P3P 是另一种解 PnP 的方法。它仅使用 3 对匹配点，对数据要求较少，因此这里简单介绍（这部分推导借鉴了参考文献 [61]）。

P3P 需要利用给定的 3 个点的几何关系。它的输入数据为 3 对 3D–2D 匹配点。记 3D 点为 A, B, C，2D 点为 a, b, c，其中小写字母代表的点为对应大写字母代表的点在相机成像平面上的投影，如图 7-13 所示。此外，P3P 还需要使用一对验证点，以从可能的解中选出正确的那一个（类似于对极几何情形）。记验证点对为 $D - d$，相机光心为 O。请注意，我们知道的是 A, B, C 在**世界坐标系中的坐标**，而不是**在相机坐标系中的坐标**。一旦 3D 点在相机坐标系下的坐标能够算出，我们就得到了 3D–3D 的对应点，把 PnP 问题转换为了 ICP 问题。

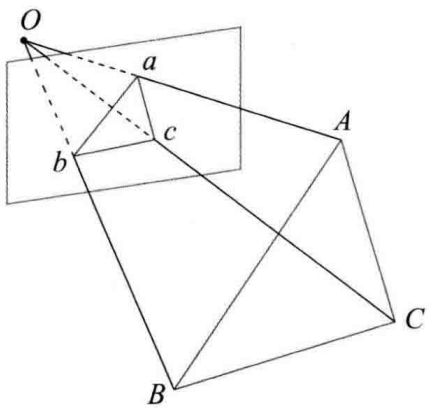


图 7-13 P3P 问题示意图

首先，显然三角形之间存在对应关系：

$$\Delta Oab - \Delta OAB, \quad \Delta Obc - \Delta OBC, \quad \Delta Oac - \Delta OAC. \tag{7.29}$$

来考虑 Oab 和 OAB 的关系。利用余弦定理，有

$$OA^2 + OB^2 - 2OA \cdot OB \cdot \cos\langle a, b\rangle = AB^2. \tag{7.30}$$

对于其他两个三角形也有类似性质，于是有

$$\begin{aligned} OA^2 + OB^2 - 2OA \cdot OB \cdot \cos\langle a, b\rangle &= AB^2 \\ OB^2 + OC^2 - 2OB \cdot OC \cdot \cos\langle b, c\rangle &= BC^2 \\ OA^2 + OC^2 - 2OA \cdot OC \cdot \cos\langle a, c\rangle &= AC^2. \end{aligned} \tag{7.31}$$

对以上三式全体除以 OC^2，并且记 $x = OA/OC, y = OB/OC$，得

$$\begin{aligned} x^2 + y^2 - 2xy\cos\langle a, b\rangle &= AB^2/OC^2 \\ y^2 + 1^2 - 2y\cos\langle b, c\rangle &= BC^2/OC^2 \\ x^2 + 1^2 - 2x\cos\langle a, c\rangle &= AC^2/OC^2. \end{aligned} \tag{7.32}$$

记 $v = AB^2/OC^2, uv = BC^2/OC^2, wv = AC^2/OC^2$，有

$$\begin{aligned} x^2 + y^2 - 2xy\cos\langle a, b\rangle - v &= 0 \\ y^2 + 1^2 - 2y\cos\langle b, c\rangle - uv &= 0 \\ x^2 + 1^2 - 2x\cos\langle a, c\rangle - wv &= 0. \end{aligned} \tag{7.33}$$

我们可以把第一个式子中的 v 放到等式一边，并代入其后两式，得

$$\begin{aligned} (1-u)\,y^2 - ux^2 - \cos\langle b, c\rangle\, y + 2uxy\cos\langle a, b\rangle + 1 &= 0 \\ (1-w)\,x^2 - wy^2 - \cos\langle a, c\rangle\, x + 2wxy\cos\langle a, b\rangle + 1 &= 0. \end{aligned} \tag{7.34}$$

注意这些方程中的已知量和未知量。由于我们知道 2D 点的图像位置，3 个余弦角 $\cos\langle a, b\rangle$, $\cos\langle b, c\rangle$, $\cos\langle a, c\rangle$ 是已知的。同时，$u = BC^2/AB^2, w = AC^2/AB^2$ 可以通过 A, B, C 在世界坐标系下的坐标算出，变换到相机坐标系下之后，这个比值并不改变。该式中的 x, y 是未知的，随着相机移动会发生变化。因此，该方程组是关于 x, y 的一个二元二次方程（多项式方程）。求该

方程组的解析解是一个复杂的过程，需要用吴消元法。这里不展开对该方程解法的介绍，感兴趣的读者请参阅文献 [56]。类似于分解 $\boldsymbol{E}$ 的情况，该方程最多可能得到 4 个解，但我们可以用验证点来计算最可能的解，得到 A, B, C 在相机坐标系下的 3D 坐标。然后，根据 3D–3D 的点对，计算相机的运动 $\boldsymbol{R}, \boldsymbol{t}$。这部分将在 7.9 节介绍。

从 P3P 的原理可以看出，为了求解 PnP，我们利用了三角形相似性质，求解投影点 a, b, c 在相机坐标系下的 3D 坐标，最后把问题转换成一个 3D 到 3D 的位姿估计问题。在后文中将看到，带有匹配信息的 3D–3D 位姿求解非常容易，所以这种思路是非常有效的。一些其他的方法，例如 EPnP，也采用了这种思路。然而，P3P 也存在着一些问题：

1. P3P 只利用 3 个点的信息。当给定的配对点多于 3 组时，难以利用更多的信息。
2. 如果 3D 点或 2D 点受噪声影响，或者存在误匹配，则算法失效。

所以，人们还陆续提出了许多别的方法，如 EPnP、UPnP 等。它们利用更多的信息，而且用迭代的方式对相机位姿进行优化，以尽可能地消除噪声的影响。不过，相对 P3P 来说，它们的原理更复杂，所以我们建议读者阅读原始的论文，或通过实践来理解 PnP 的过程。在 SLAM 中，通常的做法是先使用 P3P/EPnP 等方法估计相机位姿，再构建最小二乘优化问题对估计值进行调整（即进行 Bundle Adjustment）。在相机运动足够连续时，也可以假设相机不动或匀速运动，用推测值作为初始值进行优化。接下来，我们从非线性优化的角度来看 PnP 问题。

7.7.3 最小化重投影误差求解 PnP

除了使用线性方法，我们还可以把 PnP 问题构建成一个关于重投影误差的非线性最小二乘问题。这将用到本书第 4 讲和第 5 讲的知识。前面说的线性方法，往往是**先求相机位姿，再求空间点位置**，而非线性优化则是把它们都看成优化变量，放在一起优化。这是一种非常通用的求解方式，我们可以用它对 PnP 或 ICP 给出的结果进行优化。这一类**把相机和三维点放在一起进行最小化**的问题，统称为 Bundle Adjustment[①]。

我们完全可以在 PnP 中构建一个 Bundle Adjustment 问题对相机位姿进行优化。如果相机是连续运动的（比如大多数 SLAM 过程），也可以直接用 BA 求解相机位姿。我们将在本节给出此问题在两个视图下的基本形式，然后在第 9 讲讨论较大规模的 BA 问题。

考虑 n 个三维空间点 P 及其投影 p，我们希望计算相机的位姿 $\boldsymbol{R}, \boldsymbol{t}$，它的李群表示为 $\boldsymbol{T}$。假设某空间点坐标为 $\boldsymbol{P}_i = [X_i, Y_i, Z_i]^{\mathrm{T}}$，其投影的像素坐标为 $\boldsymbol{u}_i = [u_i, v_i]^{\mathrm{T}}$。根据第 5 讲的内容，像素位置与空间点位置的关系如下：

①需要说明的是，BA 在不同文献、语境下的意义并不完全一致。有些学者仅把最小化重投影误差的问题称为 BA，而另一些学者的 BA 概念更宽泛，即使这个 BA 只有一个相机，或加入了其他类似的传感器，也可以统称为 BA。笔者个人更喜欢宽泛一些的 BA 概念，所以在这里计算 PnP 的方法也称为 BA。

$$s_i \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \boldsymbol{K}\boldsymbol{T} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}. \tag{7.35}$$

写成矩阵形式就是：

$$s_i \boldsymbol{u}_i = \boldsymbol{K}\boldsymbol{T}\boldsymbol{P}_i.$$

这个式子隐含了一次从齐次坐标到非齐次的转换，否则按矩阵的乘法来说，维度是不对的[①]。现在，由于相机位姿未知及观测点的噪声，该等式存在一个误差。因此，我们把误差求和，构建最小二乘问题，然后寻找最好的相机位姿，使它最小化：

$$\boldsymbol{T}^* = \arg\min_{\boldsymbol{T}} \frac{1}{2} \sum_{i=1}^{n} \left\| \boldsymbol{u}_i - \frac{1}{s_i} \boldsymbol{K}\boldsymbol{T}\boldsymbol{P}_i \right\|_2^2. \tag{7.36}$$

该问题的误差项，是将 3D 点的投影位置与观测位置作差，所以称为**重投影误差**。使用齐次坐标时，这个误差有 3 维。不过，由于 $\boldsymbol{u}$ 最后一维为 1，该维度的误差一直为零，因而我们更多时候使用非齐次坐标，于是误差就只有 2 维了。如图 7-14 所示，我们通过特征匹配知道了 p_1 和 p_2 是同一个空间点 P 的投影，但是不知道相机的位姿。在初始值中，P 的投影 $\hat{p}_2$ 与实际的 p_2 之间有一定的距离。于是我们调整相机的位姿，使得这个距离变小。不过，由于这个调整需要考虑很多个点，所以最后的效果是整体误差的缩小，而每个点的误差通常都不会精确为零。

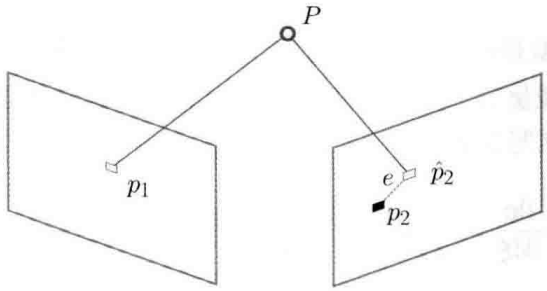


图 7-14　重投影误差示意图

最小二乘优化问题已经在第 6 讲介绍过了。使用李代数，可以构建无约束的优化问题，很方便地通过高斯牛顿法、列文伯格—马夸尔特方法等优化算法进行求解。不过，在使用高斯牛顿法和列文伯格—马夸尔特方法之前，我们需要知道每个误差项关于优化变量的导数，也就是**线性**

①$\boldsymbol{T}\boldsymbol{P}_i$ 结果是 4×1 的，而其左侧的 $\boldsymbol{K}$ 是 3×3 的，所以必须把 $\boldsymbol{T}\boldsymbol{P}_i$ 的前三维取出来，变成三维的非齐次坐标。或者，用 $\boldsymbol{R}\boldsymbol{P} + \boldsymbol{t}$ 亦无不可。

化：

$$\boldsymbol{e}(\boldsymbol{x}+\Delta\boldsymbol{x}) \approx \boldsymbol{e}(\boldsymbol{x}) + \boldsymbol{J}^{\mathrm{T}}\Delta\boldsymbol{x}. \tag{7.37}$$

这里的 $\boldsymbol{J}^{\mathrm{T}}$ 的形式是值得讨论的，甚至可以说是关键所在。我们固然可以使用数值导数，但如果能够推导出解析形式，则优先考虑解析导数。现在，当 $\boldsymbol{e}$ 为像素坐标误差（2 维），$\boldsymbol{x}$ 为相机位姿（6 维）时，$\boldsymbol{J}^{\mathrm{T}}$ 将是一个 2×6 的矩阵。我们来推导 $\boldsymbol{J}^{\mathrm{T}}$ 的形式。

回忆李代数的内容，我们介绍了如何使用扰动模型来求李代数的导数。首先，记变换到相机坐标系下的空间点坐标为 $\boldsymbol{P}'$，并且将其前 3 维取出来：

$$\boldsymbol{P}' = (\boldsymbol{T}\boldsymbol{P})_{1:3} = [X', Y', Z']^{\mathrm{T}}. \tag{7.38}$$

那么，相机投影模型相对于 $\boldsymbol{P}'$ 为

$$s\boldsymbol{u} = \boldsymbol{K}\boldsymbol{P}'. \tag{7.39}$$

展开：

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}. \tag{7.40}$$

利用第 3 行消去 s（实际上就是 $\boldsymbol{P}'$ 的距离），得

$$u = f_x \frac{X'}{Z'} + c_x, \quad v = f_y \frac{Y'}{Z'} + c_y. \tag{7.41}$$

这与第 5 讲的相机模型是一致的。当我们求误差时，可以把这里的 u, v 与实际的测量值比较，求差。在定义了中间变量后，我们对 $\boldsymbol{T}$ 左乘扰动量 $\delta\boldsymbol{\xi}$，然后考虑 $\boldsymbol{e}$ 的变化关于扰动量的导数。利用链式法则，可以列写如下：

$$\frac{\partial \boldsymbol{e}}{\partial \delta\boldsymbol{\xi}} = \lim_{\delta\boldsymbol{\xi}\to 0} \frac{\boldsymbol{e}(\delta\boldsymbol{\xi}\oplus\boldsymbol{\xi}) - \boldsymbol{e}(\boldsymbol{\xi})}{\delta\boldsymbol{\xi}} = \frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}'}\frac{\partial \boldsymbol{P}'}{\partial \delta\boldsymbol{\xi}}. \tag{7.42}$$

这里的 $\oplus$ 指李代数上的左乘扰动。第一项是误差关于投影点的导数，在式 (7.41) 中已经列出了变量之间的关系，易得

$$\frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}'} = -\begin{bmatrix} \dfrac{\partial u}{\partial X'} & \dfrac{\partial u}{\partial Y'} & \dfrac{\partial u}{\partial Z'} \\ \dfrac{\partial v}{\partial X'} & \dfrac{\partial v}{\partial Y'} & \dfrac{\partial v}{\partial Z'} \end{bmatrix} = -\begin{bmatrix} \dfrac{f_x}{Z'} & 0 & -\dfrac{f_x X'}{Z'^2} \\ 0 & \dfrac{f_y}{Z'} & -\dfrac{f_y Y'}{Z'^2} \end{bmatrix}. \tag{7.43}$$

而第二项为变换后的点关于李代数的导数，根据 4.3.5 节中的推导，得

$$\frac{\partial (\boldsymbol{TP})}{\partial \delta \boldsymbol{\xi}} = (\boldsymbol{TP})^{\odot} = \begin{bmatrix} \boldsymbol{I} & -\boldsymbol{P}'^{\wedge} \\ \boldsymbol{0}^{\mathrm{T}} & \boldsymbol{0}^{\mathrm{T}} \end{bmatrix}. \tag{7.44}$$

而在 $\boldsymbol{P}'$ 的定义中，我们取出了前 3 维，于是得

$$\frac{\partial \boldsymbol{P}'}{\partial \delta \boldsymbol{\xi}} = [\boldsymbol{I}, -\boldsymbol{P}'^{\wedge}]. \tag{7.45}$$

将这两项相乘，就得到了 2×6 的雅可比矩阵：

$$\frac{\partial \boldsymbol{e}}{\partial \delta \boldsymbol{\xi}} = -\begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} & -\frac{f_x X' Y'}{Z'^2} & f_x + \frac{f_x X'^2}{Z'^2} & -\frac{f_x Y'}{Z'} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} & -f_y - \frac{f_y Y'^2}{Z'^2} & \frac{f_y X' Y'}{Z'^2} & \frac{f_y X'}{Z'} \end{bmatrix}. \tag{7.46}$$

这个雅可比矩阵描述了重投影误差关于相机位姿李代数的一阶变化关系。我们保留了前面的负号，这是因为误差是由**观测值减预测值**定义的。当然也可以反过来，将它定义成“预测值减观测值”的形式。在那种情况下，只要去掉前面的负号即可。此外，如果 $\mathfrak{se}(3)$ 的定义方式是旋转在前，平移在后，则只要把这个矩阵的前 3 列与后 3 列对调即可。

除了优化位姿，我们还希望优化特征点的空间位置。因此，需要讨论 $\boldsymbol{e}$ 关于空间点 $\boldsymbol{P}$ 的导数。所幸这个导数矩阵相对来说容易一些。仍利用链式法则，有

$$\frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}} = \frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}'} \frac{\partial \boldsymbol{P}'}{\partial \boldsymbol{P}}. \tag{7.47}$$

第一项在前面已推导，关于第二项，按照定义

$$\boldsymbol{P}' = (\boldsymbol{TP})_{1:3} = \boldsymbol{RP} + \boldsymbol{t},$$

我们发现 $\boldsymbol{P}'$ 对 $\boldsymbol{P}$ 求导后将只剩下 $\boldsymbol{R}$。于是：

$$\frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}} = -\begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} \end{bmatrix} \boldsymbol{R}. \tag{7.48}$$

于是，我们推导出了观测相机方程关于相机位姿与特征点的两个导数矩阵。它们**十分重要**，能够在优化过程中提供重要的梯度方向，指导优化的迭代。

7.8 实践：求解 PnP

7.8.1 使用 EPnP 求解位姿

下面，我们通过实验理解 PnP 的过程。首先，我们演示如何使用 OpenCV 的 EPnP 求解 PnP 问题，然后通过非线性优化再次求解。与第 1 版相比，本书中，我们将增加一个手写优化的实验。由于 PnP 需要使用 3D 点，为了避免初始化带来的麻烦，我们使用了 RGB-D 相机中的深度图（1_depth.png）作为特征点的 3D 位置。首先，来看 OpenCV 提供的 PnP 函数：

slambook2/ch7/pose_estimation_3d2d.cpp（片段）

```
int main( int argc, char** argv ) {
    Mat r, t;
    solvePnP(pts_3d, pts_2d, K, Mat(), r, t, false); // 调用OpenCV的PnP求解，可选择EPNP、DLS等方法
    Mat R;
    cv::Rodrigues(r, R); // r为旋转向量形式，用Rodrigues公式转换为矩阵
    cout << "R=" << endl << R << endl;
    cout << "t=" << endl << t << endl;
}
```

在例程中，得到配对特征点后，我们在第一个图的深度图中寻找它们的深度，并求出空间位置。以此空间位置为 3D 点，再以第二个图像的像素位置为 2D 点，调用 EPnP 求解 PnP 问题。程序输出如下：

终端输入：

```
% build/pose_estimation_3d2d 1.png 2.png d1.png d2.png
-- Max dist : 95.000000
-- Min dist : 4.000000
一共找到了79组匹配点
3d-2d pairs: 76
R=
[0.9978662025826269, -0.05167241613316376, 0.03991244360207524;
0.0505958915956335, 0.998339762771668, 0.02752769192381471;
-0.04126860182960625, -0.025449547736074, 0.998823919929363]
t=
[-0.1272259656955879;
-0.007507297652615337;
0.06138584177157709]
```

读者可以对比先前 2D–2D 情况下求解的 $\boldsymbol{R}, \boldsymbol{t}$ 看看有什么不同。可以看到，在有 3D 信息时，估计的 $\boldsymbol{R}$ 几乎是相同的，而 $\boldsymbol{t}$ 相差得较多。这是由于引入了新的深度信息所致。不过，由于 Kinect

采集的深度图本身会有一些误差，所以这里的 3D 点也不是准确的。在较大规模的 BA 中，我们会希望把位姿和所有三维特征点同时优化。

7.8.2　手写位姿估计

下面演示如何使用非线性优化的方式计算相机位姿。我们先手写一个高斯牛顿法的 PnP，然后演示如何调用 g2o 来求解。

slambook2/ch7/pose_estimation_3d2d.cpp（片段）

```
void bundleAdjustmentGaussNewton(
const VecVector3d &points_3d,
const VecVector2d &points_2d,
const Mat &K,
Sophus::SE3d &pose) {
    typedef Eigen::Matrix<double, 6, 1> Vector6d;
    const int iterations = 10;
    double cost = 0, lastCost = 0;
    double fx = K.at<double>(0, 0);
    double fy = K.at<double>(1, 1);
    double cx = K.at<double>(0, 2);
    double cy = K.at<double>(1, 2);

    for (int iter = 0; iter < iterations; iter++) {
        Eigen::Matrix<double, 6, 6> H = Eigen::Matrix<double, 6, 6>::Zero();
        Vector6d b = Vector6d::Zero();

        cost = 0;
        // compute cost
        for (int i = 0; i < points_3d.size(); i++) {
            Eigen::Vector3d pc = pose * points_3d[i];
            double inv_z = 1.0 / pc[2];
            double inv_z2 = inv_z * inv_z;
            Eigen::Vector2d proj(fx * pc[0] / pc[2] + cx, fy * pc[1] / pc[2] + cy);
            Eigen::Vector2d e = points_2d[i] - proj;
            cost += e.squaredNorm();
            Eigen::Matrix<double, 2, 6> J;
            J << -fx * inv_z,
            0,
            fx * pc[0] * inv_z2,
            fx * pc[0] * pc[1] * inv_z2,
            -fx - fx * pc[0] * pc[0] * inv_z2,
            fx * pc[1] * inv_z,
            0,
```

```
            -fy * inv_z,
            fy * pc[1] * inv_z,
            fy + fy * pc[1] * pc[1] * inv_z2,
            -fy * pc[0] * pc[1] * inv_z2,
            -fy * pc[0] * inv_z;

            H += J.transpose() * J;
            b += -J.transpose() * e;
        }

        Vector6d dx;
        dx = H.ldlt().solve(b);

        if (isnan(dx[0])) {
            cout << "result is nan!" << endl;
            break;
        }

        if (iter > 0 && cost >= lastCost) {
            // cost increase, update is not good
            cout << "cost: " << cost << ", last cost: " << lastCost << endl;
            break;
        }

        // update your estimation
        pose = Sophus::SE3d::exp(dx) * pose;
        lastCost = cost;

        cout << "iteration " << iter << " cost=" << cout.precision(12) << cost << endl;
        if (dx.norm() < 1e-6) {
            // converge
            break;
        }
    }

    cout << "pose by g-n: \n" << pose.matrix() << endl;
}
```

在这个小函数中，我们根据前面的理论推导，实现一个简单的高斯牛顿迭代优化。之后，我们将比较 OpenCV、手写实现和 g2o 实现之间的效率差异。

7.8.3　使用 g2o 进行 BA 优化

在手写了一遍优化流程之后，我们再来看如何用 g2o 实现同样的操作（事实上，用 Ceres 也完全类似）。g2o 的基本知识在第 6 讲中已经介绍过。在使用 g2o 之前，我们要把问题建模成一个图优化问题，如图 7-15 所示。

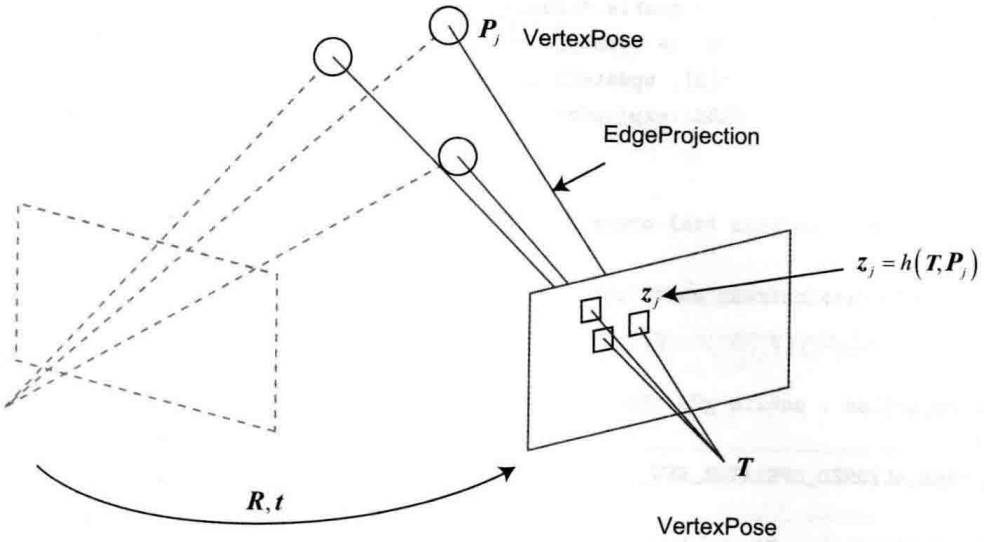


图 7-15　PnP 的 BA 的图优化表示

在这个图优化中，节点和边的选择如下：

1. **节点**：第二个相机的位姿节点 $\boldsymbol{T} \in \mathrm{SE}(3)$。
2. **边**：每个 3D 点在第二个相机中的投影，以观测方程来描述：

$$\boldsymbol{z}_j = h(\boldsymbol{T}, \boldsymbol{P}_j).$$

由于第一个相机位姿固定为零，所以我们没有把它写到优化变量里，但在更多的场合里，我们会考虑更多相机的估计。现在，我们根据一组 3D 点和第二个图像中的 2D 投影，估计第二个相机的位姿。所以我们把第一个相机画成虚线，表明不希望考虑它。

g2o 提供了许多关于 BA 的节点和边，例如 g2o/types/sba/types_six_dof_expmap.h 中提供了李代数表达的节点和边。在本书中，我们自己实现一个 VertexPose 顶点和 EdgeProjection 边，如下：

slambook2/ch7/pose_estimation_3d2d.cpp（片段）

```
/// vertex and edges used in g2o ba
class VertexPose : public g2o::BaseVertex<6, Sophus::SE3d> {
    public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW;
```

```
5
6       virtual void setToOriginImpl() override {
7           _estimate = Sophus::SE3d();
8       }
9
10      /// left multiplication on SE3
11      virtual void oplusImpl(const double *update) override {
12          Eigen::Matrix<double, 6, 1> update_eigen;
13          update_eigen << update[0], update[1], update[2], update[3], update[4], update[5];
14          _estimate = Sophus::SE3d::exp(update_eigen) * _estimate;
15      }
16
17      virtual bool read(istream &in) override {}
18
19      virtual bool write(ostream &out) const override {}
20  };
21
22  class EdgeProjection : public g2o::BaseUnaryEdge<2, Eigen::Vector2d, VertexPose> {
23      public:
24      EIGEN_MAKE_ALIGNED_OPERATOR_NEW;
25
26      EdgeProjection(const Eigen::Vector3d &pos, const Eigen::Matrix3d &K) : _pos3d(pos), _K(K)
        {}
27
28      virtual void computeError() override {
29          const VertexPose *v = static_cast<VertexPose *> (_vertices[0]);
30          Sophus::SE3d T = v->estimate();
31          Eigen::Vector3d pos_pixel = _K * (T * _pos3d);
32          pos_pixel /= pos_pixel[2];
33          _error = _measurement - pos_pixel.head<2>();
34      }
35
36      virtual void linearizeOplus() override {
37          const VertexPose *v = static_cast<VertexPose *> (_vertices[0]);
38          Sophus::SE3d T = v->estimate();
39          Eigen::Vector3d pos_cam = T * _pos3d;
40          double fx = _K(0, 0);
41          double fy = _K(1, 1);
42          double cx = _K(0, 2);
43          double cy = _K(1, 2);
44          double X = pos_cam[0];
45          double Y = pos_cam[1];
46          double Z = pos_cam[2];
```

```
        double Z2 = Z * Z;
        _jacobianOplusXi
        << -fx / Z, 0, fx * X / Z2, fx * X * Y / Z2, -fx - fx * X * X / Z2, fx * Y / Z,
        0, -fy / Z, fy * Y / (Z * Z), fy + fy * Y * Y / Z2, -fy * X * Y / Z2, -fy * X / Z;
    }

    virtual bool read(istream &in) override {}

    virtual bool write(ostream &out) const override {}

    private:
    Eigen::Vector3d _pos3d;
    Eigen::Matrix3d _K;
};
```

这里实现了顶点的更新和边的误差计算。下面将它们组成一个图优化问题：

slambook2/ch7/pose_estimation_3d2d.cpp（片段）

```
void bundleAdjustmentG2O(
const VecVector3d &points_3d,
const VecVector2d &points_2d,
const Mat &K,
Sophus::SE3d &pose) {
    // 构建图优化，先设定g2o
    typedef g2o::BlockSolver<g2o::BlockSolverTraits<6, 3>> BlockSolverType;  // pose is 6,
    landmark is 3
    typedef g2o::LinearSolverDense<BlockSolverType::PoseMatrixType> LinearSolverType; // 线性
    求解器类型
    // 梯度下降方法，可以从GN、LM、DogLeg中选
    auto solver = new g2o::OptimizationAlgorithmsGaussNewton(
        g2o::make_unique<BlockSolverType>(g2o::make_unique<LinearSolverType>()));
    g2o::SparseOptimizer optimizer;     // 图模型
    optimizer.setAlgorithm(solver);   // 设置求解器
    optimizer.setVerbose(true);       // 打开调试输出

    // vertex
    VertexPose *vertex_pose = new VertexPose(); // camera vertex_pose
    vertex_pose->setId(0);
    vertex_pose->setEstimate(Sophus::SE3d());
    optimizer.addVertex(vertex_pose);

    // K
    Eigen::Matrix3d K_eigen;
```

```
    K_eigen <<
    K.at<double>(0, 0), K.at<double>(0, 1), K.at<double>(0, 2),
    K.at<double>(1, 0), K.at<double>(1, 1), K.at<double>(1, 2),
    K.at<double>(2, 0), K.at<double>(2, 1), K.at<double>(2, 2);

    // edges
    int index = 1;
    for (size_t i = 0; i < points_2d.size(); ++i) {
        auto p2d = points_2d[i];
        auto p3d = points_3d[i];
        EdgeProjection *edge = new EdgeProjection(p3d, K_eigen);
        edge->setId(index);
        edge->setVertex(0, vertex_pose);
        edge->setMeasurement(p2d);
        edge->setInformation(Eigen::Matrix2d::Identity());
        optimizer.addEdge(edge);
        index++;
    }

    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    optimizer.setVerbose(true);
    optimizer.initializeOptimization();
    optimizer.optimize(10);
    chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
    chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2 -
     t1);
    cout << "optimization costs time: " << time_used.count() << " seconds." << endl;
    cout << "pose estimated by g2o =\n" << vertex_pose->estimate().matrix() << endl;
    pose = vertex_pose->estimate();
}
```

程序大体上和第 6 讲的 g2o 类似。我们首先声明了 g2o 图优化器，并配置优化求解器和梯度下降方法。然后，根据估计到的特征点，将位姿和空间点放到图中。最后，调用优化函数进行求解。运行的部分输出如下：

终端输入：

```
./build/pose_estimation_3d2d 1.png 2.png 1_depth.png 2_depth.png
-- Max dist : 95.000000
-- Min dist : 4.000000
一共找到了79组匹配点
3d-2d pairs: 76
solve pnp in opencv cost time: 0.000332991 seconds.
R=
```

```
[0.9978662025826269, -0.05167241613316376, 0.03991244360207524;
0.0505958915956335, 0.998339762771668, 0.02752769192381471;
-0.04126860182960625, -0.025449547736074, 0.998823919929363]
t=
[-0.1272259656955879;
-0.007507297652615337;
0.06138584177157709]
calling bundle adjustment by gauss newton
iteration 0 cost=645538.1857253
iteration 1 cost=12750.239874896
iteration 2 cost=12301.774589343
iteration 3 cost=12301.427574651
iteration 4 cost=12301.426806652
pose by g-n:
0.99786618832  -0.0516873580423    0.039893448423   -0.127218696289
0.0506143671126    0.998340854865   0.0274540224544 -0.00738695798083
-0.0412462852904  -0.0253762590968    0.998826706403   0.0617019263823
0                 0                 0                 1
solve pnp by gauss newton cost time: 0.000159492 seconds.
calling bundle adjustment by g2o
iteration= 0     chi2= 413.390599    time= 2.7291e-05    cumTime= 2.7291e-05     edges= 76
schur= 0    lambda= 79.000412   levenbergIter= 1
iteration= 1     chi2= 301.367030    time= 1.47e-05  cumTime= 4.1991e-05     edges= 76
schur= 0    lambda= 26.333471   levenbergIter= 1
iteration= 2     chi2= 301.365779    time= 1.7794e-05    cumTime= 5.9785e-05     edges= 76
schur= 0    lambda= 17.555647   levenbergIter= 1
iteration= 3     chi2= 301.365779    time= 1.4875e-05    cumTime= 7.466e-05  edges= 76
schur= 0    lambda= 11.703765   levenbergIter= 1
iteration= 4     chi2= 301.365779    time= 1.3132e-05    cumTime= 8.7792e-05     edges= 76
schur= 0    lambda= 7.802510    levenbergIter= 1
iteration= 5     chi2= 301.365779    time= 2.0379e-05    cumTime= 0.000108171    edges= 76
schur= 0    lambda= 41.613386   levenbergIter= 3
iteration= 6     chi2= 301.365779    time= 3.4186e-05    cumTime= 0.000142357    edges= 76
schur= 0    lambda= 2859650082279.672363    levenbergIter= 8
optimization costs time: 0.000763649 seconds.
pose estimated by g2o =
0.997866202583  -0.0516724161336   0.0399124436024   -0.127225965696
0.050595891596    0.998339762772   0.0275276919261 -0.00750729765631
-0.04126860183  -0.0254495477384    0.998823919929   0.0613858417711
0                 0                 0                 1
solve pnp by g2o cost time: 0.000923095 seconds.
```

从估计结果上看，三者基本一致。从优化时间来看，我们自己实现的高斯牛顿法以 0.15 毫秒

排在第一，其次是 OpenCV 的 PnP，最后是 g2o 的实现。尽管如此，三者的用时都在 1 毫秒以内，这说明位姿估计算法并不耗费计算量。

BA 是一种通用的做法。它可以不限于两幅图像。我们完全可以放入多幅图像匹配到的位姿和空间点进行迭代优化，甚至可以把整个 SLAM 过程放进来。那种做法规模较大，主要在后端使用，我们会在第 10 讲再次遇到这个问题。在前端，我们通常考虑局部相机位姿和特征点的小型 BA 问题，希望对它进行实时求解和优化。

7.9 3D-3D：ICP

最后，我们来介绍 3D-3D 的位姿估计问题。假设我们有一组配对好的 3D 点（例如我们对两幅 RGB-D 图像进行了匹配）：

$$\boldsymbol{P} = \{\boldsymbol{p}_1, \cdots, \boldsymbol{p}_n\}, \quad \boldsymbol{P}' = \{\boldsymbol{p}'_1, \cdots, \boldsymbol{p}'_n\},$$

现在，想要找一个欧氏变换 $\boldsymbol{R}, \boldsymbol{t}$，使得[①]

$$\forall i, \boldsymbol{p}_i = \boldsymbol{R}\boldsymbol{p}'_i + \boldsymbol{t}.$$

这个问题可以用迭代最近点（Iterative Closest Point，ICP）求解。读者应该注意到了，3D-3D 位姿估计问题中并没有出现相机模型，也就是说，仅考虑两组 3D 点之间的变换时，和相机并没有关系。因此，在激光 SLAM 中也会碰到 ICP，不过由于激光数据特征不够丰富，我们无从知道两个点集之间的**匹配关系**，只能认为距离最近的两个点为同一个，所以这个方法称为迭代最近点。而在视觉中，特征点为我们提供了较好的匹配关系，所以整个问题就变得更简单了。在 RGB-D SLAM 中，可以用这种方式估计相机位姿。下文我们用 ICP 指代**匹配好的**两组点间的运动估计问题。

和 PnP 类似，ICP 的求解也分为两种方式：利用线性代数的求解（主要是 SVD），以及利用非线性优化方式的求解（类似于 BA）。下面分别进行介绍。

7.9.1 SVD 方法

首先来看以 SVD 为代表的代数方法。根据前面描述的 ICP 问题，我们先定义第 i 对点的误差项：

$$\boldsymbol{e}_i = \boldsymbol{p}_i - (\boldsymbol{R}\boldsymbol{p}'_i + \boldsymbol{t}). \tag{7.49}$$

---

①这个例子和前两节的符号稍有不同。如果把它们关联起来，那么把 $\boldsymbol{p}_i$ 看成第二个图像中的数据，把 $\boldsymbol{p}'_i$ 看成第一个图像中的数据，得到的 $\boldsymbol{R}, \boldsymbol{t}$ 是一致的。

然后，构建最小二乘问题，求使误差平方和达到极小的 $\boldsymbol{R}, \boldsymbol{t}$：

$$\min_{\boldsymbol{R},\boldsymbol{t}} \frac{1}{2}\sum_{i=1}^{n} \left\| (\boldsymbol{p}_i - (\boldsymbol{R}\boldsymbol{p}_i' + \boldsymbol{t})) \right\|_2^2. \tag{7.50}$$

下面来推导它的求解方法。首先，定义两组点的质心：

$$\boldsymbol{p} = \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{p}_i), \quad \boldsymbol{p}' = \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{p}_i'). \tag{7.51}$$

请注意，质心是没有下标的。随后，在误差函数中做如下处理：

$$\begin{aligned}
\frac{1}{2}\sum_{i=1}^{n} \left\| \boldsymbol{p}_i - (\boldsymbol{R}\boldsymbol{p}_i' + \boldsymbol{t}) \right\|^2 &= \frac{1}{2}\sum_{i=1}^{n} \left\| \boldsymbol{p}_i - \boldsymbol{R}\boldsymbol{p}_i' - \boldsymbol{t} - \boldsymbol{p} + \boldsymbol{R}\boldsymbol{p}' + \boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' \right\|^2 \\
&= \frac{1}{2}\sum_{i=1}^{n} \left\| (\boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i' - \boldsymbol{p}')) + (\boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' - \boldsymbol{t}) \right\|^2 \\
&= \frac{1}{2}\sum_{i=1}^{n} (\left\| \boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i' - \boldsymbol{p}') \right\|^2 + \left\| \boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' - \boldsymbol{t} \right\|^2 + \\
&\quad 2(\boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i' - \boldsymbol{p}'))^{\mathrm{T}} (\boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' - \boldsymbol{t})).
\end{aligned}$$

注意到交叉项部分中 $(\boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i' - \boldsymbol{p}'))$ 在求和之后为零，因此优化目标函数可以简化为

$$\min_{\boldsymbol{R},\boldsymbol{t}} J = \frac{1}{2}\sum_{i=1}^{n} \left\| \boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i' - \boldsymbol{p}') \right\|^2 + \left\| \boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' - \boldsymbol{t} \right\|^2. \tag{7.52}$$

仔细观察左右两项，我们发现左边只和旋转矩阵 $\boldsymbol{R}$ 相关，而右边既有 $\boldsymbol{R}$ 也有 $\boldsymbol{t}$，但只和质心相关。只要我们获得了 $\boldsymbol{R}$，令第二项为零就能得到 $\boldsymbol{t}$。于是，ICP 可以分为以下三个步骤求解：

1. 计算两组点的质心位置 $\boldsymbol{p}, \boldsymbol{p}'$，然后计算每个点的**去质心坐标**：

$$\boldsymbol{q}_i = \boldsymbol{p}_i - \boldsymbol{p}, \quad \boldsymbol{q}_i' = \boldsymbol{p}_i' - \boldsymbol{p}'.$$

2. 根据以下优化问题计算旋转矩阵：

$$\boldsymbol{R}^* = \arg\min_{\boldsymbol{R}} \frac{1}{2}\sum_{i=1}^{n} \left\| \boldsymbol{q}_i - \boldsymbol{R}\boldsymbol{q}_i' \right\|^2. \tag{7.53}$$

3. 根据第 2 步的 $\boldsymbol{R}$ 计算 $\boldsymbol{t}$：

$$\boldsymbol{t}^* = \boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}'. \tag{7.54}$$

我们看到，只要求出了两组点之间的旋转，平移量是非常容易得到的。所以我们重点关注 $\boldsymbol{R}$ 的计算。展开关于 $\boldsymbol{R}$ 的误差项，得

$$\frac{1}{2}\sum_{i=1}^{n}\left\|\boldsymbol{q}_i - \boldsymbol{R}\boldsymbol{q}_i'\right\|^2 = \frac{1}{2}\sum_{i=1}^{n}\left(\boldsymbol{q}_i^{\mathrm{T}}\boldsymbol{q}_i + \boldsymbol{q}_i'^{\mathrm{T}}\boldsymbol{R}^{\mathrm{T}}\boldsymbol{R}\boldsymbol{q}_i' - 2\boldsymbol{q}_i^{\mathrm{T}}\boldsymbol{R}\boldsymbol{q}_i'\right). \tag{7.55}$$

注意到第一项和 $\boldsymbol{R}$ 无关，第二项由于 $\boldsymbol{R}^{\mathrm{T}}\boldsymbol{R} = \boldsymbol{I}$，亦与 $\boldsymbol{R}$ 无关。因此，实际上优化目标函数变为

$$\sum_{i=1}^{n} -\boldsymbol{q}_i^{\mathrm{T}}\boldsymbol{R}\boldsymbol{q}_i' = \sum_{i=1}^{n} -\mathrm{tr}\left(\boldsymbol{R}\boldsymbol{q}_i'\boldsymbol{q}_i^{\mathrm{T}}\right) = -\mathrm{tr}\left(\boldsymbol{R}\sum_{i=1}^{n}\boldsymbol{q}_i'\boldsymbol{q}_i^{\mathrm{T}}\right). \tag{7.56}$$

接下来，我们介绍怎样通过 SVD 解出上述问题中最优的 $\boldsymbol{R}$。关于最优性的证明较为复杂，感兴趣的读者请阅读参考文献 [62, 63]。为了解 $\boldsymbol{R}$，先定义矩阵：

$$\boldsymbol{W} = \sum_{i=1}^{n}\boldsymbol{q}_i\boldsymbol{q}_i'^{\mathrm{T}}. \tag{7.57}$$

$\boldsymbol{W}$ 是一个 3×3 的矩阵，对 $\boldsymbol{W}$ 进行 SVD 分解，得

$$\boldsymbol{W} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}. \tag{7.58}$$

其中，$\boldsymbol{\Sigma}$ 为奇异值组成的对角矩阵，对角线元素从大到小排列，而 $\boldsymbol{U}$ 和 $\boldsymbol{V}$ 为对角矩阵。当 $\boldsymbol{W}$ 满秩时，$\boldsymbol{R}$ 为

$$\boldsymbol{R} = \boldsymbol{U}\boldsymbol{V}^{\mathrm{T}}. \tag{7.59}$$

解得 $\boldsymbol{R}$ 后，按式 (7.54) 求解 $\boldsymbol{t}$ 即可。如果此时 $\boldsymbol{R}$ 的行列式为负，则取 $-\boldsymbol{R}$ 作为最优值。

7.9.2 非线性优化方法

求解 ICP 的另一种方式是使用非线性优化，以迭代的方式去找最优值。该方法和我们前面讲述的 PnP 非常相似。以李代数表达位姿时，目标函数可以写成

$$\min_{\boldsymbol{\xi}} = \frac{1}{2}\sum_{i=1}^{n}\left\|\left(\boldsymbol{p}_i - \exp\left(\boldsymbol{\xi}^{\wedge}\right)\boldsymbol{p}_i'\right)\right\|_2^2. \tag{7.60}$$

单个误差项关于位姿的导数在前面已推导，使用李代数扰动模型即可：

$$\frac{\partial e}{\partial \delta \boldsymbol{\xi}} = -(\exp(\boldsymbol{\xi}^\wedge)\boldsymbol{p}_i')^\odot. \tag{7.61}$$

于是，在非线性优化中只需不断迭代，就能找到极小值。而且，可以证明[6]，ICP 问题存在唯一解或无穷多解的情况。在唯一解的情况下，只要能找到极小值解，**这个极小值就是全局最优值**——因此不会遇到局部极小而非全局最小的情况。这也意味着 ICP 求解可以任意选定初始值。这是已匹配点时求解 ICP 的一大好处。

需要说明的是，我们这里讲的 ICP 是指已由图像特征给定了匹配的情况下进行位姿估计的问题。在匹配已知的情况下，这个最小二乘问题实际上具有解析解[64–66]，所以并没有必要进行迭代优化。ICP 的研究者们往往更加关心匹配未知的情况。那么，为什么我们要介绍基于优化的 ICP 呢？这是因为，某些场合下，例如在 RGB-D SLAM 中，一个像素的深度数据可能有，也可能测量不到，所以我们可以混合着使用 PnP 和 ICP 优化：对于深度已知的特征点，建模它们的 3D–3D 误差；对于深度未知的特征点，则建模 3D–2D 的重投影误差。于是，可以将所有的误差放在同一个问题中考虑，使得求解更加方便。

7.10　实践：求解 ICP

7.10.1　实践：SVD 方法

下面演示如何使用 SVD 及非线性优化来求解 ICP。本节我们使用两幅 RGB-D 图像，通过特征匹配获取两组 3D 点，最后用 ICP 计算它们的位姿变换。由于 OpenCV 目前还没有计算两组带匹配点的 ICP 的方法，而且它的原理也并不复杂，所以我们自己来实现一个 ICP。

slambook2/ch7/pose_estimation_3d3d.cpp（片段）

```
void pose_estimation_3d3d(
const vector<Point3f> &pts1,
const vector<Point3f> &pts2,
Mat &R, Mat &t) {
    Point3f p1, p2;     // center of mass
    int N = pts1.size();
    for (int i = 0; i < N; i++) {
        p1 += pts1[i];
        p2 += pts2[i];
    }
    p1 = Point3f(Vec3f(p1) / N);
    p2 = Point3f(Vec3f(p2) / N);
    vector<Point3f> q1(N), q2(N); // remove the center
```

```
    for (int i = 0; i < N; i++) {
        q1[i] = pts1[i] - p1;
        q2[i] = pts2[i] - p2;
    }

    // compute q1*q2^T
    Eigen::Matrix3d W = Eigen::Matrix3d::Zero();
    for (int i = 0; i < N; i++) {
        W += Eigen::Vector3d(q1[i].x, q1[i].y, q1[i].z) * Eigen::Vector3d(q2[i].x, q2[i].y,
        q2[i].z).transpose();
    }
    cout << "W=" << W << endl;

    // SVD on W
    Eigen::JacobiSVD<Eigen::Matrix3d> svd(W, Eigen::ComputeFullU | Eigen::ComputeFullV);
    Eigen::Matrix3d U = svd.matrixU();
    Eigen::Matrix3d V = svd.matrixV();

    cout << "U=" << U << endl;
    cout << "V=" << V << endl;

    Eigen::Matrix3d R_ = U * (V.transpose());
    if (R_.determinant() < 0) {
        R_ = -R_;
    }
    Eigen::Vector3d t_ = Eigen::Vector3d(p1.x, p1.y, p1.z) - R_ * Eigen::Vector3d(p2.x, p2.y,
     p2.z);

    // convert to cv::Mat
    R = (Mat_<double>(3, 3) <<
        R_(0, 0), R_(0, 1), R_(0, 2),
        R_(1, 0), R_(1, 1), R_(1, 2),
        R_(2, 0), R_(2, 1), R_(2, 2)
    );
    t = (Mat_<double>(3, 1) << t_(0, 0), t_(1, 0), t_(2, 0));
}
```

ICP 的实现方式和前文讲述的是一致的。我们调用 Eigen 进行 SVD，然后计算 $\boldsymbol{R},\boldsymbol{t}$ 矩阵。我们输出了匹配后的结果，不过请注意，由于前面的推导是按照 $\boldsymbol{p}_i = \boldsymbol{R}\boldsymbol{p}'_i + \boldsymbol{t}$ 进行的，这里的 $\boldsymbol{R},\boldsymbol{t}$ 是第二帧到第一帧的变换，与前面 PnP 部分相反。在输出结果中，我们同时打印了逆变换：

终端输入：

```
./build/pose_estimation_3d3d 1.png 2.png 1_depth.png 2_depth.png
```

```
-- Max dist : 95.000000
-- Min dist : 4.000000
一共找到了79组匹配点
3d-3d pairs: 74
W=  11.9404 -0.567258   1.64182
-1.79283   4.31299  -6.57615
3.12791  -6.55815   10.8576
U=  0.474144  -0.880373 -0.0114952
-0.460275  -0.258979   0.849163
0.750556   0.397334   0.528006
V=  0.535211  -0.844064 -0.0332488
-0.434767  -0.309001    0.84587
0.724242   0.438263   0.532352
ICP via SVD results:
R = [0.9972395977366739, 0.05617039856770099, -0.04855997354553433;
-0.05598345194682017, 0.9984181427731508, 0.005202431117423125;
0.0487753812298326, -0.002469515369266572, 0.9988067198811421]
t = [0.1417248739257469;
-0.05551033302525193;
-0.03119093188273858]
R_inv = [0.9972395977366739, -0.05598345194682017, 0.0487753812298326;
0.05617039856770099, 0.9984181427731508, -0.002469515369266572;
-0.04855997354553433, 0.005202431117423125, 0.9988067198811421]
t_inv = [-0.1429199667309695;
0.04738475446275858;
0.03832465717628181]
```

读者可以比较 ICP 与 PnP、对极几何的运动估计结果之间的差异。可以认为，在这个过程中我们使用了越来越多的信息（没有深度—有一个图的深度—有两个图的深度），因此，在深度准确的情况下，得到的估计也将越来越准确。但是，由于 Kinect 的深度图存在噪声，而且有可能存在数据丢失的情况，所以我们不得不丢弃一些没有深度数据的特征点。这可能导致 ICP 的估计不够准确，并且，如果特征点丢弃得太多，可能引起由于特征点太少，无法进行运动估计的情况。

7.10.2　实践：非线性优化方法

下面用非线性优化来计算 ICP。我们依然使用李代数来优化相机位姿。对我们来说，RGB-D 相机每次可以观测到路标点的三维位置，从而产生一个 3D 观测数据。我们使用上一个实验中的 VertexPose，然后定义 3D-3D 的一元边：

slambook2/ch7/pose_estimation_3d3d.cpp

```

/// g2o edge
```

```
class EdgeProjectXYZRGBDPoseOnly : public g2o::BaseUnaryEdge<3, Eigen::Vector3d, VertexPose>
{
    public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW;

    EdgeProjectXYZRGBDPoseOnly(const Eigen::Vector3d &point) : _point(point) {}

    virtual void computeError() override {
        const VertexPose *pose = static_cast<const VertexPose *> ( _vertices[0] );
        _error = _measurement - pose->estimate() * _point;
    }

    virtual void linearizeOplus() override {
        VertexPose *pose = static_cast<VertexPose *>(_vertices[0]);
        Sophus::SE3d T = pose->estimate();
        Eigen::Vector3d xyz_trans = T * _point;
        _jacobianOplusXi.block<3, 3>(0, 0) = -Eigen::Matrix3d::Identity();
        _jacobianOplusXi.block<3, 3>(0, 3) = Sophus::SO3d::hat(xyz_trans);
    }

    bool read(istream &in) {}

    bool write(ostream &out) const {}

    protected:
    Eigen::Vector3d _point;
};
```

这是一个一元边，写法类似于前面提到的 g2o::EdgeSE3ProjectXYZ，不过观测量从 2 维变成了 3 维，内部没有相机模型，并且只关联到一个节点。请读者注意这里雅可比矩阵的书写，它必须与我们前面的推导一致。雅可比矩阵给出了关于相机位姿的导数，是一个 3×6 的矩阵。

调用 g2o 进行优化的代码是相似的，我们设定好图优化的节点和边即可。这部分代码请读者查看源文件，这里不再列出。现在，来看看优化的结果：

终端输出：

```
iteration= 0     chi2= 1.811539  time= 1.7046e-05    cumTime= 1.7046e-05     edges= 74
schur= 0
iteration= 1     chi2= 1.811051  time= 1.0422e-05    cumTime= 2.7468e-05     edges= 74
schur= 0
iteration= 2     chi2= 1.811050  time= 9.589e-06     cumTime= 3.7057e-05     edges= 74
schur= 0
...中间略
```

```
iteration= 9     chi2= 1.811050  time= 9.113e-06     cumTime= 0.000100604    edges= 74
schur= 0
optimization costs time: 0.000559208 seconds.

after optimization:
T=
0.99724  0.0561704   -0.04856   0.141725
-0.0559834   0.998418 0.00520242 -0.0555103
0.0487754 -0.0024695   0.998807 -0.0311913
0          0          0          1
```

我们发现，只迭代一次后总体误差就已经稳定不变，说明仅在一次迭代之后算法已收敛。从位姿求解的结果可以看出，它和前面 SVD 给出的位姿结果几乎一模一样，这说明 SVD 已经给出了优化问题的解析解。所以，本实验中可以认为 SVD 给出的结果是相机位姿的最优值。

需要说明的是，在本例的 ICP 中，我们使用了在两个图都有深度读数的特征点。然而事实上，只要其中一个图深度确定，我们就能用类似于 PnP 的误差方式，把它们也加到优化中来。同时，除了相机位姿，将空间点也作为优化变量考虑，也是一种解决问题的方式。我们应当清楚，实际的求解是非常灵活的，不必拘泥于某种固定的形式。如果同时考虑点和相机，整个问题就变得**更自由**，你可能会得到其他的解。例如，可以让相机少转一些角度，而把点多移动一些。这从另一个侧面反映出，在 BA 里，我们会希望有尽可能多的约束，因为多次观测会带来更多的信息，使我们能够更准确地估计每个变量。

7.11　小结

本讲介绍了基于特征点的视觉里程计中的几个重要的问题。包括：

1. 特征点是如何提取并匹配的。
2. 如何通过 2D-2D 的特征点估计相机运动。
3. 如何从 2D-2D 的匹配估计一个点的空间位置。
4. 3D-2D 的 PnP 问题，其线性解法和 BA 解法。
5. 3D-3D 的 ICP 问题，其线性解法和 BA 解法。

本讲内容较为丰富，且结合应用了前几讲的基本知识。读者若觉得理解有困难，可以对前面的知识稍加回顾。最好亲自做一遍实验，以理解整个运动估计的内容。

需要解释的是，为保证行文流畅，我们省略了大量关于某些特殊情况的讨论。例如，如果在对极几何求解过程中给定的特征点共面，会发生什么情况（这在单应矩阵 $\boldsymbol{H}$ 中提到了）？共线又会发生什么情况？在 PnP 和 ICP 中若给定这样的解，又会导致什么情况？求解算法能否识别这些特殊的情况，并报告所得的解可能不可靠？能否给出估计的 $\boldsymbol{T}$ 的不确定度？它们都是值得研

究和探索的，对它们的讨论更适合留到具体的论文中。本书的目标是覆盖广泛的知识面和基础知识，我们对这些问题暂不展开，在工程实现中，这些情况也甚少出现。如果你关心这些少见的情况，可以阅读参考文献 [3] 等论文。

习题

1. 除了本书介绍的 ORB 特征点，你还能找到哪些特征点？请说说 SIFT 或 SURF 的原理，并对比它们与 ORB 之间的优劣。
2. 设计程序调用 OpenCV 中的其他种类特征点。统计在提取 1000 个特征点时在你的机器上所用的时间。
3.* 我们发现，OpenCV 提供的 ORB 特征点在图像中分布不够均匀。你是否能够找到或提出让特征点分布更均匀的方法？
4. 研究 FLANN 为何能够快速处理匹配问题。除了 FLANN，还有哪些可以加速匹配的手段？
5. 把演示程序使用的 EPnP 改成其他 PnP 方法，并研究它们的工作原理。
6. 在 PnP 优化中，将第一个相机的观测也考虑进来，程序应如何书写？最后结果会有何变化？
7. 在 ICP 程序中，将空间点也作为优化变量考虑进来，程序应如何书写？最后结果会有何变化？
8.* 在特征点匹配过程中，不可避免地会遇到误匹配的情况。如果我们把错误匹配输入到 PnP 或 ICP 中，会发生怎样的情况？你能想到哪些避免误匹配的方法？
9.* 使用 Sophus 的 SE3 类，自己设计 g2o 的节点与边，实现 PnP 和 ICP 的优化。
10.* 在 Ceres 中实现 PnP 和 ICP 的优化。

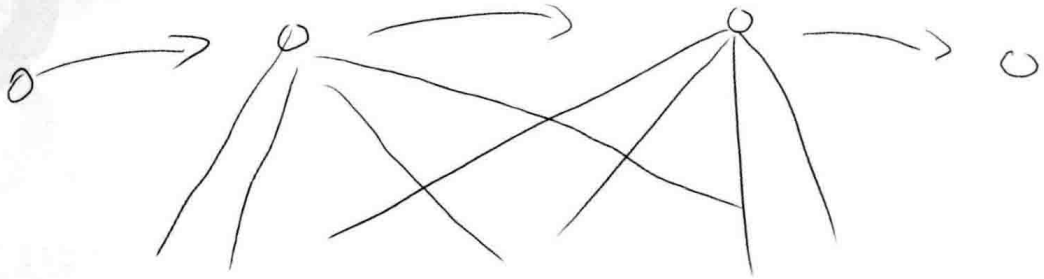
第 8 讲

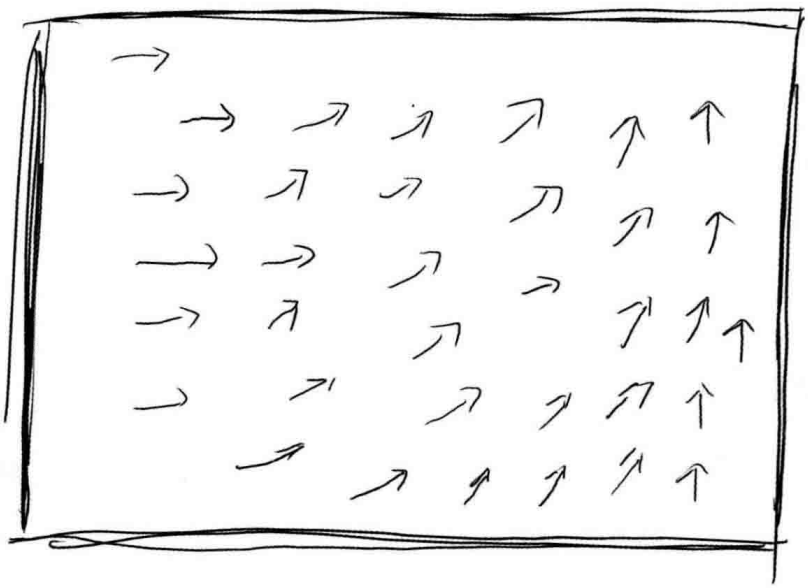
视觉里程计 2

主要目标

1. 理解光流法跟踪特征点的原理。
2. 理解直接法是如何估计相机位姿的。
3. 实现多层直接法的计算。

直接法是视觉里程计的另一个主要分支，它与特征点法有很大不同。虽然它还没有成为现在视觉里程计中的主流，但经过近几年的发展，直接法在一定程度上已经能和特征点法平分秋色。本讲，我们将介绍直接法的原理，并实现直接法中的核心部分。





$$\min \left\| I_1(p) - I_2\left(K(Rp+t)\right) \right\|_2$$

$$J = \frac{\partial I}{\partial u} \frac{\partial u}{\partial \xi} \frac{\partial \xi}{\partial \xi}$$

8.1　直接法的引出

第 7 讲我们介绍了使用特征点估计相机运动的方法。尽管特征点法在视觉里程计中占据主流地位，但研究者们还是认识到它至少有以下几个缺点：

1. 关键点的提取与描述子的计算非常耗时。实践中，SIFT 目前在 CPU 上是无法实时计算的，而 ORB 也需要近 20 毫秒的计算时长。如果整个 SLAM 以 30 毫秒/帧的速度运行，那么一大半时间都将花在计算特征点上。
2. 使用特征点时，忽略了除特征点以外的所有信息。一幅图像有几十万个像素，而特征点只有几百个。只使用特征点丢弃了大部分**可能有用的**图像信息。
3. 相机有时会运动到**特征缺失**的地方，这些地方往往没有明显的纹理信息。例如，有时我们会面对一堵白墙，或者一个空荡荡的走廊。这些场景下特征点数量会明显减少，我们可能找不到足够的匹配点来计算相机运动。

我们看到使用特征点确实存在一些问题。有没有办法能够克服这些缺点呢？我们有以下几种思路：

- 保留特征点，但只计算关键点，不计算描述子。同时，使用**光流法**（Optical Flow）跟踪特征点的运动。这样可以回避计算和匹配描述子带来的时间，而光流本身的计算时间要小于描述子的计算与匹配。
- 只计算关键点，不计算描述子。同时，使用**直接法**（Direct Method）计算特征点在下一时刻图像中的位置。这同样可以跳过描述子的计算过程，也省去了光流的计算时间。

第一种方法仍然使用特征点，只是把匹配描述子替换成了光流跟踪，估计相机运动时仍使用对极几何、PnP 或 ICP 算法。这依然会要求提取到的关键点具有可区别性，即我们需要提到角点。而在直接法中，我们会根据图像的**像素灰度信息**同时估计相机运动和点的投影，不要求提取到的点必须为角点。在后文中将看到，它们甚至可以是随机的选点。

使用特征点法估计相机运动时，我们把特征点看作固定在三维空间的不动点。根据它们在相机中的投影位置，通过最小化**重投影误差**（Reprojection error）优化相机运动。在这个过程中，我们需要精确地知道空间点在两个相机中投影后的像素位置——这也就是我们要对特征进行匹配或跟踪的原因。同时，我们也知道，计算、匹配特征需要付出大量的计算量。相对地，在直接法中，我们并不需要知道点与点之间的对应关系，而是通过最小化**光度误差**（Photometric error）来求得它们。

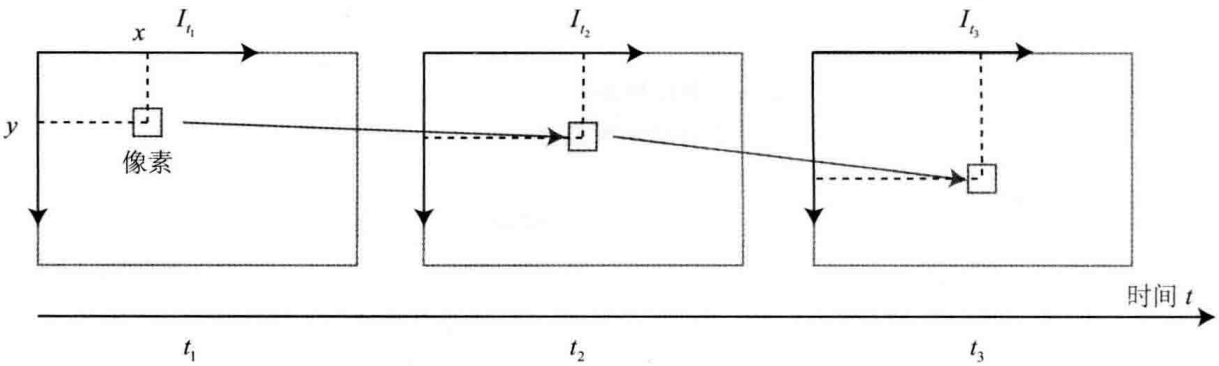
直接法是本讲介绍的重点。它是为了克服特征点法的上述缺点而存在的。直接法根据像素的亮度信息估计相机的运动，可以完全不用计算关键点和描述子，于是，既避免了特征的计算时间，也避免了特征缺失的情况。只要场景中存在明暗变化（可以是渐变，不形成局部的图像梯度），直接法就能工作。根据使用像素的数量，直接法分为稀疏、稠密和半稠密三种。与特征点法只能重构稀疏特征点（稀疏地图）相比，直接法还具有恢复稠密或半稠密结构的能力。

历史上，早期也有对直接法的使用[67]。随着一些使用直接法的开源项目的出现（如 SVO[68]、LSD-SLAM[69]、DSO[70] 等），它们逐渐走上主流舞台，成为视觉里程计算法中重要的一部分。

8.2 2D 光流

直接法是从光流演变而来的。它们非常相似，具有相同的假设条件。光流描述了像素在图像中的运动，而直接法则附带着一个相机运动模型。为了说明直接法，我们先来介绍光流。

光流是一种描述像素随时间在图像之间运动的方法，如图 8-1 所示。随着时间的流逝，同一个像素会在图像中运动，而我们希望追踪它的运动过程。其中，计算部分像素运动的称为**稀疏光流**，计算所有像素的称为**稠密光流**。稀疏光流以 Lucas-Kanade 光流[71] 为代表，并可以在 SLAM 中用于跟踪特征点位置。稠密光流以 Horn-Schunck 光流[72] 为代表。因此，本节主要介绍 Lucas-Kanade 光流，也称为 LK 光流。



灰度不变假设：$I(x_1,y_1,t_1)=I(x_2,y_2,t_2)=I(x_3,y_3,t_3)$

图 8-1 LK 光流法示意图

Lucas-Kanade 光流

在 LK 光流中，我们认为来自相机的图像是随时间变化的。图像可以看作时间的函数：$\boldsymbol{I}(t)$。那么，一个在 t 时刻，位于 (x,y) 处的像素，它的灰度可以写成

$$\boldsymbol{I}(x,y,t).$$

这种方式把图像看成了关于位置与时间的函数，它的值域就是图像中像素的灰度。现在考虑某个固定的空间点，它在 t 时刻的像素坐标为 x,y。由于相机的运动，它的图像坐标将发生变化。我们希望估计这个空间点在其他时刻图像中的位置。怎么估计呢？这里要引入光流法的基本假设。

灰度不变假设：同一个空间点的像素灰度值，在各个图像中是固定不变的。

对于 t 时刻位于 (x, y) 处的像素，我们设 $t + \mathrm{d}t$ 时刻它运动到 $(x + \mathrm{d}x, y + \mathrm{d}y)$ 处。由于灰度不变，我们有

$$\boldsymbol{I}(x + \mathrm{d}x, y + \mathrm{d}y, t + \mathrm{d}t) = \boldsymbol{I}(x, y, t). \tag{8.1}$$

注意灰度不变假设是一个很强的假设，实际中很可能不成立。事实上，由于物体的材质不同，像素会出现高光和阴影部分；有时，相机会自动调整曝光参数，使得图像整体变亮或变暗。这时灰度不变假设都是不成立的，因此光流的结果也不一定可靠。然而，从另一方面来说，所有算法都是在一定假设下工作的。如果我们什么假设都不做，就没法设计实用的算法。所以，让我们暂且认为该假设成立，看看如何计算像素的运动。

对左边进行泰勒展开，保留一阶项，得

$$\boldsymbol{I}(x + \mathrm{d}x, y + \mathrm{d}y, t + \mathrm{d}t) \approx \boldsymbol{I}(x, y, t) + \frac{\partial \boldsymbol{I}}{\partial x}\mathrm{d}x + \frac{\partial \boldsymbol{I}}{\partial y}\mathrm{d}y + \frac{\partial \boldsymbol{I}}{\partial t}\mathrm{d}t. \tag{8.2}$$

因为我们假设了灰度不变，于是下一个时刻的灰度等于之前的灰度，从而：

$$\frac{\partial \boldsymbol{I}}{\partial x}\mathrm{d}x + \frac{\partial \boldsymbol{I}}{\partial y}\mathrm{d}y + \frac{\partial \boldsymbol{I}}{\partial t}\mathrm{d}t = 0. \tag{8.3}$$

两边除以 $\mathrm{d}t$，得

$$\frac{\partial \boldsymbol{I}}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial \boldsymbol{I}}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t} = -\frac{\partial \boldsymbol{I}}{\partial t}. \tag{8.4}$$

其中 $\mathrm{d}x/\mathrm{d}t$ 为像素在 x 轴上的运动速度，而 $\mathrm{d}y/\mathrm{d}t$ 为 y 轴上的速度，把它们记为 u, v。同时，$\partial \boldsymbol{I}/\partial x$ 为图像在该点处 x 方向的梯度，另一项则是在 y 方向的梯度，记为 $\boldsymbol{I}_x, \boldsymbol{I}_y$。把图像灰度对时间的变化量记为 $\boldsymbol{I}_t$，写成矩阵形式，有

$$\begin{bmatrix} \boldsymbol{I}_x & \boldsymbol{I}_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = -\boldsymbol{I}_t. \tag{8.5}$$

我们想计算的是像素的运动 u, v，但是该式是带有两个变量的一次方程，仅凭它无法计算出 u, v。因此，必须引入额外的约束来计算 u, v。在 LK 光流中，我们假设**某一个窗口内的像素具有相同的运动**。

考虑一个大小为 $w \times w$ 的窗口，它含有 w^2 数量的像素。该窗口内像素具有同样的运动，因此我们共有 w^2 个方程：

$$\begin{bmatrix} \boldsymbol{I}_x & \boldsymbol{I}_y \end{bmatrix}_k \begin{bmatrix} u \\ v \end{bmatrix} = -\boldsymbol{I}_{tk}, \quad k = 1, \ldots, w^2. \tag{8.6}$$

记：

$$\boldsymbol{A} = \begin{bmatrix} [\boldsymbol{I}_x, \boldsymbol{I}_y]_1 \\ \vdots \\ [\boldsymbol{I}_x, \boldsymbol{I}_y]_k \end{bmatrix}, \boldsymbol{b} = \begin{bmatrix} \boldsymbol{I}_{t1} \\ \vdots \\ \boldsymbol{I}_{tk} \end{bmatrix}. \tag{8.7}$$

于是整个方程为

$$\boldsymbol{A} \begin{bmatrix} u \\ v \end{bmatrix} = -\boldsymbol{b}. \tag{8.8}$$

这是一个关于 u, v 的超定线性方程，传统解法是求最小二乘解。最小二乘经常被用到：

$$\begin{bmatrix} u \\ v \end{bmatrix}^* = -\left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{b}. \tag{8.9}$$

这样就得到了像素在图像间的运动速度 u, v。当 t 取离散的时刻而不是连续时间时，我们可以估计某块像素在若干个图像中出现的位置。由于像素梯度仅在局部有效，所以如果一次迭代不够好，我们会多迭代几次这个方程。在 SLAM 中，LK 光流常被用来跟踪角点的运动，我们不妨通过程序体会一下。

8.3 实践：LK 光流

8.3.1 使用 LK 光流

在实践部分，我们将使用几张示例图像，用 OpenCV 的光流来追踪上面的特征点。同时，我们也将手动实现一个 LK 光流，以达到加深理解的效果。我们使用两张来自 Euroc 数据集的示例图像，在第一张图像中提取角点，然后用光流追踪它们在第二张图像中的位置。首先，我们使用 OpenCV 中的 LK 光流：

slambook2/ch8/optical_flow.cpp（片段）

```
// use opencv's flow for validation
vector<Point2f> pt1, pt2;
for (auto &kp: kp1) pt1.push_back(kp.pt);
vector<uchar> status;
vector<float> error;
cv::calcOpticalFlowPyrLK(img1, img2, pt1, pt2, status, error);
```

OpenCV 的光流在使用上十分简单，只需调用 cv::calcOpticalFlowPyrLK 函数，提供前后两张图像及对应的特征点，即可得到追踪后的点，以及各点的状态、误差。我们可以根据 status 变量

是否为 1 来确定对应的点是否被正确追踪到。该函数还有一些可选的参数，但是在演示中我们只使用默认参数。我们在此省略其他提取特征、画出结果的代码，这些在之前的程序中已经展示过了。

8.3.2　用高斯牛顿法实现光流

单层光流

光流也可以看成一个优化问题：通过最小化灰度误差估计最优的像素偏移。所以，类似于之前实现的各种高斯牛顿法化，我们现在也来实现一个基于高斯牛顿法的光流。

slambook2/ch8/optical_flow.cpp（片段）

```
class OpticalFlowTracker {
public:
    OpticalFlowTracker(
        const Mat &img1_,
        const Mat &img2_,
        const vector<KeyPoint> &kp1_,
        vector<KeyPoint> &kp2_,
        vector<bool> &success_,
        bool inverse_ = true, bool has_initial_ = false) :
        img1(img1_), img2(img2_), kp1(kp1_), kp2(kp2_), success(success_), inverse(inverse_),
        has_initial(has_initial_) {}

    void calculateOpticalFlow(const Range &range);

private:
    const Mat &img1;
    const Mat &img2;
    const vector<KeyPoint> &kp1;
    vector<KeyPoint> &kp2;
    vector<bool> &success;
    bool inverse = true;
    bool has_initial = false;
};

void OpticalFlowSingleLevel(
    const Mat &img1,
    const Mat &img2,
    const vector<KeyPoint> &kp1,
    vector<KeyPoint> &kp2,
    vector<bool> &success,
```

```
    bool inverse, bool has_initial) {
    kp2.resize(kp1.size());
    success.resize(kp1.size());
    OpticalFlowTracker tracker(img1, img2, kp1, kp2, success, inverse, has_initial);
    parallel_for_(Range(0, kp1.size()),
        std::bind(&OpticalFlowTracker::calculateOpticalFlow, &tracker, placeholders::_1));
}

void OpticalFlowTracker::calculateOpticalFlow(const Range &range) {
    // parameters
    int half_patch_size = 4;
    int iterations = 10;
    for (size_t i = range.start; i < range.end; i++) {
        auto kp = kp1[i];
        double dx = 0, dy = 0; // dx,dy need to be estimated
        if (has_initial) {
            dx = kp2[i].pt.x - kp.pt.x;
            dy = kp2[i].pt.y - kp.pt.y;
        }

        double cost = 0, lastCost = 0;
        bool succ = true; // indicate if this point succeeded

        // Gauss-Newton iterations
        Eigen::Matrix2d H = Eigen::Matrix2d::Zero();    // hessian
        Eigen::Vector2d b = Eigen::Vector2d::Zero();    // bias
        Eigen::Vector2d J;  // jacobian
        for (int iter = 0; iter < iterations; iter++) {
            if (inverse == false) {
                H = Eigen::Matrix2d::Zero();
                b = Eigen::Vector2d::Zero();
            } else {
                // only reset b
                b = Eigen::Vector2d::Zero();
            }

            cost = 0;

            // compute cost and jacobian
            for (int x = -half_patch_size; x < half_patch_size; x++)
            for (int y = -half_patch_size; y < half_patch_size; y++) {
                double error = GetPixelValue(img1, kp.pt.x + x, kp.pt.y + y) -
                    GetPixelValue(img2, kp.pt.x + x + dx, kp.pt.y + y + dy);;  // Jacobian
```

```
            if (inverse == false) {
                J = -1.0 * Eigen::Vector2d(
                    0.5 * (GetPixelValue(img2, kp.pt.x + dx + x + 1, kp.pt.y + dy + y) -
                        GetPixelValue(img2, kp.pt.x + dx + x - 1, kp.pt.y + dy + y)),
                    0.5 * (GetPixelValue(img2, kp.pt.x + dx + x, kp.pt.y + dy + y + 1) -
                        GetPixelValue(img2, kp.pt.x + dx + x, kp.pt.y + dy + y - 1))
                );
            } else if (iter == 0) {
                // in inverse mode, J keeps same for all iterations
                // NOTE this J does not change when dx, dy is updated, so we can store it
                 and only compute error
                J = -1.0 * Eigen::Vector2d(
                    0.5 * (GetPixelValue(img1, kp.pt.x + x + 1, kp.pt.y + y) -
                        GetPixelValue(img1, kp.pt.x + x - 1, kp.pt.y + y)),
                    0.5 * (GetPixelValue(img1, kp.pt.x + x, kp.pt.y + y + 1) -
                        GetPixelValue(img1, kp.pt.x + x, kp.pt.y + y - 1))
                );
            }
            // compute H, b and set cost;
            b += -error * J;
            cost += error * error;
            if (inverse == false || iter == 0) {
                // also update H
                H += J * J.transpose();
            }
        }

        // compute update
        Eigen::Vector2d update = H.ldlt().solve(b);

        if (std::isnan(update[0])) {
            // sometimes occurred when we have a black or white patch and H is
            irreversible
            cout << "update is nan" << endl;
            succ = false;
            break;
        }

        if (iter > 0 && cost > lastCost) {
            break;
        }

        // update dx, dy
```

```
            dx += update[0];
            dy += update[1];
            lastCost = cost;
            succ = true;

            if (update.norm() < 1e-2) {
                // converge
                break;
            }
        }

        success[i] = succ;

        // set kp2
        kp2[i].pt = kp.pt + Point2f(dx, dy);
    }
}
```

我们在 OpticalFlowSingleLevel 函数中实现了单层光流函数，其中调用了 cv::parallel_for_ 并行调用 OpticalFlowTracker::calculateOpticalFlow，该函数计算指定范围内特征点的光流。这个并行 for 循环内部是 Intel tbb 库实现的，我们只需按照其接口，将函数本体定义出来，然后将函数作为 std::function 对象传递给它。

在具体函数实现中（即 calculateOpticalFlow），我们求解这样一个问题：

$$\min_{\Delta x,\Delta y} \|\boldsymbol{I}_1(x,y) - \boldsymbol{I}_2(x+\Delta x, y+\Delta y)\|_2^2. \tag{8.10}$$

因此，残差为括号内部的部分，对应的雅可比为第二个图像在 $x+\Delta x, y+\Delta y$ 处的梯度。此外，根据参考文献 [73]，这里的梯度也可以用第一个图像的梯度 $\boldsymbol{I}_1(x,y)$ 来代替。这种代替的方法称为**反向**（Inverse）光流法。在反向光流中，$\boldsymbol{I}_1(x,y)$ 的梯度是保持不变的，所以我们可以在第一次迭代时保留计算的结果，在后续迭代中使用。当雅可比不变时，$\boldsymbol{H}$ 矩阵不变，每次迭代只需计算残差，这可以节省一部分计算量。

多层光流

我们把光流写成了优化问题，就必须假设优化的初始值靠近最优值，才能在一定程度上保障算法的收敛。如果相机运动较快，两张图像差异较明显，那么单层图像光流法容易达到一个局部极小值。这种情况可以通过引入图像金字塔来改善。

图像金字塔是指对同一个图像进行缩放，得到不同分辨率下的图像，如图 8-2 所示。以原始图像作为金字塔底层，每往上一层，就对下层图像进行一定倍率的缩放，就得到了一个金字塔。

然后，在计算光流时，先从顶层的图像开始计算，然后把上一层的追踪结果，作为下一层光流的初始值。由于上层的图像相对粗糙，所以这个过程也称为**由粗至精**（Coarse-to-fine）的光流，也是实用光流法的通常流程。

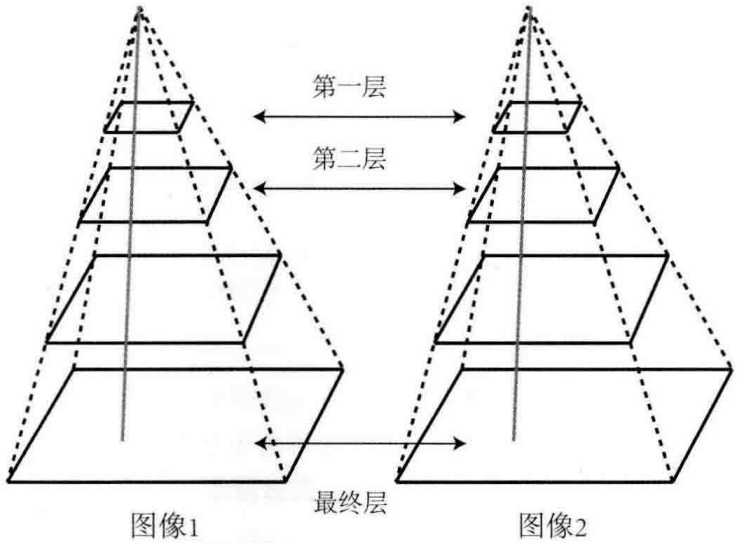


图 8-2　图像金字塔和光流由粗至精的过程

由粗至精的好处在于，当原始图像的像素运动较大时，在金字塔顶层的图像看来，运动仍然在一个很小范围内。例如，原始图像的特征点运动了 20 个像素，很容易由于图像非凸性导致优化困在极小值里。但现在假设有缩放倍率为 0.5 倍的金字塔，那么往上两层图像里，像素运动就只有 5 个像素了，这时结果就明显好于直接在原始图像上优化。

我们在程序中实现了多层光流，代码如下：

slambook2/ch8/optical_flow.cpp（片段）

```
void OpticalFlowMultiLevel(
    const Mat &img1,
    const Mat &img2,
    const vector<KeyPoint> &kp1,
    vector<KeyPoint> &kp2,
    vector<bool> &success,
    bool inverse) {

    // parameters
    int pyramids = 4;
    double pyramid_scale = 0.5;
    double scales[] = {1.0, 0.5, 0.25, 0.125};

```

```
    // create pyramids
    vector<Mat> pyr1, pyr2; // image pyramids
    for (int i = 0; i < pyramids; i++) {
        if (i == 0) {
            pyr1.push_back(img1);
            pyr2.push_back(img2);
        } else {
            Mat img1_pyr, img2_pyr;
            cv::resize(pyr1[i - 1], img1_pyr,
            cv::Size(pyr1[i - 1].cols * pyramid_scale, pyr1[i - 1].rows * pyramid_scale));
            cv::resize(pyr2[i - 1], img2_pyr,
            cv::Size(pyr2[i - 1].cols * pyramid_scale, pyr2[i - 1].rows * pyramid_scale));
            pyr1.push_back(img1_pyr);
            pyr2.push_back(img2_pyr);
        }
    }

    // coarse-to-fine LK tracking in pyramids
    vector<KeyPoint> kp1_pyr, kp2_pyr;
    for (auto &kp:kp1) {
        auto kp_top = kp;
        kp_top.pt *= scales[pyramids - 1];
        kp1_pyr.push_back(kp_top);
        kp2_pyr.push_back(kp_top);
    }

    for (int level = pyramids - 1; level >= 0; level--) {
        // from coarse to fine
        success.clear();
        OpticalFlowSingleLevel(pyr1[level], pyr2[level], kp1_pyr, kp2_pyr, success, inverse,
        true);

        if (level > 0) {
            for (auto &kp: kp1_pyr)
            kp.pt /= pyramid_scale;
            for (auto &kp: kp2_pyr)
            kp.pt /= pyramid_scale;
        }
    }

    for (auto &kp: kp2_pyr)
        kp2.push_back(kp);
}
```

这段代码构造了一个四层的、倍率为 0.5 的金字塔，并调用单层光流函数实现了多层光流。在主函数中，我们分别对两张图像测试了 OpenCV 的光流、单层光流和多层光流的表现，计算了它们的运行时间：

终端输入：

```
./build/optical_flow
build pyramid time: 0.000150349
track pyr 3 cost time: 0.000304633
track pyr 2 cost time: 0.000392889
track pyr 1 cost time: 0.000382347
track pyr 0 cost time: 0.000375099
optical flow by gauss-newton: 0.00189268
optical flow by opencv: 0.00220134
```

从运行时间上看，多层光流法的耗时和 OpenCV 的大致相当。由于并行化程序在每次运行时的表现不尽相同，在读者机器上这些数字不会精确相同。光流的对比图如图 8-3 所示。从结果图上看，多层光流与 OpenCV 的效果相当，单层光流要明显弱于多层光流。



单层光流



多层光流



OpenCV光流

图 8-3　各种光流的结果对比

8.3.3 光流实践小结

我们看到，LK 光流跟踪能够直接得到特征点的对应关系。这个对应关系就像是描述子的匹配，只是光流对图像的连续性和光照稳定性要求更高一些。我们可以通过光流跟踪的特征点，用 PnP、ICP 或对极几何来估计相机运动，这些方法在第 7 讲中介绍过，这里不再讨论。

从运行时间上来看，演示实验大约有 230 个特征点，OpenCV 和多层光流需要大约 2 毫秒完成追踪（笔者用的 CPU 是 Intel I7-8550U），这实际上是相当快的。如果我们前面使用 FAST 这样的关键点，那么整个光流计算可以做到 5 毫秒左右，相比于特征匹配来说算是非常快了。不过，如果角点提的位置不好，光流也容易跟丢或给出错误的结果，这就需要后续算法拥有一定的异常值去除机制，这部分内容我们留到工程章节再谈。

总而言之，光流法可以加速基于特征点的视觉里程计算法，避免计算和匹配描述子的过程，但要求相机运动较平滑（或采集频率较高）。

8.4 直接法

接下来，我们讨论与光流有一定相似性的直接法。与前面内容相似，我们先介绍直接法的原理，然后实现一遍直接法。

8.4.1 直接法的推导

在光流中，我们会首先追踪特征点的位置，再根据这些位置确定相机的运动。这样一种两步走的方案，很难保证全局的最优性。读者可以会问，能不能在后一步中，调整前一步的结果呢？例如，如果认为相机右转了 15°，那么光流能不能以这个 15° 运动作为初始值的假设，调整光流的计算结果呢？直接法就是遵循这样的思路得到的结果。

如图 8-4 所示，考虑某个空间点 P 和两个时刻的相机。P 的世界坐标为 $[X, Y, Z]$，它在两个相机上成像，记像素坐标为 $\boldsymbol{p}_1, \boldsymbol{p}_2$。

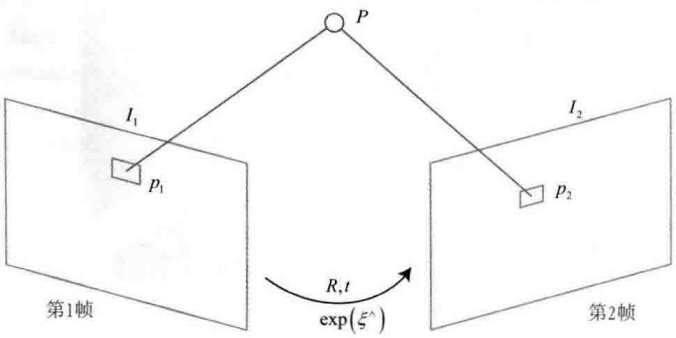


图 8-4 直接法示意图

我们的目标是求第一个相机到第二个相机的相对位姿变换。我们以第一个相机为参照系，设第二个相机的旋转和平移为 $\boldsymbol{R},\boldsymbol{t}$（对应李群为 $\boldsymbol{T}$）。同时，两相机的内参相同，记为 $\boldsymbol{K}$。为清楚起见，我们列写完整的投影方程：

$$\boldsymbol{p}_1 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_1 = \frac{1}{Z_1}\boldsymbol{K}\boldsymbol{P},$$

$$\boldsymbol{p}_2 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_2 = \frac{1}{Z_2}\boldsymbol{K}\left(\boldsymbol{R}\boldsymbol{P}+\boldsymbol{t}\right) = \frac{1}{Z_2}\boldsymbol{K}\left(\boldsymbol{T}\boldsymbol{P}\right)_{1:3}.$$

其中 Z_1 是 P 的深度，Z_2 是 P 在第二个相机坐标系下的深度，也就是 $\boldsymbol{R}\boldsymbol{P}+\boldsymbol{t}$ 的第 3 个坐标值。由于 $\boldsymbol{T}$ 只能和齐次坐标相乘，所以我们乘完之后要取出前 3 个元素。这和第 5 讲的内容是一致的。

回忆特征点法中，由于我们通过匹配描述子知道了 $\boldsymbol{p}_1,\boldsymbol{p}_2$ 的像素位置，所以可以计算重投影的位置。但在直接法中，由于没有特征匹配，我们无从知道哪一个 $\boldsymbol{p}_2$ 与 $\boldsymbol{p}_1$ 对应着同一个点。直接法的思路是根据当前相机的位姿估计值寻找 $\boldsymbol{p}_2$ 的位置。但若相机位姿不够好，$\boldsymbol{p}_2$ 的外观和 $\boldsymbol{p}_1$ 会有明显差别。于是，为了减小这个差别，我们优化相机的位姿，来寻找与 $\boldsymbol{p}_1$ 更相似的 $\boldsymbol{p}_2$。这同样可以通过解一个优化问题完成，但此时最小化的不是重投影误差，而是**光度误差**，也就是 P 的两个像素的亮度误差：

$$e = \boldsymbol{I}_1\left(\boldsymbol{p}_1\right) - \boldsymbol{I}_2\left(\boldsymbol{p}_2\right). \tag{8.11}$$

注意，这里的 e 是一个标量。同样地，优化目标为该误差的二范数，暂时取不加权的形式，为

$$\min_{\boldsymbol{T}} J\left(\boldsymbol{T}\right) = \|e\|^2. \tag{8.12}$$

能够做这种优化的理由，仍是基于**灰度不变假设**。我们假设一个空间点在各个视角下成像的灰度是不变的。我们有许多个（比如 N 个）空间点 P_i，那么，整个相机位姿估计问题变为

$$\min_{\boldsymbol{T}} J\left(\boldsymbol{T}\right) = \sum_{i=1}^{N} e_i^{\mathrm{T}} e_i, \quad e_i = \boldsymbol{I}_1\left(\boldsymbol{p}_{1,i}\right) - \boldsymbol{I}_2\left(\boldsymbol{p}_{2,i}\right). \tag{8.13}$$

注意，这里的优化变量是相机位姿 $\boldsymbol{T}$，而不像光流那样优化各个特征点的运动。为了求解这个优化问题，我们关心误差 e 是如何随着相机位姿 $\boldsymbol{T}$ 变化的，需要分析它们的导数关系。因此，定义两个中间变量：

$$\boldsymbol{q} = \boldsymbol{T}\boldsymbol{P},$$
$$\boldsymbol{u} = \frac{1}{Z_2}\boldsymbol{K}\boldsymbol{q}.$$

这里的 $\boldsymbol{q}$ 为 P 在第二个相机坐标系下的坐标，而 $\boldsymbol{u}$ 为它的像素坐标。显然 $\boldsymbol{q}$ 是 $\boldsymbol{T}$ 的函数，$\boldsymbol{u}$ 是 $\boldsymbol{q}$ 的函数，从而也是 $\boldsymbol{T}$ 的函数。考虑李代数的左扰动模型，利用一阶泰勒展开，因为：

$$e(\boldsymbol{T}) = \boldsymbol{I}_1(\boldsymbol{p}_1) - \boldsymbol{I}_2(\boldsymbol{u}), \tag{8.14}$$

所以：

$$\frac{\partial e}{\partial \boldsymbol{T}} = \frac{\partial \boldsymbol{I}_2}{\partial \boldsymbol{u}} \frac{\partial \boldsymbol{u}}{\partial \boldsymbol{q}} \frac{\partial \boldsymbol{q}}{\partial \delta\boldsymbol{\xi}} \delta\boldsymbol{\xi}, \tag{8.15}$$

其中 $\delta\boldsymbol{\xi}$ 为 $\boldsymbol{T}$ 的左扰动。我们看到，一阶导数由于链式法则分成了 3 项，而这 3 项都是容易计算的：

1. $\partial \boldsymbol{I}_2 / \partial \boldsymbol{u}$ 为 $\boldsymbol{u}$ 处的像素梯度。
2. $\partial \boldsymbol{u} / \partial \boldsymbol{q}$ 为投影方程关于相机坐标系下的三维点的导数。记 $\boldsymbol{q} = [X, Y, Z]^{\mathrm{T}}$，根据第 7 讲的推导，导数为

$$\frac{\partial \boldsymbol{u}}{\partial \boldsymbol{q}} = \begin{bmatrix} \frac{\partial u}{\partial X} & \frac{\partial u}{\partial Y} & \frac{\partial u}{\partial Z} \\ \frac{\partial v}{\partial X} & \frac{\partial v}{\partial Y} & \frac{\partial v}{\partial Z} \end{bmatrix} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} \end{bmatrix}. \tag{8.16}$$

3. $\partial \boldsymbol{q} / \partial \delta\boldsymbol{\xi}$ 为变换后的三维点对变换的导数，这在李代数一讲介绍过：

$$\frac{\partial \boldsymbol{q}}{\partial \delta\boldsymbol{\xi}} = [\boldsymbol{I}, -\boldsymbol{q}^{\wedge}]. \tag{8.17}$$

在实践中，由于后两项只与三维点 $\boldsymbol{q}$ 有关，而与图像无关，我们经常把它合并在一起：

$$\frac{\partial \boldsymbol{u}}{\partial \delta\boldsymbol{\xi}} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & -\frac{f_x XY}{Z^2} & f_x + \frac{f_x X^2}{Z^2} & -\frac{f_x Y}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & -f_y - \frac{f_y Y^2}{Z^2} & \frac{f_y XY}{Z^2} & \frac{f_y X}{Z} \end{bmatrix}. \tag{8.18}$$

这个 2×6 的矩阵在第 7 讲中也出现过。于是，我们推导出误差相对于李代数的雅可比矩阵：

$$\boldsymbol{J} = -\frac{\partial \boldsymbol{I}_2}{\partial \boldsymbol{u}} \frac{\partial \boldsymbol{u}}{\partial \delta\boldsymbol{\xi}}. \tag{8.19}$$

对于 N 个点的问题，我们可以用这种方法计算优化问题的雅可比矩阵，然后使用高斯牛顿

法或列文伯格—马夸尔特方法计算增量，迭代求解。至此，我们推导了直接法估计相机位姿的整个流程，下面通过程序来演示直接法是如何使用的。

8.4.2　直接法的讨论

在上面的推导中，P 是一个已知位置的空间点，它是怎么来的呢？在 RGB-D 相机下，我们可以把任意像素反投影到三维空间，然后投影到下一幅图像中。如果在双目相机中，那么同样可以根据视差来计算像素的深度。如果在单目相机中，这件事情要更为困难，因为我们还须考虑由 P 的深度带来的不确定性。详细的深度估计放到第 13 讲中讨论。现在我们先来考虑简单的情况，即 P 深度已知的情况。

根据 P 的来源，我们可以把直接法进行分类：

1. P 来自于稀疏关键点，我们称之为稀疏直接法。通常，我们使用数百个至上千个关键点，并且像 L-K 光流那样，假设它周围像素也是不变的。这种稀疏直接法不必计算描述子，并且只使用数百个像素，因此速度最快，但只能计算稀疏的重构。
2. P 来自部分像素。我们看到式 (8.19) 中，如果像素梯度为零，那么整项雅可比矩阵就为零，不会对计算运动增量有任何贡献。因此，可以考虑只使用带有梯度的像素点，舍弃像素梯度不明显的地方。这称为半稠密（Semi-Dense）的直接法，可以重构一个半稠密结构。
3. P 为所有像素，称为稠密直接法。稠密重构需要计算所有像素（一般几十万至几百万个），因此多数不能在现有的 CPU 上实时计算，需要 GPU 的加速。但是，如前面讨论的，像素梯度不明显的点，在运动估计中不会有太大贡献，在重构时也会难以估计位置。

可以看到，从稀疏到稠密重构，都可以用直接法计算。它们的计算量是逐渐增长的。稀疏方法可以快速地求解相机位姿，而稠密方法可以建立完整地图。具体使用哪种方法，需要视机器人的应用环境而定。特别地，在低端的计算平台上，稀疏直接法可以做到非常快速的效果，适用于实时性较高且计算资源有限的场合[70]。

8.5　实践：直接法

8.5.1　单层直接法

现在，我们来演示如何使用稀疏的直接法。由于本书不涉及 GPU 编程，稠密的直接法就省略了。同时，为了保持程序简单，我们使用带深度的数据而非单目数据，这样可以省略单目的深度恢复部分。基于特征点的深度恢复（即三角化）已经在第 7 讲介绍过，而基于块匹配的深度恢复将在后面介绍。所以本节我们考虑双目的稀疏直接法。

求解直接法最后等价于求解一个优化问题，因此可以使用 g2o 或 Ceres 这些优化库来帮助求

解，也可以自己实现高斯牛顿法。和光流类似，直接法也可以分为单层直接法和金字塔式的多层直接法。我们同样先来实现单层直接法，进而拓展到多层直接法。

在单层直接法中，类似于并行的光流，我们也可以并行地计算每个像素点的误差和雅可比，为此我们定义一个求雅可比的类：

slambook2/ch8/direct_method.cpp（片段）

```
/// class for accumulator jacobians in parallel
class JacobianAccumulator {
public:
    JacobianAccumulator(
        const cv::Mat &img1_,
        const cv::Mat &img2_,
        const VecVector2d &px_ref_,
        const vector<double> depth_ref_,
        Sophus::SE3d &T21_) :
    img1(img1_), img2(img2_), px_ref(px_ref_), depth_ref(depth_ref_), T21(T21_) {
        projection = VecVector2d(px_ref.size(), Eigen::Vector2d(0, 0));
    }

    /// accumulate jacobians in a range
    void accumulate_jacobian(const cv::Range &range);

    /// get hessian matrix
    Matrix6d hessian() const { return H; }

    /// get bias
    Vector6d bias() const { return b; }

    /// get total cost
    double cost_func() const { return cost; }

    /// get projected points
    VecVector2d projected_points() const { return projection; }

    /// reset h, b, cost to zero
    void reset() {
        H = Matrix6d::Zero();
        b = Vector6d::Zero();
        cost = 0;
    }

private:
    const cv::Mat &img1;
```

```
    const cv::Mat &img2;
    const VecVector2d &px_ref;
    const vector<double> depth_ref;
    Sophus::SE3d &T21;
    VecVector2d projection; // projected points

    std::mutex hessian_mutex;
    Matrix6d H = Matrix6d::Zero();
    Vector6d b = Vector6d::Zero();
    double cost = 0;
};

void JacobianAccumulator::accumulate_jacobian(const cv::Range &range) {

    // parameters
    const int half_patch_size = 1;
    int cnt_good = 0;
    Matrix6d hessian = Matrix6d::Zero();
    Vector6d bias = Vector6d::Zero();
    double cost_tmp = 0;

    for (size_t i = range.start; i < range.end; i++) {
        // compute the projection in the second image
        Eigen::Vector3d point_ref =
        depth_ref[i] * Eigen::Vector3d((px_ref[i][0] - cx) / fx, (px_ref[i][1] - cy) / fy, 1)
        ;
        Eigen::Vector3d point_cur = T21 * point_ref;
        if (point_cur[2] < 0)   // depth invalid
            continue;

        float u = fx * point_cur[0] / point_cur[2] + cx, v = fy * point_cur[1] / point_cur[2]
         + cy;
        if (u < half_patch_size || u > img2.cols - half_patch_size || v < half_patch_size ||
        v > img2.rows - half_patch_size)
            continue;

        projection[i] = Eigen::Vector2d(u, v);
        double X = point_cur[0], Y = point_cur[1], Z = point_cur[2],
        Z2 = Z * Z, Z_inv = 1.0 / Z, Z2_inv = Z_inv * Z_inv;
        cnt_good++;

        // and compute error and jacobian
        for (int x = -half_patch_size; x <= half_patch_size; x++)
```

```
79          for (int y = -half_patch_size; y <= half_patch_size; y++) {
80              double error = GetPixelValue(img1, px_ref[i][0] + x, px_ref[i][1] + y) -
81                  GetPixelValue(img2, u + x, v + y);
82              Matrix26d J_pixel_xi;
83              Eigen::Vector2d J_img_pixel;
84
85              J_pixel_xi(0, 0) = fx * Z_inv;
86              J_pixel_xi(0, 1) = 0;
87              J_pixel_xi(0, 2) = -fx * X * Z2_inv;
88              J_pixel_xi(0, 3) = -fx * X * Y * Z2_inv;
89              J_pixel_xi(0, 4) = fx + fx * X * X * Z2_inv;
90              J_pixel_xi(0, 5) = -fx * Y * Z_inv;
91
92              J_pixel_xi(1, 0) = 0;
93              J_pixel_xi(1, 1) = fy * Z_inv;
94              J_pixel_xi(1, 2) = -fy * Y * Z2_inv;
95              J_pixel_xi(1, 3) = -fy - fy * Y * Y * Z2_inv;
96              J_pixel_xi(1, 4) = fy * X * Y * Z2_inv;
97              J_pixel_xi(1, 5) = fy * X * Z_inv;
98
99              J_img_pixel = Eigen::Vector2d(
100                 0.5 * (GetPixelValue(img2, u + 1 + x, v + y) - GetPixelValue(img2, u - 1 + x,
                     v + y)),
101                 0.5 * (GetPixelValue(img2, u + x, v + 1 + y) - GetPixelValue(img2, u + x, v -
                     1 + y))
102             );
103
104             // total jacobian
105             Vector6d J = -1.0 * (J_img_pixel.transpose() * J_pixel_xi).transpose();
106             hessian += J * J.transpose();
107             bias += -error * J;
108             cost_tmp += error * error;
109         }
110     }
111
112     if (cnt_good) {
113         // set hessian, bias and cost
114         unique_lock<mutex> lck(hessian_mutex);
115         H += hessian;
116         b += bias;
117         cost += cost_tmp / cnt_good;
118     }
119 }
```

在这个类的 accumulate_jacobian 函数中，我们对指定范围内的像素点，按照之前的推导计算像素误差和雅可比矩阵，最后加到整体的 $\boldsymbol{H}$ 矩阵中。然后，定义一个函数来迭代这个过程：

slambook2/ch8/direct_method.cpp（片段）

```
void DirectPoseEstimationSingleLayer(
    const cv::Mat &img1,
    const cv::Mat &img2,
    const VecVector2d &px_ref,
    const vector<double> depth_ref,
    Sophus::SE3d &T21) {
    const int iterations = 10;
    double cost = 0, lastCost = 0;
    JacobianAccumulator jaco_accu(img1, img2, px_ref, depth_ref, T21);

    for (int iter = 0; iter < iterations; iter++) {
        jaco_accu.reset();
        cv::parallel_for_(cv::Range(0, px_ref.size()),
            std::bind(&JacobianAccumulator::accumulate_jacobian, &jaco_accu, std::
            placeholders::_1));
        Matrix6d H = jaco_accu.hessian();
        Vector6d b = jaco_accu.bias();

        // solve update and put it into estimation
        Vector6d update = H.ldlt().solve(b);;
        T21 = Sophus::SE3d::exp(update) * T21;
        cost = jaco_accu.cost_func();

        if (std::isnan(update[0])) {
            // sometimes occurred when we have a black or white patch and H is irreversible
            cout << "update is nan" << endl;
            break;
        }
        if (iter > 0 && cost > lastCost) {
            cout << "cost increased: " << cost << ", " << lastCost << endl;
            break;
        }
        if (update.norm() < 1e-3) {
            // converge
            break;
        }

        lastCost = cost;
        cout << "iteration: " << iter << ", cost: " << cost << endl;
```

```
    }
}
```

该函数根据计算的 $\boldsymbol{H}$ 和 $\boldsymbol{b}$，求出对应的位姿更新量，然后更新到当前的估计值上。因为我们在理论部分已经把细节都介绍清楚了，所以这部分代码看起来不会特别困难。

8.5.2 多层直接法

然后，类似于光流，我们再把单层直接法拓展到金字塔式的多层直接法上，用 Coarse-to-fine 的过程计算相对运动。这部分代码和光流的也非常相似：

slambook2/ch8/direct_method.cpp（片段）

```
void DirectPoseEstimationMultiLayer(
    const cv::Mat &img1,
    const cv::Mat &img2,
    const VecVector2d &px_ref,
    const vector<double> depth_ref,
    Sophus::SE3d &T21) {
    // parameters
    int pyramids = 4;
    double pyramid_scale = 0.5;
    double scales[] = {1.0, 0.5, 0.25, 0.125};

    // create pyramids
    vector<cv::Mat> pyr1, pyr2; // image pyramids
    for (int i = 0; i < pyramids; i++) {
        if (i == 0) {
            pyr1.push_back(img1);
            pyr2.push_back(img2);
        } else {
            cv::Mat img1_pyr, img2_pyr;
            cv::resize(pyr1[i - 1], img1_pyr,
                cv::Size(pyr1[i - 1].cols * pyramid_scale, pyr1[i - 1].rows * pyramid_scale))
                ;
            cv::resize(pyr2[i - 1], img2_pyr,
                cv::Size(pyr2[i - 1].cols * pyramid_scale, pyr2[i - 1].rows * pyramid_scale))
                ;
            pyr1.push_back(img1_pyr);
            pyr2.push_back(img2_pyr);
        }
    }

    double fxG = fx, fyG = fy, cxG = cx, cyG = cy;  // backup the old values
```

```
    for (int level = pyramids - 1; level >= 0; level--) {
        VecVector2d px_ref_pyr; // set the keypoints in this pyramid level
        for (auto &px: px_ref) {
            px_ref_pyr.push_back(scales[level] * px);
        }

        // scale fx, fy, cx, cy in different pyramid levels
        fx = fxG * scales[level];
        fy = fyG * scales[level];
        cx = cxG * scales[level];
        cy = cyG * scales[level];
        DirectPoseEstimationSingleLayer(pyr1[level], pyr2[level], px_ref_pyr, depth_ref,T21);
    }
}
```

需要注意的是，直接法求雅可比的时候带上了相机的内参，而当金字塔对图像进行缩放时，对应的内参也需要乘以相应的倍率。

8.5.3　结果讨论

最后，我们用一些示例图片测试直接法的结果。我们会用到几张 Kitti[74] 自动驾驶数据集的图像。首先，我们读取第一个图像 left.png，在对应的视差图 disparity.png 中，计算每个像素对应的深度，然后对 000001.png~000005.png 这五张图像，利用直接法计算相机的位姿。为了展示直接法对特征点的不敏感性，我们随机地在第一张图像中选取一些点，不使用任何角点或特征点提取算法，来看看它的结果。

slambook2/ch8/direct_method.cpp（片段）

```
int main(int argc, char **argv) {

    cv::Mat left_img = cv::imread(left_file, 0);
    cv::Mat disparity_img = cv::imread(disparity_file, 0);

    // let's randomly pick pixels in the first image and generate some 3d points in the first
     image's frame
    cv::RNG rng;
    int nPoints = 2000;
    int boarder = 20;
    VecVector2d pixels_ref;
    vector<double> depth_ref;

    // generate pixels in ref and load depth data
    for (int i = 0; i < nPoints; i++) {
```

```
        int x = rng.uniform(boarder, left_img.cols - boarder);  // don't pick pixels close to
         boarder
        int y = rng.uniform(boarder, left_img.rows - boarder);  // don't pick pixels close to
         boarder
        int disparity = disparity_img.at<uchar>(y, x);
        double depth = fx * baseline / disparity; // you know this is disparity to depth
        depth_ref.push_back(depth);
        pixels_ref.push_back(Eigen::Vector2d(x, y));
    }

    // estimates 01~05.png's pose using this information
    Sophus::SE3d T_cur_ref;

    for (int i = 1; i < 6; i++) {  // 1~10
        cv::Mat img = cv::imread((fmt_others % i).str(), 0);
        DirectPoseEstimationMultiLayer(left_img, img, pixels_ref, depth_ref, T_cur_ref);
    }
    return 0;
}
```

读者可以尝试在你的计算机上运行本段程序，它将输出每个图像的每层金字塔上的追踪点，并输出运行时间。多层直接法的结果如图 8-5 所示。根据程序输出结果，可以看到第五张追踪图像反映的大约是相机往前运动 3.8 米时的情况。可见，即使我们随机选点，直接法也能够正确追踪大部分的像素，同时估计相机的运动。这中间没有任何的特征提取、匹配或光流的过程。从运行时间上看，在 2000 个点时，直接法每迭代一层需要 1~2 毫秒，所以四层金字塔约耗时 8 毫秒。相比之下，2000 个点的光流耗时大约在十几毫秒，还不包括后续的位姿估计。所以，直接法相比于传统的特征点和光流通常更快一些。



原始图像



原始视差图



第五张追踪图像



直接法追踪点

图 8-5 直接法的实验结果。左上：原始图像；右上：原始图像对应的视差图；左下：第五张追踪图像；右下：追踪结果

下面我们简单地对直接法的迭代过程做一点解释。相比于特征点法，直接法完全依靠优化来求解相机位姿。从式 (8.19) 中可以看到，像素梯度引导着优化的方向。如果想要得到正确的优化结果，就必须保证**大部分像素梯度能够把优化引导到正确的方向**。

这是什么意思呢？我们不妨设身处地地扮演一下优化算法。假设对于参考图像，我们测量到一个灰度值为 229 的像素。并且，由于我们知道它的深度，可以推断出空间点 P 的位置（图 8-6 所示在 I_1 中测量到的灰度）。

此时，我们又得到了一幅新的图像，需要估计它的相机位姿。这个位姿是由一个初值不断地优化迭代得到的。假设我们的初值比较差，在这个初值下，空间点 P 投影后的像素灰度值是 126。于是，此像素的误差为 $229-126=103$。为了减小这个误差，我们希望**微调相机的位姿，使像素更亮一些**。

怎么知道往哪里微调像素会更亮呢？这就需要用到局部的像素梯度。我们在图像中发现，沿 u 轴往前走一步，该处的灰度值变成了 123，即减去了 3。同样地，沿 v 轴往前走一步，灰度值减了 18，变成 108。在这个像素周围，我们看到梯度是 $[-3,-18]$，为了提高亮度，我们会建议优化算法微调相机，使 P 的像往**左上方**移动。在这个过程中，我们用像素的局部梯度近似了它附近的灰度分布，不过请注意，真实图像并不是光滑的，所以这个梯度在远处就不成立了。

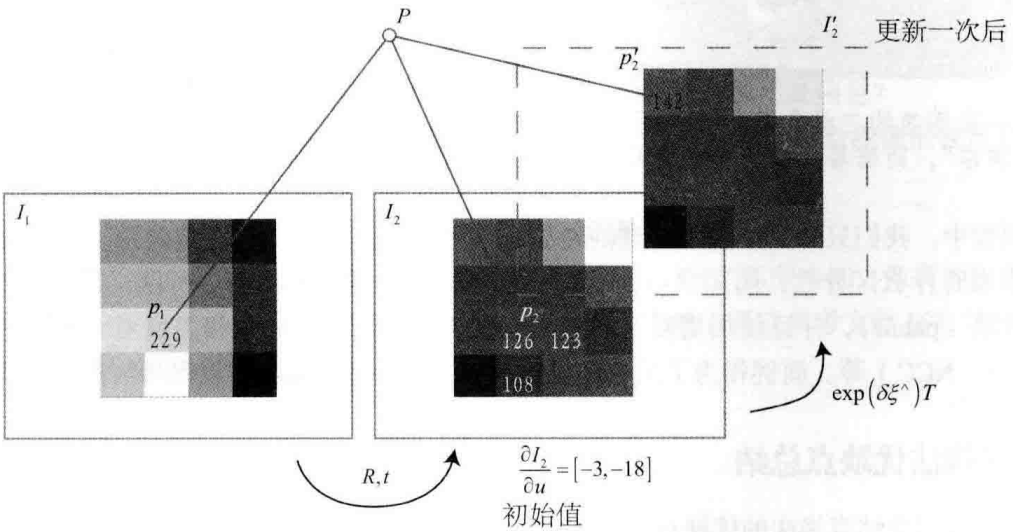


图 8-6　一次迭代的图形化显示

但是，优化算法不能只听这个像素的一面之词，还需要听取其他像素的建议[①]。综合听取了许多像素的意见之后，优化算法选择了一个和我们建议的方向偏离不远的地方，计算出一个更新量 $\exp(\boldsymbol{\xi}^\wedge)$。加上更新量后，图像从 I_2 移动到了 I_2'，像素的投影位置也变到了一个更亮的地方。我们看到，通过这次更新，**误差变小了**。在理想情况下，我们期望误差会不断下降，最后收敛。

①这可能是一种不严谨的拟人化说法，不过有助于理解。

但是实际是不是这样呢？我们是否真的只要沿着梯度方向走，就能走到一个最优值呢？注意，直接法的梯度是直接由图像梯度确定的，因此我们必须保证**沿着图像梯度走时，灰度误差会不断下降**。然而，图像通常是一个很强烈的**非凸函数**，如图 8-7 所示。实际中，如果我们沿着图像梯度前进，很容易由于图像本身的非凸性（或噪声）落进一个局部极小值中，无法继续优化。只有当相机运动很小，图像中的梯度不会有很强的非凸性时，直接法才能成立。

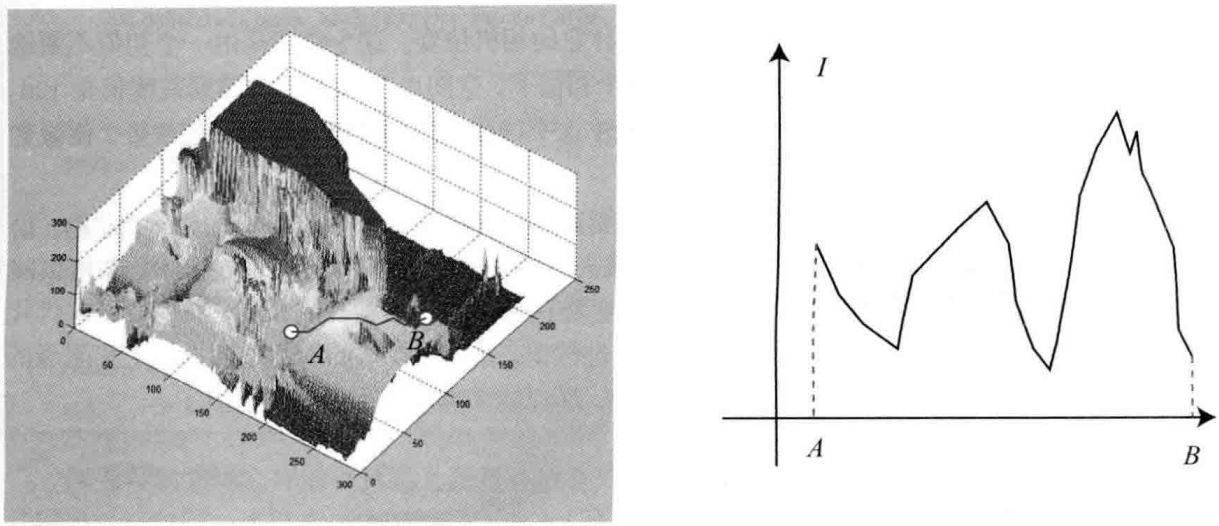


图 8-7 一张图像的三维化显示。从图像中的一个点运动到另一个点的路径不见得是“笔直的下坡路”，而需要经常“翻山越岭”。这体现了图像本身的非凸性

在例程中，我们只计算了单个像素的差异，并且这个差异是由灰度直接相减得到的。然而，单个像素没有什么区分性，周围很可能有好多像素和它的亮度差不多。所以，我们有时会使用小的图像块（patch），并且使用更复杂的差异度量方式，例如归一化相关性（Normalized Cross Correlation，NCC）等。而例程为了简单起见，使用了误差的平方和，以保持与推导的一致性。

8.5.4 直接法优缺点总结

最后，我们总结直接法的优缺点。大体上，它的优点如下：

- 可以省去计算特征点、描述子的时间。
- 只要求有像素梯度即可，不需要特征点。因此，直接法可以在特征缺失的场合下使用。比较极端的例子是只有渐变的一幅图像。它可能无法提取角点类特征，但可以用直接法估计它的运动。在演示实验中，我们看到直接法对随机选取的点亦能正常工作。这一点在实用中非常关键，因为实用场景很有可能没有很多角点可供使用。
- 可以构建半稠密乃至稠密的地图，这是特征点法无法做到的。

它的缺点也很明显：

- **非凸性**。直接法完全依靠梯度搜索，降低目标函数来计算相机位姿。其目标函数中需要取像素点的灰度值，而图像是强烈非凸的函数。这使得优化算法容易进入极小，只在运动很小时直接法才能成功。针对于此，金字塔的引入可以在一定程度上减小非凸性的影响。
- **单个像素没有区分度**。和它像的实在太多了！于是我们要么计算图像块，要么计算复杂的相关性。由于每个像素对改变相机运动的“意见”不一致，只能少数服从多数，以数量代替质量。所以，直接法在选点较少时的表现下降明显，我们通常建议用 500 个点以上。
- **灰度值不变是很强的假设**。如果相机是自动曝光的，当它调整曝光参数时，会使得图像整体变亮或变暗。光照变化时也会出现这种情况。特征点法对光照具有一定的容忍性，而直接法由于计算灰度间的差异，整体灰度变化会破坏灰度不变假设，使算法失败。针对这一点，实用的直接法会同时估计相机的曝光参数[70]，以便在曝光时间变化时也能工作。

习题

1. 除了 LK 光流，还有哪些光流方法？它们各有什么特点？
2. 在本节程序的求图像梯度过程中，我们简单地求了 $u+1$ 和 $u-1$ 的灰度之差除以 2，作为 u 方向上的梯度值。这种做法有什么缺点？提示：对于距离较近的特征，变化应该较快；而距离较远的特征在图像中变化较慢，求梯度时能否利用此信息？
3. 直接法是否能和光流一样，提出“反向法”的概念？即，使用原始图像的梯度代替目标图像的梯度？
4.* 使用 Ceres 或 g2o 实现稀疏直接法和半稠密直接法。
5. 相比于 RGB-D 的直接法，单目直接法往往更加复杂。除了匹配未知，像素的距离也是待估计的，我们需要在优化时把像素深度也作为优化变量。阅读参考文献 [69, 75]，你能理解它的原理吗？

第 9 讲

后端 1

主要目标

1. 理解后端的概念。
2. 理解以 EKF 为代表的滤波器后端的工作原理。
3. 理解非线性优化的后端，明白稀疏性是如何利用的。
4. 使用 g2o 和 Ceres 实际操作后端优化。

从本讲开始，我们转入 SLAM 系统的另一个重要模块：后端优化。

我们看到，前端视觉里程计能给出一个短时间内的轨迹和地图，但由于不可避免的误差累积，这个地图在长时间内是不准确的。所以，在视觉里程计的基础上，我们还希望构建一个尺度、规模更大的优化问题，以考虑长时间内的最优轨迹和地图。不过，考虑到精度与性能的平衡，实际中存在着许多不同的做法。


$z_{11} = h(x_1, y_1)$
y_1
x_1
$J = \begin{bmatrix} \square & \square \\ \square & \square \end{bmatrix}$
$z_{21} = h(x_2, y_1)$
f_1
$H = \begin{bmatrix} \square & \square \\ \square & \square \end{bmatrix}$
x_2
f_2
x_3
$z_{22} = h(x_2, y_2)$
y_2
$z_{33} = h(x_3, y_3)$
$z_{34} = h(x_3, y_4)$
$z_{23} = h(x_2, y_3)$
f_3
y_4
y_3
$z_{43} = h(x_4, y_3)$
x_4

9.1 概述

9.1.1 状态估计的概率解释

第 2 讲中提到，视觉里程计只有短暂的记忆，而我们希望整个运动轨迹在较长时间内都能保持最优的状态。我们可能会用最新的知识，更新较久远的状态——站在“久远的状态”的角度上看，仿佛是未来的信息告诉它“你应该在哪里”。所以，在后端优化中，我们通常考虑一段更长时间内（或所有时间内）的状态估计问题，而且不仅使用过去的信息更新自己的状态，也会用未来的信息来更新，这种处理方式称为“批量的”（Batch）。否则，如果当前的状态只由过去的时刻决定，甚至只由前一个时刻决定，则称为“渐进的”（Incremental）。

我们已经知道 SLAM 过程可以由运动方程和观测方程来描述。那么，假设在 $t=0$ 到 $t=N$ 的时间内，有位姿 $\boldsymbol{x}_0$ 到 $\boldsymbol{x}_N$，并且有路标 $\boldsymbol{y}_1,\cdots,\boldsymbol{y}_M$。按照之前的写法，运动和观测方程为

$$\begin{cases}\boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1},\boldsymbol{u}_k\right)+\boldsymbol{w}_k\\ \boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j,\boldsymbol{x}_k\right)+\boldsymbol{v}_{k,j}\end{cases}\qquad k=1,\ldots,N,\ j=1,\ldots,M. \tag{9.1}$$

注意以下几点：

1. 观测方程中，只有当 $\boldsymbol{x}_k$ 看到了 $\boldsymbol{y}_j$ 时，才会产生观测数据，否则就没有。事实上，在一个位置通常只能看到一小部分路标。而且，由于视觉 SLAM 特征点数量众多，所以实际中观测方程的数量会远远大于运动方程。
2. 我们可能没有测量运动的装置，也可能没有运动方程。在这个情况下，有若干种处理方式：认为确实没有运动方程，或假设相机不动，或假设相机匀速运动。这几种方式都是可行的。在没有运动方程的情况下，整个优化问题就只由许多个观测方程组成。这就非常类似于 SfM 问题，相当于我们通过一组图像来恢复运动和结构。不同的是，SLAM 中的图像有时间上的先后顺序，而 SfM 中允许使用完全无关的图像。

我们知道每个方程都受噪声影响，所以要把这里的位姿 $\boldsymbol{x}$ 和路标 $\boldsymbol{y}$ 看成**服从某种概率分布的随机变量**，而不是单独的一个数。因此，我们关心的问题就变成：当我们拥有某些运动数据 $\boldsymbol{u}$ 和观测数据 $\boldsymbol{z}$ 时，如何确定状态量 $\boldsymbol{x},\boldsymbol{y}$ 的分布？进而，如果得到了新时刻的数据，它们的分布又将发生怎样的变化？在比较常见且合理的情况下，我们假设状态量和噪声项服从高斯分布——这意味着在程序中只需要存储它们的均值和协方差矩阵即可。均值可看作对变量最优值的估计，而协方差矩阵则度量了它的不确定性。那么，问题就转变为：当存在一些运动数据和观测数据时，我们如何估计状态量的高斯分布？

我们依然设身处地地扮演小萝卜。只有运动方程时，相当于我们蒙着眼睛在一个未知的地方走路。尽管我们知道自己每一步走了多远，但是随着时间流逝，我们越来越不确定自己的位置——内心也就越不安。这说明当输入数据受噪声影响时，**误差是逐渐累积的**，我们对位置方差

的估计将越来越大。但是，当我们睁开眼睛时，由于能够不断地观测到外部场景，使得位置估计的不确定性变小，我们就会越来越自信。如果用椭圆或椭球直观地表达协方差阵，那么这个过程有点像是在手机地图软件中走路的感觉。以图 9-1 为例，读者可以想象，当没有观测数据时，这个圆会随着运动越来越大；而如果有正确的观测数据，圆就会缩小至一定的大小，保持稳定。

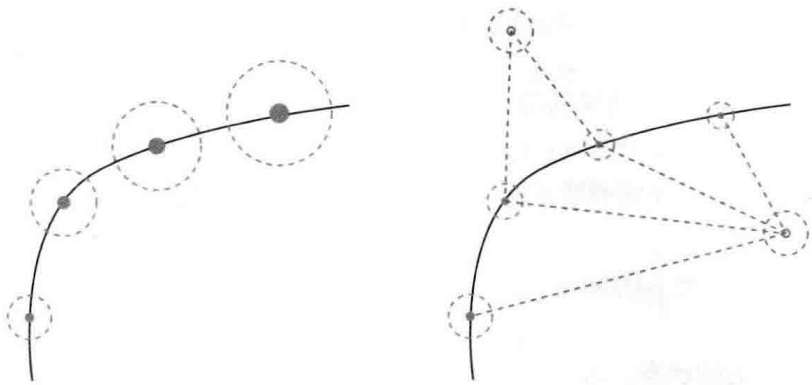


图 9-1　不确定性的直观描述。左侧：只有运动方程时，由于下一时刻的位姿是在上一时刻的基础上添加了噪声，所以不确定性越来越大。右侧：存在路标点时，不确定性会明显减小。不过请注意，这只是一个直观的示意图，并非实际数据

上面的过程以比喻的形式解释了状态估计中的问题，下面我们要以定量的方式来看待它。在第 6 讲中，我们介绍了最大似然估计，提到**批量状态估计问题可以转化为最大似然估计问题，并使用最小二乘法进行求解**。在本节，我们将探讨如何将该结论应用于渐进式问题，得到一些经典的结论。同时，在视觉 SLAM 里，最小二乘法又有何特殊的结构。

首先，由于位姿和路标点都是待估计的变量，我们改变记号，令 $\boldsymbol{x}_k$ 为 k 时刻的所有未知量。它包含了当前时刻的相机位姿与 m 个路标点。在这种记号的意义下（虽然与之前稍有不同，但含义是清楚的），写成

$$\boldsymbol{x}_k \overset{\text{def}}{=} \{\boldsymbol{x}_k, \boldsymbol{y}_1, \ldots, \boldsymbol{y}_m\}. \tag{9.2}$$

同时，把 k 时刻的所有观测记作 $\boldsymbol{z}_k$。于是，运动方程与观测方程的形式可写得更简洁。这里不会出现 $\boldsymbol{y}$，但我们心里要明白这时 $\boldsymbol{x}$ 中已经包含了之前的 $\boldsymbol{y}$：

$$\begin{cases} \boldsymbol{x}_k = f(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k) + \boldsymbol{w}_k \\ \boldsymbol{z}_k = h(\boldsymbol{x}_k) + \boldsymbol{v}_k \end{cases} \quad k = 1, \ldots, N. \tag{9.3}$$

现在考虑第 k 时刻的情况。我们希望用过去 0 到 k 中的数据来估计现在的状态分布：

$$P(\boldsymbol{x}_k | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k}). \tag{9.4}$$

下标 $0:k$ 表示从 0 时刻到 k 时刻的所有数据。请注意，$\boldsymbol{z}_k$ 表示所有在 k 时刻的观测数据，它可能不止一个，只是这种记法更方便。同时，$\boldsymbol{x}_k$ 实际上和 $\boldsymbol{x}_{k-1}, \boldsymbol{x}_{k-2}$ 这些量都有关，但是此式没有显式地将它们写出来。

下面我们来看如何对状态进行估计。按照贝叶斯法则，把 $\boldsymbol{z}_k$ 与 $\boldsymbol{x}_k$ 交换位置，有

$$P(\boldsymbol{x}_k|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k}) \propto P(\boldsymbol{z}_k|\boldsymbol{x}_k) P(\boldsymbol{x}_k|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}). \tag{9.5}$$

读者应该不会感到陌生。这里的第一项称为**似然**，第二项称为**先验**。似然由观测方程给定，而先验部分，我们要明白当前状态 $\boldsymbol{x}_k$ 是基于过去所有的状态估计得来的。至少，它会受 $\boldsymbol{x}_{k-1}$ 影响，于是以 $\boldsymbol{x}_{k-1}$ 时刻为条件概率展开：

$$P(\boldsymbol{x}_k|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) = \int P(\boldsymbol{x}_k|\boldsymbol{x}_{k-1}, \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) P(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) \mathrm{d}\boldsymbol{x}_{k-1}. \tag{9.6}$$

如果考虑更久之前的状态，也可以继续对此式进行展开，但现在我们只关心 k 时刻和 $k-1$ 时刻的情况。至此，我们给出了贝叶斯估计，因为上式还没有具体的概率分布形式，所以没法实际操作它。对这一步的后续处理，方法上产生了一些分歧。大体上讲，存在若干种选择：一种方法是假设**马尔可夫性**，简单的一阶马氏性认为，k 时刻状态只与 $k-1$ 时刻状态有关，而与再之前的无关。如果做出这样的假设，我们就会得到以**扩展卡尔曼滤波**（EKF）为代表的滤波器方法。在滤波方法中，我们会从某时刻的状态估计，推导到下一个时刻。另一种方法是依然考虑 k 时刻状态与之前**所有**状态的关系，此时将得到**非线性优化**为主体的优化框架。非线性优化的基本知识已在前文介绍过。目前，视觉 SLAM 的主流为非线性优化方法。不过，为了让本书更全面，我们要先介绍卡尔曼滤波器（KF）和 EKF 的原理。

9.1.2 线性系统和 KF

我们首先来看滤波器模型。当我们假设了马尔可夫性，从数学角度会发生哪些变化呢？首先，当前时刻状态只和上一个时刻有关，式 (9.6) 中右侧第一部分可进一步简化：

$$P(\boldsymbol{x}_k|\boldsymbol{x}_{k-1}, \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) = P(\boldsymbol{x}_k|\boldsymbol{x}_{k-1}, \boldsymbol{u}_k). \tag{9.7}$$

这里，由于 k 时刻状态与 $k-1$ 之前的无关，所以就简化成只与 $\boldsymbol{x}_{k-1}$ 和 $\boldsymbol{u}_k$ 有关的形式，与 k 时刻的运动方程对应。第二部分可简化为

$$P(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) = P(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0, \boldsymbol{u}_{1:k-1}, \boldsymbol{z}_{1:k-1}). \tag{9.8}$$

考虑到 k 时刻的输入量 $\boldsymbol{u}_k$ 与 $k-1$ 时刻的状态无关，所以我们把 $\boldsymbol{u}_k$ 拿掉。可以看到，这一

项实际上是 $k-1$ 时刻的状态分布。于是，这一系列方程说明，我们实际在做的是“如何把 $k-1$ 时刻的状态分布推导至 k 时刻”这样一件事。也就是说，在程序运行期间，我们只要维护一个状态量，对它不断地进行迭代和更新即可。如果假设状态量服从高斯分布，那么我们只需考虑维护状态量的均值和协方差即可。你可以想象成小萝卜上的定位系统一直在向外输出两个定位信息：一是自己的位姿，二是自己的不确定性。实际中往往也是如此。

我们从形式最简单的线性高斯系统开始，最后得到卡尔曼滤波器。明确了起点和终点之后，我们再来考虑中间的路线。线性高斯系统是指，运动方程和观测方程可以由线性方程来描述：

$$\begin{cases} \boldsymbol{x}_k = \boldsymbol{A}_k \boldsymbol{x}_{k-1} + \boldsymbol{u}_k + \boldsymbol{w}_k \\ \boldsymbol{z}_k = \boldsymbol{C}_k \boldsymbol{x}_k + \boldsymbol{v}_k \end{cases} \quad k = 1, \dots, N. \tag{9.9}$$

并假设所有的状态和噪声均满足高斯分布。记这里的噪声服从零均值高斯分布：

$$\boldsymbol{w}_k \sim N(\boldsymbol{0}, \boldsymbol{R}). \quad \boldsymbol{v}_k \sim N(\boldsymbol{0}, \boldsymbol{Q}). \tag{9.10}$$

为了简洁，笔者省略了 $\boldsymbol{R}$ 和 $\boldsymbol{Q}$ 的下标。现在，利用马尔可夫性，假设我们知道了 $k-1$ 时刻的后验（在 $k-1$ 时刻看来）状态估计 $\hat{\boldsymbol{x}}_{k-1}$ 及其协方差 $\hat{\boldsymbol{P}}_{k-1}$，现在要根据 k 时刻的输入和观测数据，确定 $\boldsymbol{x}_k$ 的后验分布。为区分推导中的先验和后验，我们在记号上做一点区别：以上帽子 $\hat{\boldsymbol{x}}_k$ 表示后验，以下帽子 $\check{\boldsymbol{x}}_k$ 表示先验分布，请读者不要混淆。

卡尔曼滤波器的第一步，通过运动方程确定 $\boldsymbol{x}_k$ 的先验分布。这一步是线性的，而高斯分布的线性变换仍是高斯分布。所以，显然有

$$P(\boldsymbol{x}_k | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) = N\left(\boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{u}_k, \boldsymbol{A}_k \hat{\boldsymbol{P}}_{k-1} \boldsymbol{A}_k^{\mathrm{T}} + \boldsymbol{R}\right). \tag{9.11}$$

这一步称为**预测**（Predict），原理见附录 A.3。它显示了如何从上一个时刻的状态，根据输入信息（但是有噪声）推断当前时刻的状态分布。这个分布也就是先验。记：

$$\check{\boldsymbol{x}}_k = \boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{u}_k, \quad \check{\boldsymbol{P}}_k = \boldsymbol{A}_k \hat{\boldsymbol{P}}_{k-1} \boldsymbol{A}_k^{\mathrm{T}} + \boldsymbol{R}. \tag{9.12}$$

这非常自然。一方面，显然这一步状态的不确定度要变大，因为系统中添加了噪声。另一方面，由观测方程，我们可以计算在某个状态下应该产生怎样的观测数据：

$$P(\boldsymbol{z}_k | \boldsymbol{x}_k) = N(\boldsymbol{C}_k \boldsymbol{x}_k, \boldsymbol{Q}). \tag{9.13}$$

为了得到后验概率，我们想要计算它们的乘积，也就是由式 (9.5) 给出的贝叶斯公式。然而，虽然我们知道最后会得到一个关于 $\boldsymbol{x}_k$ 的高斯分布，但计算上是有一点点麻烦的，我们先把结果

设为 $\boldsymbol{x}_k \sim N(\hat{\boldsymbol{x}}_k, \hat{\boldsymbol{P}}_k)$，那么：

$$N(\hat{\boldsymbol{x}}_k, \hat{\boldsymbol{P}}_k) = \eta N\left(\boldsymbol{C}_k \boldsymbol{x}_k, \boldsymbol{Q}\right) \cdot N(\check{\boldsymbol{x}}_k, \check{\boldsymbol{P}}_k). \tag{9.14}$$

这里我们稍微用点讨巧的方法。既然我们已经知道等式两侧都是高斯分布，那就只需比较指数部分，无须理会高斯分布前面的因子部分。指数部分很像是一个二次型的配方，我们来推导一下。首先把指数部分展开，有[①]

$$\left(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k\right)^{\mathrm{T}} \hat{\boldsymbol{P}}_k^{-1} \left(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k\right) = \left(\boldsymbol{z}_k - \boldsymbol{C}_k \boldsymbol{x}_k\right)^{\mathrm{T}} \boldsymbol{Q}^{-1} \left(\boldsymbol{z}_k - \boldsymbol{C}_k \boldsymbol{x}_k\right) + \left(\boldsymbol{x}_k - \check{\boldsymbol{x}}_k\right)^{\mathrm{T}} \check{\boldsymbol{P}}_k^{-1} \left(\boldsymbol{x}_k - \check{\boldsymbol{x}}_k\right). \tag{9.15}$$

为了求左侧的 $\hat{\boldsymbol{x}}_k$ 和 $\hat{\boldsymbol{P}}_k$，我们把两边展开，并比较 $\boldsymbol{x}_k$ 的二次和一次系数。对于二次系数，有

$$\hat{\boldsymbol{P}}_k^{-1} = \boldsymbol{C}_k^{\mathrm{T}} \boldsymbol{Q}^{-1} \boldsymbol{C}_k + \check{\boldsymbol{P}}_k^{-1}. \tag{9.16}$$

该式给出了协方差的计算过程。为了便于后面列写式子，定义一个中间变量：

$$\boldsymbol{K} = \hat{\boldsymbol{P}}_k \boldsymbol{C}_k^{\mathrm{T}} \boldsymbol{Q}^{-1}. \tag{9.17}$$

根据此定义，在式 (9.16) 的左右各乘 $\hat{\boldsymbol{P}}_k$，有

$$\boldsymbol{I} = \hat{\boldsymbol{P}}_k \boldsymbol{C}_k^{\mathrm{T}} \boldsymbol{Q}^{-1} \boldsymbol{C}_k + \hat{\boldsymbol{P}}_k \check{\boldsymbol{P}}_k^{-1} = \boldsymbol{K} \boldsymbol{C}_k + \hat{\boldsymbol{P}}_k \check{\boldsymbol{P}}_k^{-1}. \tag{9.18}$$

于是有[②]

$$\hat{\boldsymbol{P}}_k = (\boldsymbol{I} - \boldsymbol{K} \boldsymbol{C}_k) \check{\boldsymbol{P}}_k. \tag{9.19}$$

然后比较一次项的系数，有

$$-2 \hat{\boldsymbol{x}}_k^{\mathrm{T}} \hat{\boldsymbol{P}}_k^{-1} \boldsymbol{x}_k = -2 \boldsymbol{z}_k^{\mathrm{T}} \boldsymbol{Q}^{-1} \boldsymbol{C}_k \boldsymbol{x}_k - 2 \check{\boldsymbol{x}}_k^{\mathrm{T}} \check{\boldsymbol{P}}_k^{-1} \boldsymbol{x}_k. \tag{9.20}$$

整理（取系数并转置）得

$$\hat{\boldsymbol{P}}_k^{-1} \hat{\boldsymbol{x}}_k = \boldsymbol{C}_k^{\mathrm{T}} \boldsymbol{Q}^{-1} \boldsymbol{z}_k + \check{\boldsymbol{P}}_k^{-1} \check{\boldsymbol{x}}_k. \tag{9.21}$$

两侧乘以 $\hat{\boldsymbol{P}}_k$ 并代入式 (9.17)，得

---

①这里的等号并不严格，实际允许相差与 $\boldsymbol{x}_k$ 无关的常数。

②这里看似有一点儿循环定义的意思。我们由 $\hat{\boldsymbol{P}}_k$ 定义了 $\boldsymbol{K}$，又把 $\hat{\boldsymbol{P}}_k$ 写成了 $\boldsymbol{K}$ 的表达式。然而，实际中 $\boldsymbol{K}$ 可以不依靠 $\hat{\boldsymbol{P}}_k$ 算得，但是这需要引入 SMW（Sherman-Morrison-Woodbury）恒等式[76]，参见本讲的习题。

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{P}}_k \boldsymbol{C}_k^{\mathrm{T}} \boldsymbol{Q}^{-1} \boldsymbol{z}_k + \hat{\boldsymbol{P}}_k \check{\boldsymbol{P}}_k^{-1} \check{\boldsymbol{x}}_k \tag{9.22}$$

$$= \boldsymbol{K}\boldsymbol{z}_k + (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{C}_k)\check{\boldsymbol{x}}_k = \check{\boldsymbol{x}}_k + \boldsymbol{K}\left(\boldsymbol{z}_k - \boldsymbol{C}_k \check{\boldsymbol{x}}_k\right). \tag{9.23}$$

于是我们又得到了后验均值的表达。总而言之，上面的两个步骤可以归纳为“预测”和“更新”（Update）两个步骤：

1. 预测：

$$\check{\boldsymbol{x}}_k = \boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{u}_k, \quad \check{\boldsymbol{P}}_k = \boldsymbol{A}_k \hat{\boldsymbol{P}}_{k-1} \boldsymbol{A}_k^{\mathrm{T}} + \boldsymbol{R}. \tag{9.24}$$

2. 更新：先计算 $\boldsymbol{K}$，它又称为卡尔曼增益。

$$\boldsymbol{K} = \check{\boldsymbol{P}}_k \boldsymbol{C}_k^{\mathrm{T}} \left(\boldsymbol{C}_k \check{\boldsymbol{P}}_k \boldsymbol{C}_k^{\mathrm{T}} + \boldsymbol{Q}_k\right)^{-1}. \tag{9.25}$$

然后计算后验概率的分布。

$$\begin{aligned} \hat{\boldsymbol{x}}_k &= \check{\boldsymbol{x}}_k + \boldsymbol{K}\left(\boldsymbol{z}_k - \boldsymbol{C}_k \check{\boldsymbol{x}}_k\right) \\ \hat{\boldsymbol{P}}_k &= (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{C}_k)\check{\boldsymbol{P}}_k. \end{aligned} \tag{9.26}$$

至此，我们推导了经典的卡尔曼滤波器的整个过程。事实上，卡尔曼滤波器有若干种推导方式，而我们使用的是从概率角度出发的最大后验概率估计的方式。我们看到，在线性高斯系统中，卡尔曼滤波器构成了该系统中的最大后验概率估计。而且，由于高斯分布经过线性变换后仍服从高斯分布，所以整个过程中我们没有进行任何的近似。可以说，卡尔曼滤波器构成了线性系统的最优无偏估计。

9.1.3　非线性系统和 EKF

在理解了卡尔曼滤波之后，我们必须要澄清一点：SLAM 中的运动方程和观测方程通常是非线性函数，尤其是视觉 SLAM 中的相机模型，需要使用相机内参模型及李代数表示的位姿，更不可能是一个线性系统。一个高斯分布，经过非线性变换后，往往不再是高斯分布，所以在非线性系统中，我们必须取一定的近似，将一个非高斯分布近似成高斯分布。

我们希望把卡尔曼滤波器的结果拓展到非线性系统中，称为扩展卡尔曼滤波器。通常的做法是，在某个点附近考虑运动方程及观测方程的一阶泰勒展开，只保留一阶项，即线性的部分，然后按照线性系统进行推导。令 $k-1$ 时刻的均值与协方差矩阵为 $\hat{\boldsymbol{x}}_{k-1}, \hat{\boldsymbol{P}}_{k-1}$。在 k 时刻，我们把运动方程和观测方程在 $\hat{\boldsymbol{x}}_{k-1}, \hat{\boldsymbol{P}}_{k-1}$ 处进行**线性化**（相当于一阶泰勒展开），有

$$\boldsymbol{x}_k \approx f\left(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k\right) + \left.\frac{\partial f}{\partial \boldsymbol{x}_{k-1}}\right|_{\hat{\boldsymbol{x}}_{k-1}} \left(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}\right) + \boldsymbol{w}_k. \tag{9.27}$$

记这里的偏导数为

$$\boldsymbol{F}=\left.\frac{\partial f}{\partial \boldsymbol{x}_{k-1}}\right|_{\hat{\boldsymbol{x}}_{k-1}}. \tag{9.28}$$

同样，对于观测方程，亦有

$$\boldsymbol{z}_k \approx h\left(\check{\boldsymbol{x}}_k\right)+\left.\frac{\partial h}{\partial \boldsymbol{x}_k}\right|_{\check{\boldsymbol{x}}_k}\left(\boldsymbol{x}_k-\check{\boldsymbol{x}}_k\right)+\boldsymbol{n}_k. \tag{9.29}$$

记这里的偏导数为

$$\boldsymbol{H}=\left.\frac{\partial h}{\partial \boldsymbol{x}_k}\right|_{\check{\boldsymbol{x}}_k}. \tag{9.30}$$

那么，在**预测**步骤中，根据运动方程有

$$P\left(\boldsymbol{x}_k|\boldsymbol{x}_0,\boldsymbol{u}_{1:k},\boldsymbol{z}_{0:k-1}\right)=N(f\left(\hat{\boldsymbol{x}}_{k-1},\boldsymbol{u}_k\right),\boldsymbol{F}\hat{\boldsymbol{P}}_{k-1}\boldsymbol{F}^{\mathrm{T}}+\boldsymbol{R}_k). \tag{9.31}$$

这些推导和卡尔曼滤波是十分相似的。为方便表述，记这里的先验和协方差的均值为

$$\check{\boldsymbol{x}}_k=f\left(\hat{\boldsymbol{x}}_{k-1},\boldsymbol{u}_k\right),\quad \check{\boldsymbol{P}}_k=\boldsymbol{F}\hat{\boldsymbol{P}}_{k-1}\boldsymbol{F}^{\mathrm{T}}+\boldsymbol{R}_k. \tag{9.32}$$

然后，考虑在观测中我们有

$$P\left(\boldsymbol{z}_k|\boldsymbol{x}_k\right)=N(h\left(\check{\boldsymbol{x}}_k\right)+\boldsymbol{H}\left(\boldsymbol{x}_k-\check{\boldsymbol{x}}_k\right),\boldsymbol{Q}_k). \tag{9.33}$$

最后，根据最开始的贝叶斯展开式，可以推导出 $\boldsymbol{x}_k$ 的后验概率形式。我们略去中间的推导过程，只介绍其结果。读者可以仿照卡尔曼滤波器的方式，推导 EKF 的预测与更新方程。简而言之，我们会先定义一个**卡尔曼增益** $\boldsymbol{K}_k$：

$$\boldsymbol{K}_k=\check{\boldsymbol{P}}_k\boldsymbol{H}^{\mathrm{T}}\left(\boldsymbol{H}\check{\boldsymbol{P}}_k\boldsymbol{H}^{\mathrm{T}}+\boldsymbol{Q}_k\right)^{-1}. \tag{9.34}$$

在卡尔曼增益的基础上，后验概率的形式为

$$\hat{\boldsymbol{x}}_k=\check{\boldsymbol{x}}_k+\boldsymbol{K}_k\left(\boldsymbol{z}_k-h\left(\check{\boldsymbol{x}}_k\right)\right),\hat{\boldsymbol{P}}_k=\left(\boldsymbol{I}-\boldsymbol{K}_k\boldsymbol{H}\right)\check{\boldsymbol{P}}_k. \tag{9.35}$$

卡尔曼滤波器给出了在线性化之后状态变量分布的变化过程。在线性系统和高斯噪声下，卡尔曼滤波器给出了无偏最优估计。而在 SLAM 这种非线性的情况下，它给出了单次线性近似下的最大后验估计。

9.1.4 EKF 的讨论

EKF 以形式简洁、应用广泛著称。当想要在某段时间内估计某个不确定量时，我们首先想到的就是 EKF。在早期的 SLAM 中，EKF 占据了很长一段时间的主导地位，研究者们讨论了各种各样的滤波器在 SLAM 中的应用，如 IF（信息滤波器）[77]、IKF[78]（Iterated KF）、UKF[79]（Unscented KF）和粒子滤波器[80–82]、SWF（Sliding Window Filter）[83]，等等[17]①，或者用分治法等思路改进 EKF 的效率[84, 85]。时至今日，尽管我们认识到非线性优化比滤波器占有明显的优势，但是在计算资源受限，或待估计量比较简单的场合，EKF 仍不失为一种有效的方式。

EKF 有哪些局限呢？

1. 滤波器方法在一定程度上假设了**马尔可夫性**，也就是 k 时刻的状态只与 $k-1$ 时刻相关，而与 $k-1$ 之前的状态和观测都无关（或者和前几个有限时刻的状态相关）。这有点像是在视觉里程计中只考虑相邻两帧的关系。如果当前帧确实与很久之前的数据有关（例如回环），那么滤波器会难以处理。

   而非线性优化方法则倾向于使用所有的历史数据。它不光考虑邻近时刻的特征点与轨迹关系，更会把很久之前的状态也考虑进来，称为全体时间上的 SLAM（Full-SLAM）。在这种意义下，非线性优化方法使用了更多信息，当然也需要更多的计算。
2. 与第 6 讲介绍的优化方法相比，EKF 滤波器仅在 $\hat{\boldsymbol{x}}_{k-1}$ 处做了**一次**线性化，就直接根据这次线性化的结果，把后验概率给算了出来。这相当于在说，我们认为**该点处的线性化近似在后验概率处仍然是有效的**。而实际上，当我们离工作点较远时，一阶泰勒展开并不一定能够近似整个函数，这取决于运动模型和观测模型的非线性情况。如果它们有强烈的非线性，那么线性近似就只在很小范围内成立，不能认为在很远的地方仍能用线性来近似。这就是 EKF 的**非线性误差**，也是它的主要问题所在。

   在优化问题中，尽管我们也做一阶（最速下降）或二阶（高斯牛顿法或列文伯格—马夸尔特方法）的近似，但每迭代一次，状态估计发生改变之后，我们会重新对新的估计点做泰勒展开，而不像 EKF 那样只在固定点上做一次泰勒展开。这就使得优化的方法适用范围更广，在状态变化较大时也能适用。所以大体来说，可以粗略地认为 **EKF 仅是优化中的一次迭代**②。
3. 从程序实现上来说，EKF 需要存储状态量的均值和方差，并对它们进行维护和更新。如果把路标也放进状态，由于视觉 SLAM 中路标数量很大，则这个存储量是相当可观的，且与状态量呈平方增长（因为要存储协方差矩阵）。因此，普遍认为 EKF SLAM 不适用于大型场景。
4. EKF 等滤波器方法没有异常检测机制，导致系统在存在异常值的时候很容易发散。而在

①粒子滤波器的原理与卡尔曼滤波有较大不同。

②更详细地说，比一次迭代要好一些，因为更新步骤的线性化是在预测基础之上。如果在预测时刻就同时线性化运动和观测模型，就完全和一次迭代一样了。

视觉 SLAM 中，异常值却是很常见的：无论特征匹配还是光流法，都容易追踪或匹配到错误的点。没有异常值检测机制会让系统在实用中非常不稳定。

由于 EKF 存在这些明显的缺点，我们通常认为，在同等计算量的情况下，非线性优化能取得更好的效果[13]。这里“更好”是指精度和鲁棒性同时达到更好的意思。下面我们来讨论以非线性优化为主的后端。我们将主要介绍图优化，并用 g2o 和 Ceres 演示后端优化。

9.2 BA 与图优化

如果你做过视觉三维重建，那么应该对这个概念再熟悉不过了。所谓的 Bundle Adjustment[①]（BA），是指从视觉图像中提炼出最优的 3D 模型和相机参数（内参数和外参数）。考虑从任意特征点发射出来的几束光线（bundles of light rays），它们会在几个相机的成像平面上变成像素或是检测到的特征点。如果我们调整（adjustment）各相机姿态和各特征点的空间位置，使得这些光线最终收束到相机的光心[34]，就称为 BA。

我们在第 5 讲和第 7 讲已经简单介绍过 BA 的原理，本节的重点是介绍它对应的图模型结构的特点，然后介绍一些通用的快速求解方法。

9.2.1 投影模型和 BA 代价函数

首先，我们复习整个投影的过程。从一个世界坐标系中的点 $\boldsymbol{p}$ 出发，把相机的内外参数和畸变都考虑进来，最后投影成像素坐标，需要如下步骤。

1. 把世界坐标转换到相机坐标，这里将用到相机外参数 $(\boldsymbol{R}, \boldsymbol{t})$：

$$\boldsymbol{P}' = \boldsymbol{R}\boldsymbol{p} + \boldsymbol{t} = [X', Y', Z']^{\mathrm{T}}. \tag{9.36}$$

2. 将 $\boldsymbol{P}'$ 投至归一化平面，得到归一化坐标：

$$\boldsymbol{P}_{\mathrm{c}} = [u_{\mathrm{c}}, v_{\mathrm{c}}, 1]^{\mathrm{T}} = [X'/Z', Y'/Z', 1]^{\mathrm{T}}. \tag{9.37}$$

3. 考虑归一化坐标的畸变情况，得到去畸变前的原始像素坐标。这里暂时只考虑径向畸变：

$$\begin{cases} u_{\mathrm{c}}' = u_{\mathrm{c}}\left(1 + k_1 r_{\mathrm{c}}^2 + k_2 r_{\mathrm{c}}^4\right) \\ v_{\mathrm{c}}' = v_{\mathrm{c}}\left(1 + k_1 r_{\mathrm{c}}^2 + k_2 r_{\mathrm{c}}^4\right) \end{cases}. \tag{9.38}$$

4. 根据内参模型，计算像素坐标：

①也译为光束法平差、捆集调整等，但笔者觉得没有 Bundle Adjustment 这个英文来得直观，所以这里保留英文名称。

$$\begin{cases} u_s = f_x u'_{\mathrm{c}} + c_x \\ v_s = f_y v'_{\mathrm{c}} + c_y \end{cases}. \tag{9.39}$$

这一系列计算流程看似复杂。我们用流程图 9-2 形象化地表示整个过程，以帮助读者理解。读者应该能领会到，这个过程也就是前面讲的**观测方程**，之前我们把它抽象地记成：

$$\boldsymbol{z} = h(\boldsymbol{x}, \boldsymbol{y}). \tag{9.40}$$

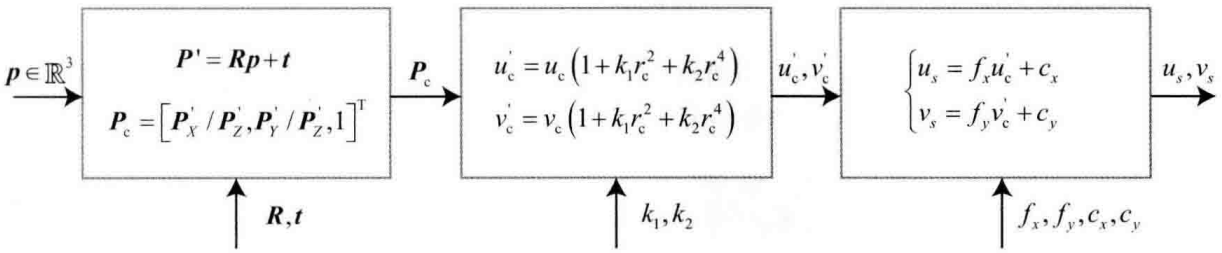


图 9-2　计算流程示意图。左侧的 $\boldsymbol{p}$ 是全局坐标系下的三维坐标点，右侧的 u_s, v_s 是该点在图像平面上的最终像素坐标。中间畸变模块中的 $r_{\mathrm{c}}^2 = u_{\mathrm{c}}^2 + v_{\mathrm{c}}^2$

现在，我们给出了它的详细参数化过程。具体地说，这里的 $\boldsymbol{x}$ 指代此时相机的位姿，即外参 $\boldsymbol{R}, \boldsymbol{t}$，它对应的李群为 $\boldsymbol{T}$，李代数为 $\boldsymbol{\xi}$。路标 $\boldsymbol{y}$ 即这里的三维点 $\boldsymbol{p}$，而观测数据则是像素坐标 $\boldsymbol{z} \overset{\mathrm{def}}{=} [u_s, v_s]^{\mathrm{T}}$。以最小二乘的角度来考虑，那么可以列写关于此次观测的误差：

$$\boldsymbol{e} = \boldsymbol{z} - h(\boldsymbol{T}, \boldsymbol{p}). \tag{9.41}$$

然后，把其他时刻的观测量也考虑进来，我们可以给误差添加一个下标。设 $\boldsymbol{z}_{ij}$ 为在位姿 $\boldsymbol{T}_i$ 处观察路标 $\boldsymbol{p}_j$ 产生的数据，那么整体的**代价函数**为

$$\frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{n}\|\boldsymbol{e}_{ij}\|^2 = \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{n}\|\boldsymbol{z}_{ij} - h(\boldsymbol{T}_i, \boldsymbol{p}_j)\|^2. \tag{9.42}$$

对这个最小二乘进行求解，相当于对位姿和路标同时做了调整，也就是所谓的 BA。接下来，我们会根据该目标函数和第 6 讲介绍的非线性优化内容，逐步深入地探讨该模型的求解。

9.2.2　BA 的求解

观察 9.2.1 节中的观测模型 $h(\boldsymbol{T}, \boldsymbol{p})$，很容易判断该函数不是线性函数。所以我们希望使用 6.2 节介绍的一些非线性优化手段来优化它。根据非线性优化的思想，我们应该从某个初始值开始，不断地寻找下降方向 $\Delta\boldsymbol{x}$，来找到目标函数的最优解，即不断地求解增量方程 (6.33) 中的增

量 $\Delta \boldsymbol{x}$。尽管误差项都是针对单个位姿和路标点的，但在整体 BA 目标函数上，我们应该把自变量定义成所有待优化的变量：

$$\boldsymbol{x} = [\boldsymbol{T}_1, \ldots, \boldsymbol{T}_m, \boldsymbol{p}_1, \ldots, \boldsymbol{p}_n]^{\mathrm{T}}. \tag{9.43}$$

相应地，增量方程中的 $\Delta \boldsymbol{x}$ 则是对整体自变量的增量。在这个意义下，当我们给自变量一个增量时，目标函数变为

$$\frac{1}{2}\left\|f(\boldsymbol{x}+\Delta \boldsymbol{x})\right\|^2 \approx \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{n}\left\|\boldsymbol{e}_{ij}+\boldsymbol{F}_{ij}\Delta \boldsymbol{\xi}_i+\boldsymbol{E}_{ij}\Delta \boldsymbol{p}_j\right\|^2. \tag{9.44}$$

其中，$\boldsymbol{F}_{ij}$ 表示整个代价函数在当前状态下对相机姿态的偏导数，而 $\boldsymbol{E}_{ij}$ 表示该函数对路标点位置的偏导。我们曾在 7.7.3 节中介绍了它们的具体形式，所以这里就不再展开推导了。现在，把相机位姿变量放在一起：

$$\boldsymbol{x}_{\mathrm{c}} = [\boldsymbol{\xi}_1, \boldsymbol{\xi}_2, \ldots, \boldsymbol{\xi}_m]^{\mathrm{T}} \in \mathbb{R}^{6m}, \tag{9.45}$$

并把空间点的变量也放在一起：

$$\boldsymbol{x}_p = [\boldsymbol{p}_1, \boldsymbol{p}_2, \ldots, \boldsymbol{p}_n]^{\mathrm{T}} \in \mathbb{R}^{3n}, \tag{9.46}$$

那么，式 (9.44) 可以简化表达如下：

$$\frac{1}{2}\left\|f(\boldsymbol{x}+\Delta \boldsymbol{x})\right\|^2 = \frac{1}{2}\left\|\boldsymbol{e}+\boldsymbol{F}\Delta \boldsymbol{x}_c+\boldsymbol{E}\Delta \boldsymbol{x}_p\right\|^2. \tag{9.47}$$

需要注意的是，该式从一个由很多个小型二次项之和，变成矩阵形式。这里的雅可比矩阵 $\boldsymbol{E}$ 和 $\boldsymbol{F}$ 必须是整体目标函数对整体变量的导数，它将是一个很大块的矩阵，而里头每个小分块，需要由每个误差项的导数 $\boldsymbol{F}_{ij}$ 和 $\boldsymbol{E}_{ij}$ “拼凑”起来。然后，无论我们使用高斯牛顿法还是列文伯格—马夸尔特方法，最后都将面对增量线性方程：

$$\boldsymbol{H}\Delta \boldsymbol{x} = \boldsymbol{g}. \tag{9.48}$$

根据第 6 讲的知识，我们知道高斯牛顿法和列文伯格—马夸尔特方法的主要差别在于，这里的 $\boldsymbol{H}$ 是取 $\boldsymbol{J}^{\mathrm{T}}\boldsymbol{J}$ 还是 $\boldsymbol{J}^{\mathrm{T}}\boldsymbol{J}+\lambda \boldsymbol{I}$ 的形式。由于我们把变量归类成了位姿和空间点两种，所以雅可比矩阵可以分块为

$$\boldsymbol{J} = [\boldsymbol{F}\ \boldsymbol{E}]. \tag{9.49}$$

那么，以高斯牛顿法为例，则 $\boldsymbol{H}$ 矩阵为

$$\boldsymbol{H} = \boldsymbol{J}^{\mathrm{T}}\boldsymbol{J} = \begin{bmatrix} \boldsymbol{F}^{\mathrm{T}}\boldsymbol{F} & \boldsymbol{F}^{\mathrm{T}}\boldsymbol{E} \\ \boldsymbol{E}^{\mathrm{T}}\boldsymbol{F} & \boldsymbol{E}^{\mathrm{T}}\boldsymbol{E} \end{bmatrix}. \tag{9.50}$$

当然，在列文伯格—马夸尔特方法中我们也需要计算这个矩阵。不难发现，因为考虑了所有的优化变量，所以这个线性方程的维度将非常大，包含了所有的相机位姿和路标点。尤其是在视觉 SLAM 中，一幅图像会提出数百个特征点，大大增加了这个线性方程的规模。如果直接对 $\boldsymbol{H}$ 求逆来计算增量方程，由于矩阵求逆是复杂度为 $O(n^3)$ 的操作[86]，那么消耗的计算资源会非常多。幸运的是，这里的 $\boldsymbol{H}$ 矩阵是有一定的特殊结构的。利用这个特殊结构，我们可以加速求解过程。

9.2.3　稀疏性和边缘化

21 世纪视觉 SLAM 的一个重要进展是认识到了矩阵 $\boldsymbol{H}$ 的稀疏结构，并发现该结构可以自然、显式地用图优化来表示[36, 87]。本节将详细讨论该矩阵的稀疏结构。

$\boldsymbol{H}$ 矩阵的稀疏性是由雅可比矩阵 $\boldsymbol{J}(\boldsymbol{x})$ 引起的。考虑这些代价函数当中的其中一个 $\boldsymbol{e}_{ij}$。注意，这个误差项只描述了在 $\boldsymbol{T}_i$ 看到 $\boldsymbol{p}_j$ 这件事，只涉及第 i 个相机位姿和第 j 个路标点，对其余部分的变量的导数都为 0。所以该误差项对应的雅可比矩阵有下面的形式：

$$\boldsymbol{J}_{ij}(\boldsymbol{x}) = \left(\boldsymbol{0}_{2\times 6}, \ldots \boldsymbol{0}_{2\times 6}, \frac{\partial \boldsymbol{e}_{ij}}{\partial \boldsymbol{T}_i}, \boldsymbol{0}_{2\times 6}, \ldots \boldsymbol{0}_{2\times 3}, \ldots \boldsymbol{0}_{2\times 3}, \frac{\partial \boldsymbol{e}_{ij}}{\partial \boldsymbol{p}_j}, \boldsymbol{0}_{2\times 3}, \ldots \boldsymbol{0}_{2\times 3}\right). \tag{9.51}$$

其中 $\boldsymbol{0}_{2\times 6}$ 表示维度为 2×6 的 $\boldsymbol{0}$ 矩阵，同理，$\boldsymbol{0}_{2\times 3}$ 也是一样的。该误差项对相机姿态的偏导 $\partial \boldsymbol{e}_{ij}/\partial \boldsymbol{\xi}_i$ 维度为 2×6，对路标点的偏导 $\partial \boldsymbol{e}_{ij}/\partial \boldsymbol{p}_j$ 维度是 2×3。这个误差项的雅可比矩阵，除了这两处为非零块，其余地方都为零。这体现了该误差项与其他路标和轨迹无关的特性。从图优化角度来说，这条观测边只和两个顶点有关。那么，它对增量方程有何影响？$\boldsymbol{H}$ 矩阵为什么会产生稀疏性呢？

以图 9-3 为例，我们设 $\boldsymbol{J}_{ij}$ 只在 i, j 处有非零块，那么它对 $\boldsymbol{H}$ 的贡献为 $\boldsymbol{J}_{ij}^{\mathrm{T}}\boldsymbol{J}_{ij}$，具有图上所画的稀疏形式。这个 $\boldsymbol{J}_{ij}^{\mathrm{T}}\boldsymbol{J}_{ij}$ 矩阵也仅有 4 个非零块，位于 $(i,i),(i,j),(j,i),(j,j)$。对于整体的 $\boldsymbol{H}$，有

$$\boldsymbol{H} = \sum_{i,j} \boldsymbol{J}_{ij}^{\mathrm{T}}\boldsymbol{J}_{ij}, \tag{9.52}$$

请注意，i 在所有相机位姿中取值，j 在所有路标点中取值。我们把 $\boldsymbol{H}$ 进行分块：

$$\boldsymbol{H} = \begin{bmatrix} \boldsymbol{H}_{11} & \boldsymbol{H}_{12} \\ \boldsymbol{H}_{21} & \boldsymbol{H}_{22} \end{bmatrix}. \tag{9.53}$$

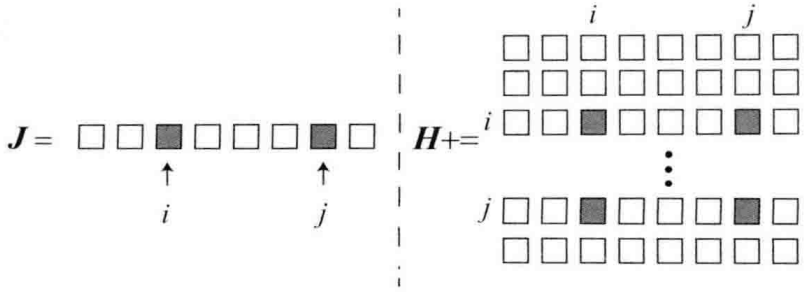


图 9-3 当某个误差项 $\boldsymbol{J}$ 具有稀疏性时，它对 $\boldsymbol{H}$ 的贡献也具有稀疏形式

这里，$\boldsymbol{H}_{11}$ 只和相机位姿有关，而 $\boldsymbol{H}_{22}$ 只和路标点有关。当我们遍历 i, j 时，以下事实总是成立的：

1. 不管 i, j 怎么变，$\boldsymbol{H}_{11}$ 都是对角阵，只在 $\boldsymbol{H}_{i,i}$ 处有非零块。
2. 同理，$\boldsymbol{H}_{22}$ 也是对角阵，只在 $\boldsymbol{H}_{j,j}$ 处有非零块。
3. 对于 $\boldsymbol{H}_{12}$ 和 $\boldsymbol{H}_{21}$，它们可能是稀疏的，也可能是稠密的，视具体的观测数据而定。

这显示了 $\boldsymbol{H}$ 的稀疏结构。之后对线性方程的求解中，也正需要利用它的稀疏结构。也许读者还没有很好地领会这里的意思，我们举一个实例来直观地说明它的情况。假设一个场景内有 2 个相机位姿（$\boldsymbol{C}_1, \boldsymbol{C}_2$）和 6 个路标点（$\boldsymbol{P}_1, \boldsymbol{P}_2, \boldsymbol{P}_3, \boldsymbol{P}_4, \boldsymbol{P}_5, \boldsymbol{P}_6$）。这些相机和点云所对应的变量为 $\boldsymbol{T}_i, i = 1, 2$ 及 $\boldsymbol{p}_j, j = 1, \cdots, 6$。相机 $\boldsymbol{C}_1$ 观测到路标点 $\boldsymbol{P}_1, \boldsymbol{P}_2, \boldsymbol{P}_3, \boldsymbol{P}_4$，相机 $\boldsymbol{C}_2$ 观测到路标点 $\boldsymbol{P}_3, \boldsymbol{P}_4, \boldsymbol{P}_5, \boldsymbol{P}_6$。我们把这个过程画成示意图，如图 9-4 所示。相机和路标以圆形节点表示。如果 i 相机能够观测到 j 点，我们就在它们对应的节点连上一条边。

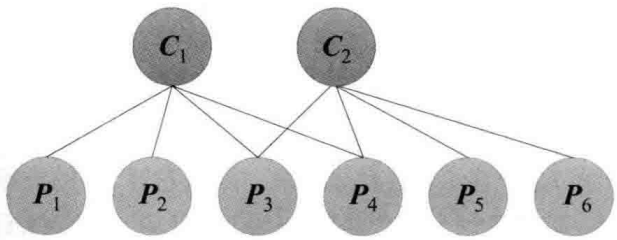


图 9-4 点和边组成的示意图。该图显示相机 $\boldsymbol{C}_1$ 观测到了路标点 $\boldsymbol{P}_1, \boldsymbol{P}_2, \boldsymbol{P}_3, \boldsymbol{P}_4$，相机 $\boldsymbol{C}_2$ 看到了路标点 $\boldsymbol{P}_3 \sim \boldsymbol{P}_6$

可以推出，场景下的 BA 目标函数应该为

$$\frac{1}{2}\left(\|\boldsymbol{e}_{11}\|^2 + \|\boldsymbol{e}_{12}\|^2 + \|\boldsymbol{e}_{13}\|^2 + \|\boldsymbol{e}_{14}\|^2 + \|\boldsymbol{e}_{23}\|^2 + \|\boldsymbol{e}_{24}\|^2 + \|\boldsymbol{e}_{25}\|^2 + \|\boldsymbol{e}_{26}\|^2\right). \tag{9.54}$$

这里的 $\boldsymbol{e}_{ij}$ 使用之前定义过的代价函数，即式 (9.42)。以 $\boldsymbol{e}_{11}$ 为例，它描述了在 $\boldsymbol{C}_1$ 看到了 $\boldsymbol{P}_1$ 这件事，与其他的相机位姿和路标无关。令 $\boldsymbol{J}_{11}$ 为 $\boldsymbol{e}_{11}$ 所对应的雅可比矩阵，不难看出 $\boldsymbol{e}_{11}$ 对相

机变量 $\boldsymbol{\xi}_2$ 和路标点 $\boldsymbol{p}_2,\cdots,\boldsymbol{p}_6$ 的偏导都为 0。我们把所有变量以 $\boldsymbol{x}=(\boldsymbol{\xi}_1,\boldsymbol{\xi}_2,\boldsymbol{p}_1,\cdots,\boldsymbol{p}_2)^{\mathrm{T}}$ 的顺序摆放，则有

$$\boldsymbol{J}_{11}=\frac{\partial \boldsymbol{e}_{11}}{\partial \boldsymbol{x}}=\left(\frac{\partial \boldsymbol{e}_{11}}{\partial \boldsymbol{\xi}_1},\boldsymbol{0}_{2\times 6},\frac{\partial \boldsymbol{e}_{11}}{\partial \boldsymbol{p}_1},\boldsymbol{0}_{2\times 3},\boldsymbol{0}_{2\times 3},\boldsymbol{0}_{2\times 3},\boldsymbol{0}_{2\times 3},\boldsymbol{0}_{2\times 3}\right). \tag{9.55}$$

为了方便表示稀疏性，我们用带有颜色的方块表示矩阵在该方块内有数值，其余没有颜色的区域表示矩阵在该处数值都为 0。那么上面的 $\boldsymbol{J}_{11}$ 则可以表示成图 9-5 所示的图案。同理，其他的雅可比矩阵也会有类似的稀疏图案。

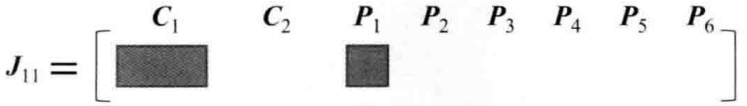


图 9-5　$\boldsymbol{J}_{11}$ 矩阵的非零块分布图。上方的标记表示矩阵该列所对应的变量。由于相机参数维数比点云参数维数大，所以 $\boldsymbol{C}_1$ 对应的矩阵块要比 $\boldsymbol{P}_1$ 对应的矩阵块宽

为了得到该目标函数对应的雅可比矩阵，我们可以将这些 $\boldsymbol{J}_{ij}$ 按照一定顺序列为向量，那么整体雅可比矩阵及相应的 $\boldsymbol{H}$ 矩阵的稀疏情况就如图 9-6 所示。

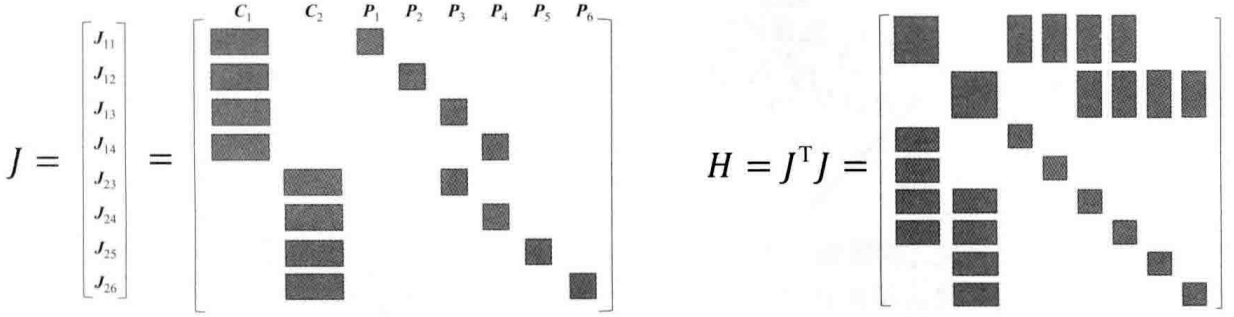


图 9-6　雅可比矩阵的稀疏性（左）和 $\boldsymbol{H}$ 矩阵的稀疏性（右），填色的方块表示矩阵在对应的矩阵块处有数值，其余没有颜色的部分表示矩阵在该处的数值始终为 0

也许你已经注意到了，图 9-4 对应的**邻接矩阵**（Adjacency Matrix）[①] 和图 9-6 中的 $\boldsymbol{H}$ 矩阵，除了对角元素外的其余部分有着完全一致的结构。事实上的确如此。上面的 $\boldsymbol{H}$ 矩阵一共有 8×8 个矩阵块，对于 $\boldsymbol{H}$ 矩阵中处于非对角线的矩阵块来说，如果该矩阵块非零，则其位置所对应的变量之间在图中会存在一条边，我们可以从图 9-7 中清晰地看到这一点。所以，$\boldsymbol{H}$ 矩阵中的非对角部分的非零矩阵块可以理解为其对应的两个变量之间存在联系，或者可以称之为约束。于是，我们发现图优化结构与增量方程的稀疏性存在着明显的联系。

①所谓邻接矩阵是这样一种矩阵，它的第 i,j 个元素描述了节点 i 和 j 是否存在一条边。如果存在此边，则设这个元素为 1，否则设为 0。

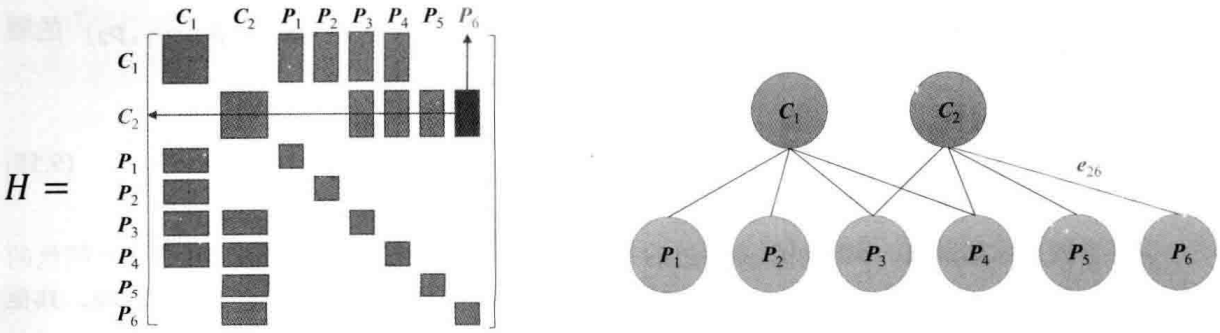


图 9-7 $\boldsymbol{H}$ 矩阵中非零矩阵块和图中边的对应关系。如左图 $\boldsymbol{H}$ 矩阵中右侧的红色矩阵块，表示在右图中其对应的变量 $\boldsymbol{C}_2$ 和 $\boldsymbol{P}_6$ 之间存在一条边 $\boldsymbol{e}_{26}$（见彩插）

现在考虑更一般的情况，假如我们有 m 个相机位姿，n 个路标点。由于通常路标的数量远远多于相机，于是有 $n \gg m$。由上面的推理可知，一般情况下的 $\boldsymbol{H}$ 矩阵如图 9-8 所示。它的左上角块显得非常小，而右下角的对角块占据了大量地方。除此之外，非对角部分则分布着散乱的观测数据。由于它的形状很像箭头，又称为箭头形（Arrow-like）矩阵[6]。同时，它也很像一把镐子，所以笔者也称其为镐形矩阵①。

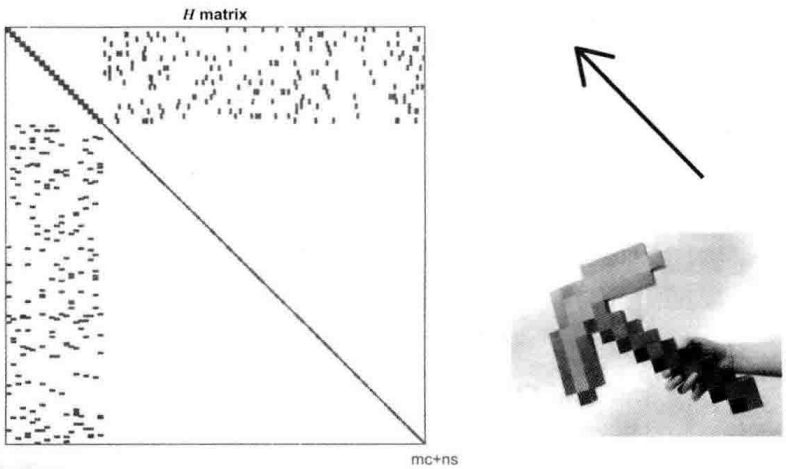


图 9-8 一般情况下的 $\boldsymbol{H}$ 矩阵

对于具有这种稀疏结构的 $\boldsymbol{H}$，线性方程 $\boldsymbol{H}\Delta\boldsymbol{x} = \boldsymbol{g}$ 的求解会有什么不同呢？现实当中存在着若干种利用 $\boldsymbol{H}$ 的稀疏性加速计算的方法，本节介绍视觉 SLAM 里一种最常用的手段：Schur 消元。在 SLAM 研究中也称为 Marginalization（边缘化）。

仔细观察图 9-8，我们不难发现这个矩阵可以分成 4 个块，和式 (9.53) 一致。左上角为对角块矩阵，每个对角块元素的维度与相机位姿的维度相同，且是一个对角块矩阵。右下角也是对角

①这是一个玩笑，请不要在正式的学术论文里这样写。

块矩阵，每个对角块的维度是路标的维度。非对角块的结构与具体观测数据相关。我们首先将这个矩阵按照图 9-9 所示的方式做区域划分，读者不难发现，这 4 个区域正好对应了公式 (9.50) 中的 4 个矩阵块。为了后续分析方便，我们记这 4 个块为 $\boldsymbol{B}, \boldsymbol{E}, \boldsymbol{E}^{\mathrm{T}}, \boldsymbol{C}$。

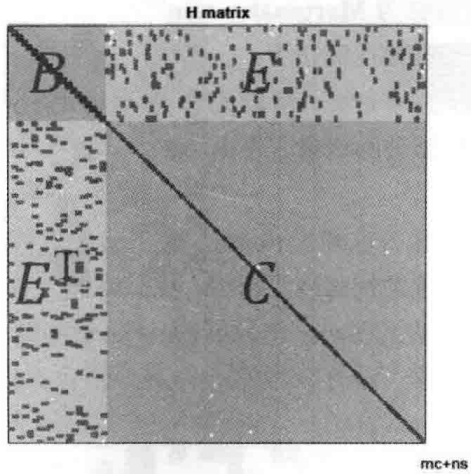


图 9-9　$\boldsymbol{H}$ 矩阵的区域划分

于是，对应的线性方程组也可以由 $\boldsymbol{H}\Delta\boldsymbol{x} = \boldsymbol{g}$ 变为如下形式：

$$\begin{bmatrix} \boldsymbol{B} & \boldsymbol{E} \\ \boldsymbol{E}^{\mathrm{T}} & \boldsymbol{C} \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{x}_{\mathrm{c}} \\ \Delta\boldsymbol{x}_p \end{bmatrix} = \begin{bmatrix} \boldsymbol{v} \\ \boldsymbol{w} \end{bmatrix}. \tag{9.56}$$

其中 $\boldsymbol{B}$ 是对角块矩阵，每个对角块的维度和相机参数的维度相同，对角块的个数是相机变量的个数。由于路标数量会远远大于相机变量个数，所以 $\boldsymbol{C}$ 往往也远大于 $\boldsymbol{B}$。三维空间中每个路标点为三维，于是 $\boldsymbol{C}$ 矩阵为对角块矩阵，每个块为 3×3 矩阵。对角块矩阵求逆的难度远小于对一般矩阵的求逆难度，因为我们只需要对那些对角线矩阵块分别求逆即可。考虑到这个特性，我们对线性方程组进行高斯消元，目标是消去右上角的非对角部分 $\boldsymbol{E}$，得

$$\begin{bmatrix} \boldsymbol{I} & -\boldsymbol{E}\boldsymbol{C}^{-1} \\ \boldsymbol{0} & \boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{B} & \boldsymbol{E} \\ \boldsymbol{E}^{\mathrm{T}} & \boldsymbol{C} \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{x}_{\mathrm{c}} \\ \Delta\boldsymbol{x}_p \end{bmatrix} = \begin{bmatrix} \boldsymbol{I} & -\boldsymbol{E}\boldsymbol{C}^{-1} \\ \boldsymbol{0} & \boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{v} \\ \boldsymbol{w} \end{bmatrix}. \tag{9.57}$$

整理，得

$$\begin{bmatrix} \boldsymbol{B} - \boldsymbol{E}\boldsymbol{C}^{-1}\boldsymbol{E}^{\mathrm{T}} & \boldsymbol{0} \\ \boldsymbol{E}^{\mathrm{T}} & \boldsymbol{C} \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{x}_{\mathrm{c}} \\ \Delta\boldsymbol{x}_p \end{bmatrix} = \begin{bmatrix} \boldsymbol{v} - \boldsymbol{E}\boldsymbol{C}^{-1}\boldsymbol{w} \\ \boldsymbol{w} \end{bmatrix}. \tag{9.58}$$

消元之后，方程组第一行变成和 $\Delta\boldsymbol{x}_p$ 无关的项。单独把它拿出来，得到关于位姿部分的增

量方程：

$$\left[\boldsymbol{B}-\boldsymbol{E}\boldsymbol{C}^{-1}\boldsymbol{E}^{\mathrm{T}}\right]\Delta\boldsymbol{x}_{\mathrm{c}}=\boldsymbol{v}-\boldsymbol{E}\boldsymbol{C}^{-1}\boldsymbol{w}. \tag{9.59}$$

这个线性方程的维度和 $\boldsymbol{B}$ 矩阵一样。我们的做法是先求解这个方程，然后把解得的 $\Delta\boldsymbol{x}_{\mathrm{c}}$ 代入原方程，求解 $\Delta\boldsymbol{x}_p$。这个过程称为 **Marginalization**[83]，或者 **Schur 消元**（Schur Elimination）。相比于直接解线性方程的做法，它的优势在于：

1. 在消元过程中，由于 $\boldsymbol{C}$ 为对角块，所以 $\boldsymbol{C}^{-1}$ 容易解出。
2. 求解了 $\Delta\boldsymbol{x}_{\mathrm{c}}$ 之后，路标部分的增量方程由 $\Delta\boldsymbol{x}_p=\boldsymbol{C}^{-1}(\boldsymbol{w}-\boldsymbol{E}^{\mathrm{T}}\Delta\boldsymbol{x}_{\mathrm{c}})$ 给出。这依然用到了 $\boldsymbol{C}^{-1}$ 易于求解的特性。

于是，边缘化的主要计算量在于求解式 (9.59)。关于这个方程，我们能说的就不多了。它仅是一个普通的线性方程，没有特殊的结构可以利用。我们将此方程的系数记为 $\boldsymbol{S}$，它的稀疏性如何呢？图 9-10 显示了对 $\boldsymbol{H}$ 矩阵进行 Schur 消元后的一个 $\boldsymbol{S}$ 实例，可以看到它的稀疏性是不规则的。



图 9-10　对 $\boldsymbol{H}$ 矩阵进行 Schur 消元后的 $\boldsymbol{S}$ 矩阵的稀疏状态

前面说到，$\boldsymbol{H}$ 矩阵的非对角块处的非零元素对应着相机和路标的关联。那么，进行了 Schur 消元后 $\boldsymbol{S}$ 的稀疏性是否具有物理意义呢？答案是肯定的。此处我们不加证明地说，$\boldsymbol{S}$ 矩阵的非对角线上的非零矩阵块，表示了该处对应的两个相机变量之间存在着共同观测的路标点，有时称为共视（Co-visibility）。反之，如果该块为零，则表示这两个相机没有共同观测。例如，图 9-11 所示的稀疏矩阵，左上角前 4×4 个矩阵块可以表示对应的相机变量 $\boldsymbol{C}_1,\boldsymbol{C}_2,\boldsymbol{C}_3,\boldsymbol{C}_4$ 之间有共同观测。

于是，$\boldsymbol{S}$ 矩阵的稀疏性结构当取决于实际观测的结果，我们无法提前预知。在实践中，例如 ORB-SLAM[88] 中的 Local Mapping 环节，在做 BA 的时候刻意选择那些具有共同观测的帧作为关键帧，在这种情况下，Schur 消元后得到的 $\boldsymbol{S}$ 就是稠密矩阵。不过，由于这个模块并不是实时执行，所以这种做法也是可以接受的。但是有另一些方法，例如 DSO[70]、OKVIS[89] 等，它们采用了滑动窗口（Sliding Window）方法。这类方法对每一帧都要求做一次 BA 来防止误差的累积，因此它们也必须采用一些技巧来保持 $\boldsymbol{S}$ 矩阵的稀疏性。如果读者希望更加深入地学习这部分内容，可以参考关于它们的论文。这里就不谈这些过于细节的事情了。

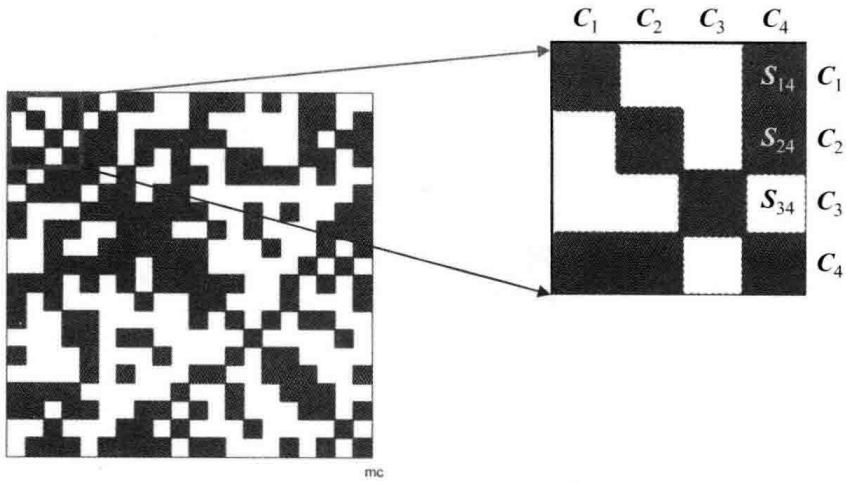


图 9-11　以 $\boldsymbol{S}$ 矩阵中前 4×4 个矩阵块为例，这个区域中的矩阵块 $\boldsymbol{S}_{14},\boldsymbol{S}_{24}$ 不为零，表示相机 $\boldsymbol{C}_4$ 和相机 $\boldsymbol{C}_1$、$\boldsymbol{C}_2$ 之间有共同观测点；而 $\boldsymbol{S}_{34}$ 为零则表示 $\boldsymbol{C}_3$ 和 $\boldsymbol{C}_4$ 之间没有共同观测的路标

从概率角度来看，我们称这一步为边缘化，是因为我们实际上把求 $(\Delta\boldsymbol{x}_c,\Delta\boldsymbol{x}_p)$ 的问题，转化成了先固定 $\Delta\boldsymbol{x}_p$，求出 $\Delta\boldsymbol{x}_c$，再求 $\Delta\boldsymbol{x}_p$ 的过程。这一步相当于做了条件概率展开：

$$P(\boldsymbol{x}_c,\boldsymbol{x}_p)=P(\boldsymbol{x}_c|\boldsymbol{x}_p)P(\boldsymbol{x}_p), \tag{9.60}$$

结果是求出了关于 $\boldsymbol{x}_p$ 的边缘分布，故称边缘化。在前面介绍的边缘化过程中，实际上我们把所有的路标点都给边缘化了。根据实际情况，我们也能选择一部分进行边缘化。同时，Schur 消元只是实现边缘化的其中一种方式，同样可以使用 Cholesky 分解进行边缘化。

读者可能会继续问，在进行了 Schur 消元后，我们还需要求解线性方程组 (9.59)，对它的求解是否有什么技巧呢？很遗憾，这部分就属于传统的矩阵数值求解，通常是用分解来计算的。不管采用哪种求解办法，我们都建议利用 $\boldsymbol{H}$ 的稀疏性进行 Schur 消元。不光是因为这样可以提高速度，也因为消元后的 $\boldsymbol{S}$ 矩阵的条件数往往比之前的 $\boldsymbol{H}$ 矩阵要小。Schur 消元也并不意味着将所有路标消元，将相机变量消元也是 SLAM 中采用的手段。

9.2.4　鲁棒核函数

在前面的 BA 问题中，我们将最小化误差项的二范数平方和作为目标函数。这种做法虽然很直观，但存在一个严重的问题：如果出于误匹配等原因，某个误差项给的数据是错误的，会发生什么呢？我们把一条原本不应该加到图中的边给加进去了，然而优化算法并不能辨别出这是个错误数据，它会把所有的数据都当作误差来处理。在算法看来，这相当于我们突然观测到了一次很不可能产生的数据。这时，在图优化中会有一条误差很大的边，它的梯度也很大，意味着调整与

它相关的变量会使目标函数下降更多。所以，算法将试图优先调整这条边所连接的节点的估计值，使它们顺应这条边的无理要求。由于这条边的误差真的很大，往往会抹平其他正确边的影响，使优化算法专注于调整一个错误的值。这显然不是我们希望看到的。

出现这种问题的原因是，当误差很大时，二范数增长得太快。于是就有了核函数的存在。核函数保证每条边的误差不会大得没边而掩盖其他的边。具体的方式是，把原先误差的二范数度量替换成一个增长没有那么快的函数，同时保证自己的光滑性质（不然无法求导）。因为它们使得整个优化结果更为稳健，所以又叫它们鲁棒核函数（Robust Kernel）。

鲁棒核函数有许多种，例如最常用的 Huber 核：

$$H(e)=\begin{cases}\frac{1}{2}e^2 & \text{当}|e|\leqslant\delta,\\ \delta\left(|e|-\frac{1}{2}\delta\right) & \text{其他}\end{cases} \tag{9.61}$$

我们看到，当误差 e 大于某个阈值 δ 后，函数增长由二次形式变成了一次形式，相当于限制了梯度的最大值。同时，Huber 核函数又是光滑的，可以很方便地求导。图 9-12 显示了 Huber 核函数与二次函数的对比，可见在误差较大时 Huber 核函数增长明显低于二次函数。

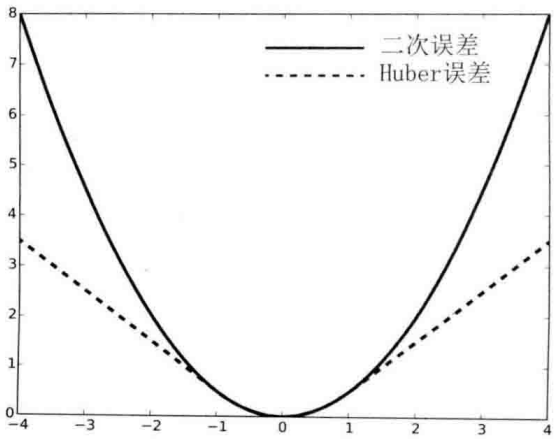


图 9-12 Huber 核函数与二次函数的对比

除了 Huber 核，还有 Cauchy 核、Tukey 核，等等，读者可以看看 g2o 和 Ceres 都提供了哪些核函数。

小结

本节我们重点介绍了 BA 中的稀疏性问题。不过，实践中，多数软件库已经为我们实现了细节操作，而我们需要做的主要是构造 BA 问题，设置 Schur 消元，然后调用稠密或者稀疏矩阵求

解器对变量进行优化。如果读者希望更深入地了解 BA，可以在阅读完本节的基础上，进一步学习参考文献 [34]。

下面的两节，我们将使用 Ceres 和 g2o 两个库来做 BA。为了体现出它们的区别，我们会使用一个公开数据集 BAL[90]，并使用共用的读写代码。

9.3 实践：Ceres BA

9.3.1 BAL 数据集

我们用 BAL 数据集进行 BA 的演示实验。BAL 数据集提供若干个场景，每一个场景里的相机和路标点信息由一个文本文件给定。在本例中，使用 problem-16-22106-pre.txt 文件作为例子。该文件以行的方式存储 BA 问题的信息，详细格式见`https://grail.cs.washington.edu/projects/bal`。我们用 common.h 中定义的 BALProblem 类读入该文件的内容，然后分别用 Ceres 和 g2o 求解。

需要注意的是，BAL 数据集有其自身的特殊之处：

1. BAL 的相机内参模型由焦距 f 和畸变参数 k_1, k_2 给出。f 类似于我们提到的 f_x 和 f_y。由于照片像素基本上是正方形，所以在很多实际场合中 f_x 非常接近 f_y，用同一个值也未尝不可。此外，这个模型中没有 c_x, c_y，因为存储的数据已经去掉了这两个值。
2. 因为 BAL 数据在投影时假设投影平面在相机光心之后，所以按照我们之前用的模型计算，需要在投影之后乘以系数 -1。不过，大部分数据集仍使用光心前面的投影平面，我们在使用数据集之前应该仔细阅读格式说明。

用 BALProblem 类读取数据之后，我们可以调用 Normalize 函数对原始数据进行归一化，或通过 Perturb 函数给数据加上噪声。归一化是指将所有路标点的中心置零，然后做一个合适尺度的缩放。这会使优化过程中数值更稳定，防止在极端情况下处理很大或者有很大偏移的 BA 问题。

请读者自行阅读 BALProblem 类的其他接口。由于这些代码只负责读写数据等外围功能，为节省篇幅，我们不在正文中给出。在解出 BA 之后，我们还可以用该类的函数将结果写入一个 ply 文件（一种点云文件格式），然后用 Meshlab 软件进行查看。Meshlab 可以通过 apt-get 安装，在此不赘述安装方式。

9.3.2 Ceres BA 的书写

在 bundle_adjustment_ceres.cpp 文件中，我们实现了 Ceres 求解 BA 的过程。用 Ceres 的关键是定义出投影误差模型，该部分代码在 SnavelyReprojectionError.h 中给出：

slambook2/ch9/SnavelyReprojectionError.cpp（片段）

```
class SnavelyReprojectionError {
public:
    SnavelyReprojectionError(double observation_x, double observation_y) : observed_x(observation_x),
    observed_y(observation_y) {}

    template<typename T>
    bool operator()(const T *const camera,
        const T *const point,
        T *residuals) const {
        // camera[0,1,2] are the angle-axis rotation
        T predictions[2];
        CamProjectionWithDistortion(camera, point, predictions);
        residuals[0] = predictions[0] - T(observed_x);
        residuals[1] = predictions[1] - T(observed_y);

        return true;
    }

    // camera : 9 dims array
    // [0-2] : angle-axis rotation
    // [3-5] : translation
    // [6-8] : camera parameter, [6] focal length, [7-8] second and forth order radial distortion
    // point : 3D location.
    // predictions : 2D predictions with center of the image plane.
    template<typename T>
    static inline bool CamProjectionWithDistortion(const T *camera, const T *point, T *predictions) {
        // Rodrigues' formula
        T p[3];
        AngleAxisRotatePoint(camera, point, p);
        // camera[3,4,5] are the translation
        p[0] += camera[3];
        p[1] += camera[4];
        p[2] += camera[5];

        // Compute the center fo distortion
        T xp = -p[0] / p[2];
        T yp = -p[1] / p[2];

        // Apply second and fourth order radial distortion
```

```
        const T &l1 = camera[7];
        const T &l2 = camera[8];

        T r2 = xp * xp + yp * yp;
        T distortion = T(1.0) + r2 * (l1 + l2 * r2);

        const T &focal = camera[6];
        predictions[0] = focal * distortion * xp;
        predictions[1] = focal * distortion * yp;

        return true;
    }

    static ceres::CostFunction *Create(const double observed_x, const double observed_y) {
        return (new ceres::AutoDiffCostFunction<SnavelyReprojectionError, 2, 9, 3>(
            new SnavelyReprojectionError(observed_x, observed_y)));
    }

private:
    double observed_x;
    double observed_y;
};
```

该类的括号运算符实现了 Ceres 计算误差的接口，实际的计算在 CamProjectionWithDistortion 函数中。注意在 Ceres 中，我们必须以 double 数组形式存储优化变量。现在，每个相机一共有 6 维的姿态、1 维焦距和 2 维畸变参数，共 9 维参数来描述，我们在实际存储中也必须按照这个顺序。该类的静态函数 Create 作为外部调用接口，直接返回一个可自动求导的 Ceres 代价函数。我们只要调用 Create 函数，把代价函数放入 ceres::Problem 即可。

接下来，我们实现 BA 搭建和求解的部分：

slambook2/ch9/SnavelyReprojectionError.cpp（片段）

```
void SolveBA(BALProblem &bal_problem) {
    const int point_block_size = bal_problem.point_block_size();
    const int camera_block_size = bal_problem.camera_block_size();
    double *points = bal_problem.mutable_points();
    double *cameras = bal_problem.mutable_cameras();

    // Observations is 2 * num_observations long array observations
    // [u_1, u_2, ... u_n], where each u_i is two dimensional, the x
    // and y position of the observation.
    const double *observations = bal_problem.observations();
    ceres::Problem problem;
```

```
    for (int i = 0; i < bal_problem.num_observations(); ++i) {
        ceres::CostFunction *cost_function;

        // Each Residual block takes a point and a camera as input
        // and outputs a 2 dimensional Residual
        cost_function =
            SnavelyReprojectionError::Create(observations[2 * i + 0], observations[2 * i +
            1]);

        // If enabled use Huber's loss function.
        ceres::LossFunction *loss_function = new ceres::HuberLoss(1.0);

        // Each observation corresponds to a pair of a camera and a point
        // which are identified by camera_index()[i] and point_index()[i]
        // respectively.
        double *camera = cameras + camera_block_size * bal_problem.camera_index()[i];
        double *point = points + point_block_size * bal_problem.point_index()[i];

        problem.AddResidualBlock(cost_function, loss_function, camera, point);
    }

    std::cout << "Solving ceres BA ... " << endl;
    ceres::Solver::Options options;
    options.linear_solver_type = ceres::LinearSolverType::SPARSE_SCHUR;
    options.minimizer_progress_to_stdout = true;
    ceres::Solver::Summary summary;
    ceres::Solve(options, &problem, &summary);
    std::cout << summary.FullReport() << "\n";
}
```

可见问题搭建部分是相当简单的。如果要添加别的代价函数，整个流程也不会有太大的变化。最后，在 ceres::Solver::Options 中，我们可以设定求解的方法。使用 SPARSE_SCHUR 会让 Ceres 实际求解的过程和我们前面描述的一致，即先对路标部分进行 Schur 边缘化，以加速的方式求解此问题。不过，在 Ceres 中我们不能控制哪部分变量被边缘化，这是由 Ceres 求解器自动寻找并计算的。

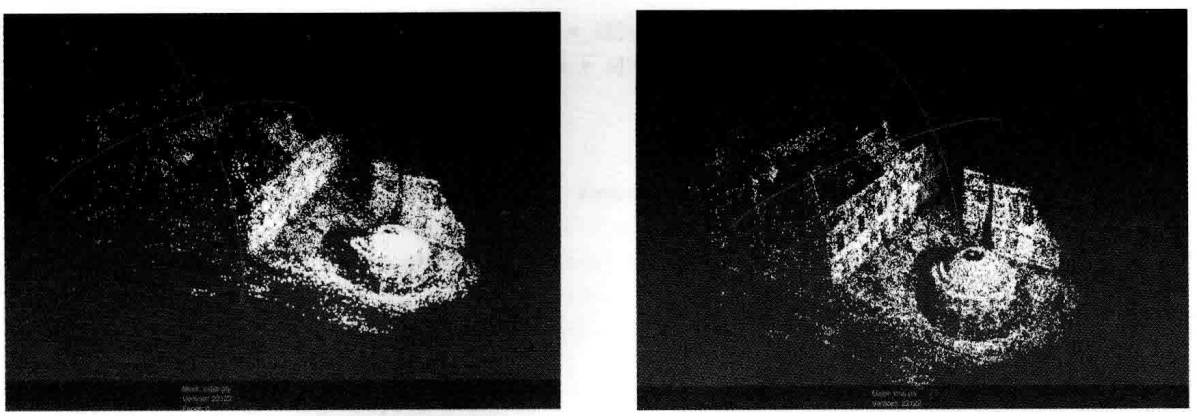
Ceres 的 BA 求解输出如下：

终端输出

```
./build/bundle_adjustment_ceres  problem-16-22106-pre.txt
Header: 16 22106 83718bal problem file loaded...
bal problem have 16 cameras and 22106 points.
```

```
Forming 83718 observations.
Solving ceres BA ...
iter      cost      cost_change  |gradient|   |step|    tr_ratio  tr_radius  ls_iter
iter_time  total_time
0  1.842900e+07    0.00e+00    2.04e+06   0.00e+00   0.00e+00  1.00e+04        0    6.10e-02
   2.24e-01
1  1.449093e+06    1.70e+07    1.75e+06   2.16e+03   1.84e+00  3.00e+04        1    1.79e-01
   4.03e-01
2  5.848543e+04    1.39e+06    1.30e+06   1.55e+03   1.87e+00  9.00e+04        1    1.56e-01
   5.59e-01
3  1.581483e+04    4.27e+04    4.98e+05   4.98e+02   1.29e+00  2.70e+05        1    1.51e-01
   7.10e-01
......
```

总体误差应该随着迭代次数的增长而不断下降。最后，我们将优化前和优化后的点云输出为 initial.ply 和 final.ply，用 Meshlab 可以直接打开这两个点云。结果图如图 9-13 所示。



（a）　　（b）

图 9-13　优化前后的可视化点云。（a）为优化前的初始值，（b）为优化后的优化值（见彩插）

9.4　实践：g2o 求解 BA

下面来考虑如何使用 g2o 求解这个 BA 问题。和以前一样，g2o 使用图模型来描述问题的结构，所以我们要用节点来表示相机和路标，然后用边来表示它们之间的观测。我们仍然使用自定义的点和边，只需覆盖一些关键函数即可。针对相机和路标，我们可以定义如下结构体，并使用 override 关键字表示对基类虚函数的覆盖：

slambook2/ch9/bundle_adjustment_g2o.cpp（片段）

```
/// 姿态和内参的结构
struct PoseAndIntrinsics {
    PoseAndIntrinsics() {}

    /// set from given data address
    explicit PoseAndIntrinsics(double *data_addr) {
        rotation = SO3d::exp(Vector3d(data_addr[0], data_addr[1], data_addr[2]));
        translation = Vector3d(data_addr[3], data_addr[4], data_addr[5]);
        focal = data_addr[6];
        k1 = data_addr[7];
        k2 = data_addr[8];
    }

    /// 将估计值放入内存
    void set_to(double *data_addr) {
        auto r = rotation.log();
        for (int i = 0; i < 3; ++i) data_addr[i] = r[i];
        for (int i = 0; i < 3; ++i) data_addr[i + 3] = translation[i];
        data_addr[6] = focal;
        data_addr[7] = k1;
        data_addr[8] = k2;
    }

    SO3d rotation;
    Vector3d translation = Vector3d::Zero();
    double focal = 0;
    double k1 = 0, k2 = 0;
};

/// 位姿加相机内参的顶点，9维，前三维为so3，接下来为t，f，k1，k2
class VertexPoseAndIntrinsics : public g2o::BaseVertex<9, PoseAndIntrinsics> {
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW;

    VertexPoseAndIntrinsics() {}

    virtual void setToOriginImpl() override {
        _estimate = PoseAndIntrinsics();
    }

    virtual void oplusImpl(const double *update) override {
        _estimate.rotation = SO3d::exp(Vector3d(update[0], update[1], update[2])) * _estimate
```

```
        .rotation;
        _estimate.translation += Vector3d(update[3], update[4], update[5]);
        _estimate.focal += update[6];
        _estimate.k1 += update[7];
        _estimate.k2 += update[8];
    }

    /// 根据估计值投影一个点
    Vector2d project(const Vector3d &point) {
        Vector3d pc = _estimate.rotation * point + _estimate.translation;
        pc = -pc / pc[2];
        double r2 = pc.squaredNorm();
        double distortion = 1.0 + r2 * (_estimate.k1 + _estimate.k2 * r2);
        return Vector2d(_estimate.focal * distortion * pc[0],
        _estimate.focal * distortion * pc[1]);
    }

    virtual bool read(istream &in) {}

    virtual bool write(ostream &out) const {}
};

class VertexPoint : public g2o::BaseVertex<3, Vector3d> {
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW;

    VertexPoint() {}

    virtual void setToOriginImpl() override {
        _estimate = Vector3d(0, 0, 0);
    }

    virtual void oplusImpl(const double *update) override {
        _estimate += Vector3d(update[0], update[1], update[2]);
    }

    virtual bool read(istream &in) {}

    virtual bool write(ostream &out) const {}
};

class EdgeProjection :
public g2o::BaseBinaryEdge<2, Vector2d, VertexPoseAndIntrinsics, VertexPoint> {
```

```
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW;

    virtual void computeError() override {
        auto v0 = (VertexPoseAndIntrinsics *) _vertices[0];
        auto v1 = (VertexPoint *) _vertices[1];
        auto proj = v0->project(v1->estimate());
        _error = proj - _measurement;
    }

    // use numeric derivatives
    virtual bool read(istream &in) {}

    virtual bool write(ostream &out) const {}
};
```

我们把旋转、平移、焦距和畸变参数定义在同一个相机顶点中，然后定义相机到路标点之间的观测边。这里，我们没有实现边的雅可比计算函数，这样 g2o 会自动提供一个数值计算的雅可比。最后，按照 BAL 中的数据，将 g2o 的优化问题搭建起来即可：

slambook2/ch9/bundle_adjustment_g2o.cpp（片段）

```
void SolveBA(BALProblem &bal_problem) {
    const int point_block_size = bal_problem.point_block_size();
    const int camera_block_size = bal_problem.camera_block_size();
    double *points = bal_problem.mutable_points();
    double *cameras = bal_problem.mutable_cameras();

    // pose dimension 9, landmark is 3
    typedef g2o::BlockSolver<g2o::BlockSolverTraits<9, 3>> BlockSolverType;
    typedef g2o::LinearSolverCSparse<BlockSolverType::PoseMatrixType> LinearSolverType;
    // use LM
    auto solver = new g2o::OptimizationAlgorithmLevenberg(
    g2o::make_unique<BlockSolverType>(g2o::make_unique<LinearSolverType>()));
    g2o::SparseOptimizer optimizer;
    optimizer.setAlgorithm(solver);
    optimizer.setVerbose(true);

    /// build g2o problem
    const double *observations = bal_problem.observations();
    // vertex
    vector<VertexPoseAndIntrinsics *> vertex_pose_intrinsics;
    vector<VertexPoint *> vertex_points;
    for (int i = 0; i < bal_problem.num_cameras(); ++i) {
```

```
        VertexPoseAndIntrinsics *v = new VertexPoseAndIntrinsics();
        double *camera = cameras + camera_block_size * i;
        v->setId(i);
        v->setEstimate(PoseAndIntrinsics(camera));
        optimizer.addVertex(v);
        vertex_pose_intrinsics.push_back(v);
    }
    for (int i = 0; i < bal_problem.num_points(); ++i) {
        VertexPoint *v = new VertexPoint();
        double *point = points + point_block_size * i;
        v->setId(i + bal_problem.num_cameras());
        v->setEstimate(Vector3d(point[0], point[1], point[2]));
        // g2o在BA中需要手动设置待Marg的顶点
        v->setMarginalized(true);
        optimizer.addVertex(v);
        vertex_points.push_back(v);
    }

    // edge
    for (int i = 0; i < bal_problem.num_observations(); ++i) {
        EdgeProjection *edge = new EdgeProjection;
        edge->setVertex(0, vertex_pose_intrinsics[bal_problem.camera_index()[i]]);
        edge->setVertex(1, vertex_points[bal_problem.point_index()[i]]);
        edge->setMeasurement(Vector2d(observations[2 * i + 0], observations[2 * i + 1]));
        edge->setInformation(Matrix2d::Identity());
        edge->setRobustKernel(new g2o::RobustKernelHuber());
        optimizer.addEdge(edge);
    }

    optimizer.initializeOptimization();
    optimizer.optimize(40);

    // set to bal problem
    for (int i = 0; i < bal_problem.num_cameras(); ++i) {
        double *camera = cameras + camera_block_size * i;
        auto vertex = vertex_pose_intrinsics[i];
        auto estimate = vertex->estimate();
        estimate.set_to(camera);
    }
    for (int i = 0; i < bal_problem.num_points(); ++i) {
        double *point = points + point_block_size * i;
        auto vertex = vertex_points[i];
        for (int k = 0; k < 3; ++k) point[k] = vertex->estimate()[k];
```

```
    }
}
```

以上定义了本问题中使用的节点和边。下面需要根据 BALProblem 类中的实际数据生成一些节点和边，交给 g2o 进行优化。值得注意的是，为了充分利用 BA 中的稀疏性，需要在这里将路标中的 setMarginalized 属性设置为 true。代码的主要片段如下：

slambook2/ch9/bundle_adjustment_g2o（片段）

```
void SolveBA(BALProblem &bal_problem) {
    const int point_block_size = bal_problem.point_block_size();
    const int camera_block_size = bal_problem.camera_block_size();
    double *points = bal_problem.mutable_points();
    double *cameras = bal_problem.mutable_cameras();

    // pose dimension 9, landmark is 3
    typedef g2o::BlockSolver<g2o::BlockSolverTraits<9, 3>> BlockSolverType;
    typedef g2o::LinearSolverCSparse<BlockSolverType::PoseMatrixType> LinearSolverType;
    // use LM
    auto solver = new g2o::OptimizationAlgorithmLevenberg(
    g2o::make_unique<BlockSolverType>(g2o::make_unique<LinearSolverType>()));
    g2o::SparseOptimizer optimizer;
    optimizer.setAlgorithm(solver);
    optimizer.setVerbose(true);

    /// build g2o problem
    const double *observations = bal_problem.observations();
    // vertex
    vector<VertexPoseAndIntrinsics *> vertex_pose_intrinsics;
    vector<VertexPoint *> vertex_points;
    for (int i = 0; i < bal_problem.num_cameras(); ++i) {
        VertexPoseAndIntrinsics *v = new VertexPoseAndIntrinsics();
        double *camera = cameras + camera_block_size * i;
        v->setId(i);
        v->setEstimate(PoseAndIntrinsics(camera));
        optimizer.addVertex(v);
        vertex_pose_intrinsics.push_back(v);
    }
    for (int i = 0; i < bal_problem.num_points(); ++i) {
        VertexPoint *v = new VertexPoint();
        double *point = points + point_block_size * i;
        v->setId(i + bal_problem.num_cameras());
        v->setEstimate(Vector3d(point[0], point[1], point[2]));
        // g2o在BA中需要手动设置待Marg的顶点
```

```
        v->setMarginalized(true);
        optimizer.addVertex(v);
        vertex_points.push_back(v);
    }

    // edge
    for (int i = 0; i < bal_problem.num_observations(); ++i) {
        EdgeProjection *edge = new EdgeProjection;
        edge->setVertex(0, vertex_pose_intrinsics[bal_problem.camera_index()[i]]);
        edge->setVertex(1, vertex_points[bal_problem.point_index()[i]]);
        edge->setMeasurement(Vector2d(observations[2 * i + 0], observations[2 * i + 1]));
        edge->setInformation(Matrix2d::Identity());
        edge->setRobustKernel(new g2o::RobustKernelHuber());
        optimizer.addEdge(edge);
    }

    optimizer.initializeOptimization();
    optimizer.optimize(40);

    // set to bal problem
    for (int i = 0; i < bal_problem.num_cameras(); ++i) {
        double *camera = cameras + camera_block_size * i;
        auto vertex = vertex_pose_intrinsics[i];
        auto estimate = vertex->estimate();
        estimate.set_to(camera);
    }
    for (int i = 0; i < bal_problem.num_points(); ++i) {
        double *point = points + point_block_size * i;
        auto vertex = vertex_points[i];
        for (int k = 0; k < 3; ++k) point[k] = vertex->estimate()[k];
    }
}
```

g2o 和 Ceres 的一大不同点是，在使用稀疏优化时，g2o 必须手动设置哪些顶点为边缘化顶点，否则就会报运行时错误（读者可以尝试注释掉 v->setMarginalized(true) 这一行）。其余地方和 Ceres 实验大同小异，我们就不多加介绍了。g2o 实验也会输出优化前后点云，供对比查看。

9.5　小结

本讲比较深入地探讨了状态估计问题与图优化的求解。我们看到在经典模型中 SLAM 可以看成状态估计问题。如果我们假设了马尔可夫性，只考虑当前状态，则得到以 EKF 为代表的滤

波器模型。如若不然，我们也可以选择考虑所有的运动和观测，它们构成一个最小二乘问题。在只有观测方程的情况下，这个问题称为 BA，并可利用非线性优化方法求解。我们仔细讨论了求解过程中的稀疏性问题，指出了该问题与图优化之间的联系。最后，我们演示了如何使用 g2o 和 Ceres 库求解同一个优化问题，让读者对 BA 有了一个直观的认识。

习题

1. 证明式 (9.25) 成立。提示：你可能会用到 SMW 公式，参考文献 [6, 76]。
2. 对比使用 g2o 和 Ceres 优化后目标函数的数值。指出为什么两者在 Meshlab 中效果一样但数值不同。
3. 对 Ceres 中的部分点云进行 Schur 消元，看看结果会有什么区别。
4. 证明 $\boldsymbol{S}$ 矩阵为半正定矩阵。
5. 阅读参考文献 [36]，看看 g2o 对核函数是如何处理的。与 Ceres 中的 Loss function 有何联系？
6.* 在两个示例中，我们优化了相机位姿、以 f, k_1, k_2 为参数的相机内参及路标点。请考虑使用第 5 讲介绍的完整的相机模型进行优化，即，至少考虑 $f_x, f_y, p_1, p_2, k_1, k_2$ 这些量。修改现在的 Ceres 和 g2o 程序以完成实验。

第 10 讲

后端 2

主要目标

1. 理解滑动窗口优化。
2. 理解位姿图优化。
3. 理解带 IMU 紧耦合的优化。
4. 通过实验掌握 g2o 的位姿图。

第 9 讲我们重点介绍了以 BA 为主的图优化。BA 能精确地优化每个相机位姿与特征点位置。不过在更大的场景中，大量特征点的存在会严重降低计算效率，导致计算量越来越大，以至于无法实时化。本讲的第一部分将介绍一种简化的 BA：位姿图。

10.1 滑动窗口滤波和优化

10.1.1 实际环境下的 BA 结构

带有相机位姿和空间点的图优化称为 BA，它能够有效地求解大规模的定位与建图问题。这在 SfM 问题中十分有用，但是在 SLAM 过程中，我们往往需要控制 BA 的规模，保持计算的实时性。倘若计算能力无限，那不妨每时每刻都计算整个 BA——但是那不符合现实需要。现实条件是，我们必须限制后端的计算时间，比如 BA 规模不能超过 1 万个路标点，迭代不超过 20 次，用时不超过 0.5 秒，等等。像 SfM 那样用一周时间重建一个城市地图的算法，在 SLAM 里不见得有效。

控制计算规模的做法有很多，比如从连续的视频中抽出一部分作为**关键帧**[89]，仅构造关键帧与路标点之间的 BA，于是非关键帧只用于定位，对建图则没有贡献。即便如此，随着时间的流逝，关键帧数量会越来越多，地图规模也将不断增长。像 BA 这样的批量优化方法，计算效率会（令人担忧地）不断下降。为了避免这种情况，我们需要用一定手段控制后端 BA 的规模。这些手段可以是理论上的，也可以是工程上的。

例如，最简单的控制 BA 规模的思路，是仅保留离当前时刻最近的 N 个关键帧，去掉时间上更早的关键帧。于是，我们的 BA 将被固定在一个时间窗口内，离开这个窗口的则被丢弃。这种方法称为滑动窗口法[91]。当然，取这 N 个关键帧的具体方法可以有一些改变，例如，不见得必须取时间上最近的，而可以按照某种原则，取时间上靠近，空间上又可以展开的关键帧，从而保证相机即使在停止不动时，BA 的结构也不至于缩成一团（这容易导致一些糟糕的退化情况）。如果我们在帧与帧的结构上再考虑得深一些，也可以像 ORB-SLAM2[88] 那样，定义一种称为“共视图”（Covisibility graph）的结构（如图 10-1 所示）。所谓共视图，就是指那些“与现在的相机存在共同观测的关键帧构成的图）。于是，在 BA 优化时，我们按照某些原则在共视图内取一些关键帧和路标进行优化，例如，仅优化与当前帧有 20 个以上共视路标的关键帧，其余部分固定不变。当共视图关系能够正确构造的时候，基于共视图的优化也会在更长时间内保持最优。

滑动窗口也好，共视图也好，大体而言，都是我们对实时计算的某种工程上的折中。不过在理论上，它们也引入了一个新问题：刚才我们谈到要“丢弃”滑动窗口之外，或者“固定”共视图之外的变量，这个“丢弃”和“固定”具体怎样操作呢？“固定”似乎很容易理解，我们只需将共视图之外的关键帧估计值保持不变即可。但是“丢弃”，是指完全弃置不用，即窗口外的变量完全不对窗口内的变量产生任何影响，还是说窗口外的数据**应该**对窗口内的有一些影响，但实际上被我们忽略了？如果有影响，这种影响应该是什么样子？它够不够明显，能不能忽略？

接下来，我们就要谈谈这些问题。它们在理论上应该如何处理，以及在工程上能不能做一些简化手段。

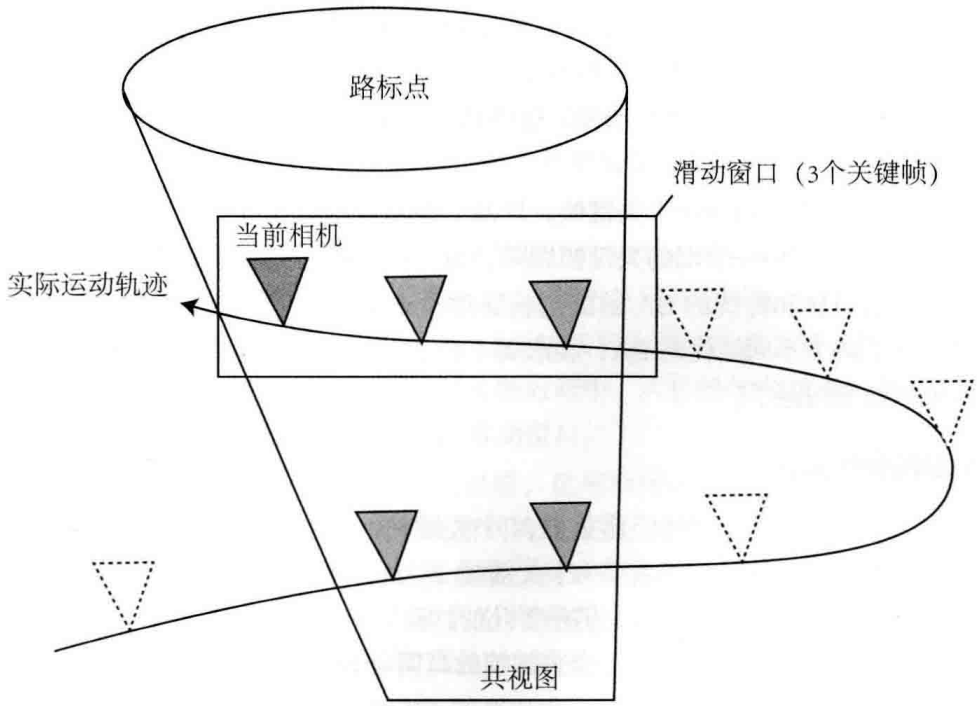


图 10-1　滑动窗口和共视图的示意图

10.1.2　滑动窗口法

现在考虑一个滑动窗口。假设这个窗口内有 N 个关键帧，它们的位姿表达为：

$$\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N,$$

我们假设它们在向量空间，即用李代数表达，那么，关于这几个关键帧，我们能谈论些什么呢？

显然，我们关心这几个关键帧的位置在哪里，以及它们的不确定度如何，这对应着它们在高斯分布假设下的均值协方差。如果这几个关键帧还对应着一个局部地图，则我们可以顺带着问整个局部系统的均值和方差应该是多少。设这个滑动窗口中还有 M 个路标点：$\boldsymbol{y}_1, \ldots, \boldsymbol{y}_N$，它们与 N 个关键帧组成了局部地图。显然我们可以用第 9 讲介绍的 BA 方法处理这个滑动窗口，包括建立图优化模型，构建整体的 Hessian 矩阵，然后边缘化所有路标点来加速求解。在边缘化时，我们考虑关键帧的位姿，即

$$[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]^{\mathrm{T}} \sim N([\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_N]^{\mathrm{T}}, \boldsymbol{\Sigma}).$$

其中 $\boldsymbol{\mu}_k$ 为第 k 个关键帧的位姿均值，$\boldsymbol{\Sigma}$ 为所有关键帧的协方差矩阵，那么显然，均值部分就是

指 BA 迭代之后的结果，而 $\boldsymbol{\Sigma}$ 就是对整个 BA 的 $\boldsymbol{H}$ 矩阵进行边缘化之后的结果，即第 9 讲提到的矩阵 $\boldsymbol{S}$。我们认为读者已经熟悉这个流程了。

在滑动窗口中，当窗口结构发生改变，这些状态变量应该如何变化？这件事情可以分成两部分讨论：

1. 我们需要在窗口中新增一个关键帧，以及它观测到的路标点。
2. 我们需要把窗口中一个旧的关键帧删除，也可能删除它观测到的路标点。

这时，滑动窗口法和传统的 BA 的区别就显现出来了。显然，如果按照传统的 BA 来处理，那么这仅仅对应于两个不同结构的 BA，在求解上没有任何差别。但在滑动窗口的情况下，我们就要讨论具体的细节问题了。

新增一个关键帧和路标点

考虑在上个时刻，滑动窗口已经建立了 N 个关键帧，我们也已知道它们服从某个高斯分布，其均值和方差如前所述。此时，新来了一个关键帧 $\boldsymbol{x}_{N+1}$，那么整个问题中的变量变为 $N+1$ 个关键帧和更多路标点的集合。这实际上仍是平凡的，我们只需按照正常的 BA 流程处理即可。对所有点进行边缘化时，即得到这 $N+1$ 个关键帧的高斯分布参数。

删除一个旧的关键帧

当考虑删除旧关键帧时，一个理论问题将显现出来。例如我们要删除旧关键帧 $\boldsymbol{x}_1$，但是 $\boldsymbol{x}_1$ 并不是孤立的，它会和其他帧观测到同样的路标。将 $\boldsymbol{x}_1$ 边缘化之后将导致整个问题不再稀疏。和第 9 讲一样，我们举一个示意图，如图 10-2 所示。

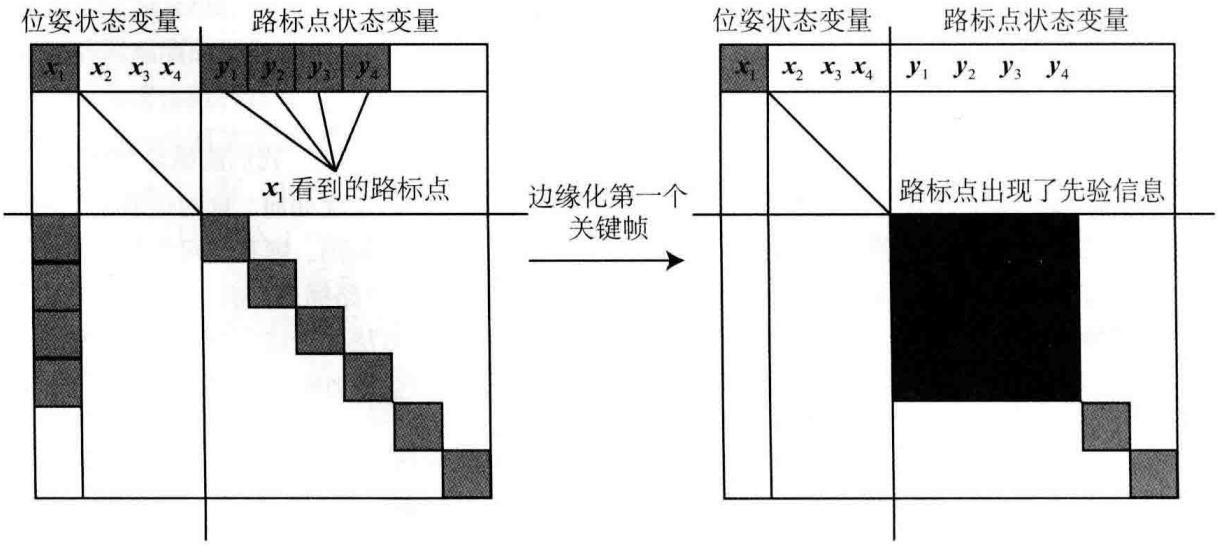


图 10-2 滑动窗口删除关键帧将破坏路标部分的对角块结构

在这个例子中，我们假设 $\boldsymbol{x}_1$ 看到了路标点 $\boldsymbol{y}_1$ 至 $\boldsymbol{y}_4$，于是，在处理之前，BA 问题的 Hessian 矩阵应该像图 10-2 中的左图一样，在 $\boldsymbol{x}_1$ 行的 $\boldsymbol{y}_1$ 到 $\boldsymbol{y}_4$ 列存在着非零块，表示 $\boldsymbol{x}_1$ 看到了它们。这时考虑边缘化 $\boldsymbol{x}_1$，那么 Schur 消元过程相当于通过矩阵行和列操作消去非对角线处几个非零矩阵块，显然这将导致右下角的路标点矩阵块不再是非对角矩阵。这个过程称为边缘化中的**填入**（Fill-in）[91]。

回顾第 9 讲中介绍的边缘化，当我们边缘化路标点时，Fill-in 将出现在左上角的位姿块中。不过，因为 BA 不要求位姿块为对角块，所以稀疏 BA 求解仍然可行。但是，当边缘化关键帧时，将破坏右下角路标点之间的对角块结构，这时 BA 就无法按照先前的稀疏方式迭代求解。这显然是个十分糟糕的问题。实际上，在早期的 EKF 滤波器后端中，人们确实保持着一个稠密的 Hessian 矩阵，这也使得 EKF 后端没法处理较大规模的滑动窗口。

不过，如果我们对边缘化的过程进行一些改造，也可以保持滑动窗口 BA 的稀疏性。例如，在边缘化某个旧的关键帧时，同时边缘化它观测到的路标点。这样，路标点的信息就会转换成剩下那些关键帧之间的共视信息，从而保持右下角部分的对角块结构。在某些 SLAM 框架[70, 89] 中，边缘化策略会更复杂。例如在 OKVIS 中，我们会判断要边缘化的那个关键帧，它看到的路标点是否在最新的关键帧中仍能看到。如果不能，就直接边缘化这个路标点；如果能，就丢弃被边缘化关键帧对这个路标点的观测，从而保持 BA 的稀疏性。

SWF 中边缘化的直观解释

我们知道边缘化在概率上的意义就是指条件概率。所以，当我们边缘化某个关键帧，即“保持这个关键帧当前的估计值，求其他状态变量以这个关键帧为条件的条件概率”。所以，当某个关键帧被边缘化，它观测到的路标点就会产生一个“**这些路标应该在哪里**”的先验信息，从而影响其余部分的估计值。如果再边缘化这些路标点，那么它们的观测者将得到一个“**观测它们的关键帧应该在哪里**”的先验信息。

从数学上看，当我们边缘化某个关键帧，整个窗口中的状态变量的描述方式，将从联合分布变成一个条件概率分布。以上面的例子来看，就是说：

$$p(\boldsymbol{x}_1, \dots \boldsymbol{x}_4, \boldsymbol{y}_1, \dots \boldsymbol{y}_6) = p(\boldsymbol{x}_2, \dots, \boldsymbol{x}_4, \boldsymbol{y}_1, \dots \boldsymbol{y}_6 | \boldsymbol{x}_1) \underbrace{p(\boldsymbol{x}_1)}_{\text{舍去}}. \tag{10.1}$$

然后舍去被边缘化部分的信息。在变量被边缘化之后，我们在工程中就不应再使用它。所以滑动窗口法比较适合 VO 系统，而不适合大规模建图的系统。

由于现在 g2o 和 Ceres 还未直接支持滑动窗口法中的边缘化操作①，我们略去本节对应的实验部分。希望理论部分可以帮助读者理解基于滑动窗口的 SLAM 系统。

---

①工程中我们可以通过某些巧妙的手段绕过 g2o 和 Ceres 的框架限制，但这往往非常烦琐，不适合在书中演示。

10.2 位姿图

10.2.1 位姿图的意义

根据前面的讨论，我们发现特征点在优化问题中占据了绝大部分。实际上，经过若干次观测之后，收敛的特征点位置变化很小，发散的外点则已被剔除。对收敛点再进行优化，似乎是有些费力不讨好的。因此，我们更倾向于在优化几次之后就把特征点固定住，只把它们看作位姿估计的约束，而不再实际地优化它们的位置估计。

沿着这个思路继续思考，我们会想到：是否能够完全不管路标而只管轨迹呢？我们完全可以构建一个只有轨迹的图优化，而位姿节点之间的边，可以由两个关键帧之间通过特征匹配之后得到的运动估计来给定初始值。不同的是，一旦初始估计完成，我们就不再优化那些路标点的位置，而只关心所有的相机位姿之间的联系。通过这种方式，我们省去了大量的特征点优化的计算，只保留了关键帧的轨迹，从而构建了所谓的位姿图（Pose Graph），如图 10-3 所示。

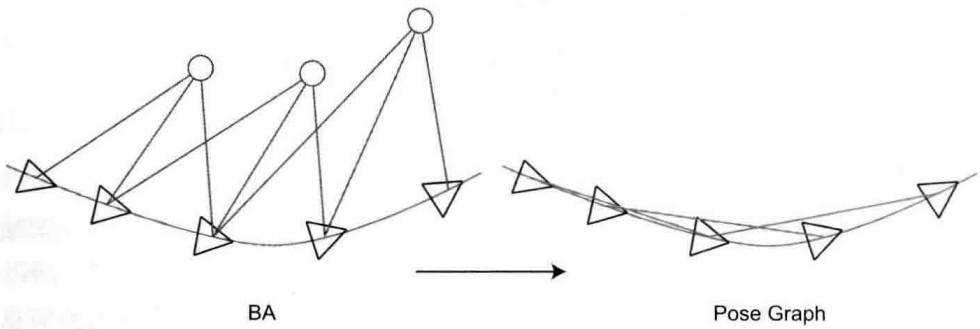


图 10-3 位姿图示意图。当我们不再优化 BA 中的路标点，仅把它们看成对姿态节点的约束时，就得到了一个计算规模减小很多的位姿图

我们知道，在 BA 中特征点数量远大于位姿节点。一个关键帧往往关联了数百个关键点，而实时 BA 的最大计算规模，即使利用稀疏性，在当前的主流 CPU 上一般也就是几万个点左右。这就限制了 SLAM 应用场景。所以，当机器人在更大范围的时间和空间中运动时，必须考虑一些解决方式：要么像滑动窗口法那样，丢弃一些历史数据[92]；要么像位姿图的做法那样，舍弃对路标点的优化，只保留 Pose 之间的边[93–95]。此外，如果我们有额外测量 Pose 的传感器，那么位姿图也是一种常见的融合 Pose 测量的方法。

10.2.2 位姿图的优化

那么，位姿图优化中的节点和边都是什么意思呢？这里的节点表示相机位姿，以 $\boldsymbol{T}_1, \cdots, \boldsymbol{T}_n$ 来表达。而边，则是两个位姿节点之间相对运动的估计，该估计可以来自于特征点法或直接法，也可以来自 GPS 或 IMU 积分。无论通过哪种手段，假设我们估计了 $\boldsymbol{T}_i$ 和 $\boldsymbol{T}_j$ 之间的一个运动

$\Delta \boldsymbol{T}_{ij}$。该运动可以有若干种表达方式，我们取比较自然的一种：

$$\Delta \boldsymbol{\xi}_{ij} = \boldsymbol{\xi}_i^{-1} \circ \boldsymbol{\xi}_j = \ln \left(\boldsymbol{T}_i^{-1} \boldsymbol{T}_j \right)^\vee, \tag{10.2}$$

或按李群的写法：

$$\boldsymbol{T}_{ij} = \boldsymbol{T}_i^{-1} \boldsymbol{T}_j. \tag{10.3}$$

按照图优化的思路，实际中该等式不会精确地成立，因此我们设立最小二乘误差，然后和以往一样，讨论误差关于优化变量的导数。这里，我们把上式的 $\boldsymbol{T}_{ij}$ 移至等式右侧，构建误差 $\boldsymbol{e}_{ij}$：

$$\boldsymbol{e}_{ij} = \Delta \boldsymbol{\xi}_{ij} \ln \left(\boldsymbol{T}_{ij}^{-1} \boldsymbol{T}_i^{-1} \boldsymbol{T}_j \right)^\vee \tag{10.4}$$

注意优化变量有两个：$\boldsymbol{\xi}_i$ 和 $\boldsymbol{\xi}_j$，因此我们求 $\boldsymbol{e}_{ij}$ 关于这两个变量的导数。按照李代数的求导方式，给 $\boldsymbol{\xi}_i$ 和 $\boldsymbol{\xi}_j$ 各一个左扰动：$\boldsymbol{\delta\xi}_i$ 和 $\boldsymbol{\delta\xi}_j$。于是误差变为

$$\hat{\boldsymbol{e}}_{ij} = \ln \left(\boldsymbol{T}_{ij}^{-1} \boldsymbol{T}_i^{-1} \exp((-\boldsymbol{\delta\xi}_i)^\wedge) \exp(\delta \boldsymbol{\xi}_j^\wedge) \boldsymbol{T}_j \right)^\vee. \tag{10.5}$$

该式中，两个扰动项被夹在了中间。为了利用 BCH 近似，我们希望把扰动项移至式子左侧或右侧。回忆第 4 讲习题中的伴随性质，即式 (4.55)。如果你没有做过这个习题，那就暂时把它当作是正确的结论来使用：

$$\exp\left((\mathrm{Ad}(\boldsymbol{T})\boldsymbol{\xi})^\wedge \right) = \boldsymbol{T} \exp(\boldsymbol{\xi}^\wedge) \boldsymbol{T}^{-1}. \tag{10.6}$$

稍加改变，有

$$\exp(\boldsymbol{\xi}^\wedge) \boldsymbol{T} = \boldsymbol{T} \exp\left(\left(\mathrm{Ad}(\boldsymbol{T}^{-1})\boldsymbol{\xi} \right)^\wedge \right). \tag{10.7}$$

该式表明，通过引入一个伴随项，我们能够“交换”扰动项左右侧的 $\boldsymbol{T}$。利用它，可以将扰动挪到最右（当然最左亦可），导出右乘形式的雅可比矩阵（挪到左边时形成左乘）：

$$\begin{aligned}
\hat{\boldsymbol{e}}_{ij} &= \ln \left(\boldsymbol{T}_{ij}^{-1} \boldsymbol{T}_i^{-1} \exp((-\boldsymbol{\delta\xi}_i)^\wedge) \exp(\delta \boldsymbol{\xi}_j^\wedge) \boldsymbol{T}_j \right)^\vee \\
&= \ln \left(\boldsymbol{T}_{ij}^{-1} \boldsymbol{T}_i^{-1} \boldsymbol{T}_j \exp\left(\left(-\mathrm{Ad}(\boldsymbol{T}_j^{-1}) \boldsymbol{\delta\xi}_i \right)^\wedge \right) \exp\left(\left(\mathrm{Ad}(\boldsymbol{T}_j^{-1}) \boldsymbol{\delta\xi}_j \right)^\wedge \right) \right)^\vee \\
&\approx \ln \left(\boldsymbol{T}_{ij}^{-1} \boldsymbol{T}_i^{-1} \boldsymbol{T}_j \left[\boldsymbol{I} - (\mathrm{Ad}(\boldsymbol{T}_j^{-1}) \boldsymbol{\delta\xi}_i)^\wedge + (\mathrm{Ad}(\boldsymbol{T}_j^{-1}) \boldsymbol{\delta\xi}_j)^\wedge \right] \right)^\vee \\
&\approx \boldsymbol{e}_{ij} + \frac{\partial \boldsymbol{e}_{ij}}{\partial \boldsymbol{\delta\xi}_i} \boldsymbol{\delta\xi}_i + \frac{\partial \boldsymbol{e}_{ij}}{\partial \boldsymbol{\delta\xi}_j} \boldsymbol{\delta\xi}_j
\end{aligned} \tag{10.8}$$

因此，按照李代数上的求导法则，我们求出了误差关于两个位姿的雅可比矩阵。关于 $\boldsymbol{T}_i$ 的：

$$\frac{\partial \boldsymbol{e}_{ij}}{\partial \delta \boldsymbol{\xi}_i} = -\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{e}_{ij})\mathrm{Ad}(\boldsymbol{T}_j^{-1}). \tag{10.9}$$

以及关于 $\boldsymbol{T}_j$ 的：

$$\frac{\partial \boldsymbol{e}_{ij}}{\partial \delta \boldsymbol{\xi}_j} = \boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{e}_{ij})\mathrm{Ad}(\boldsymbol{T}_j^{-1}). \tag{10.10}$$

如果读者觉得这部分求导理解起来有困难，可以回到第 4 讲温习李代数部分的内容。不过，前面也说过，由于 $\mathfrak{se}(3)$ 上的左右雅可比 $\boldsymbol{\mathcal{J}}_r$ 形式过于复杂，我们通常取它们的近似。如果误差接近零，我们就可以设它们近似为 $\boldsymbol{I}$ 或

$$\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{e}_{ij}) \approx \boldsymbol{I} + \frac{1}{2}\begin{bmatrix} \boldsymbol{\phi}_{\boldsymbol{e}}^{\wedge} & \boldsymbol{\rho}_{\boldsymbol{e}}^{\wedge} \\ \boldsymbol{0} & \boldsymbol{\phi}_{\boldsymbol{e}}^{\wedge} \end{bmatrix}. \tag{10.11}$$

理论上，即使在优化之后，由于每条边给定的观测数据并不一致，误差也不见得近似于零，所以简单地把这里的 $\boldsymbol{\mathcal{J}}_r$ 设置为 $\boldsymbol{I}$ 会有一定的损失。稍后我们将通过实践了解理论上的区别是否明显。

了解雅可比求导后，剩下的部分就和普通的图优化一样了。简而言之，所有的位姿顶点和位姿——位姿边构成了一个图优化，本质上是一个最小二乘问题，优化变量为各个顶点的位姿，边来自于位姿观测约束。记 $\mathcal{E}$ 为所有边的集合，那么总体目标函数为

$$\min \frac{1}{2}\sum_{i,j\in\mathcal{E}} \boldsymbol{e}_{ij}^{\mathrm{T}}\boldsymbol{\Sigma}_{ij}^{-1}\boldsymbol{e}_{ij}. \tag{10.12}$$

我们依然可以用高斯牛顿法、列文伯格—马夸尔特方法等求解此问题，除了用李代数表示优化位姿，别的都是相似的。根据先前的经验，可以用 Ceres 或 g2o 进行求解。我们不再讨论优化的详细过程，第 9 讲已经讲清楚了。

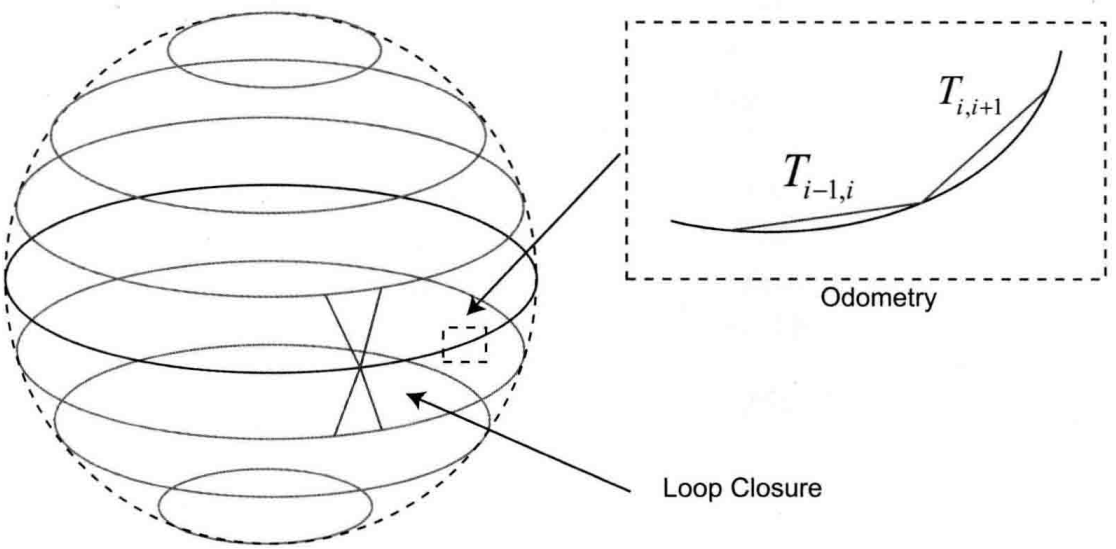
10.3 实践：位姿图优化

10.3.1 g2o 原生位姿图

下面演示如何使用 g2o 进行位姿图优化。首先，请读者用 g2o_viewer 打开我们预先生成的仿真位姿图，位于 slambook2/ch10/sphere.g2o 中，如图 10-4 所示。

该位姿图是由 g2o 自带的 create sphere 程序仿真生成的。它的真实轨迹为一个球，由从下往上的多个层组成。每层为一个正圆形，很多个大小不一的圆形层组成了一个完整的球体，共包含 2500 个位姿节点（图 10-4 左上），可以看成一个转圈上升的过程。然后，仿真程序生成了 $t-1$

到 t 时刻的边，称为里程计边。此外，又生成层与层之间的边，称为回环（回环检测算法将在第 11 讲详细介绍）。随后，在每条边上添加观测噪声，并根据里程计边的噪声，重新设置节点的初始值。这样，就得到了带累积误差的位姿图数据（图 10-4 右下）。它局部看起来像球体的一部分，但整体形状与球体相差甚远。现在，我们从这些带噪声的边和节点初始值出发，尝试优化整个位姿图，得到近似真值的数据。



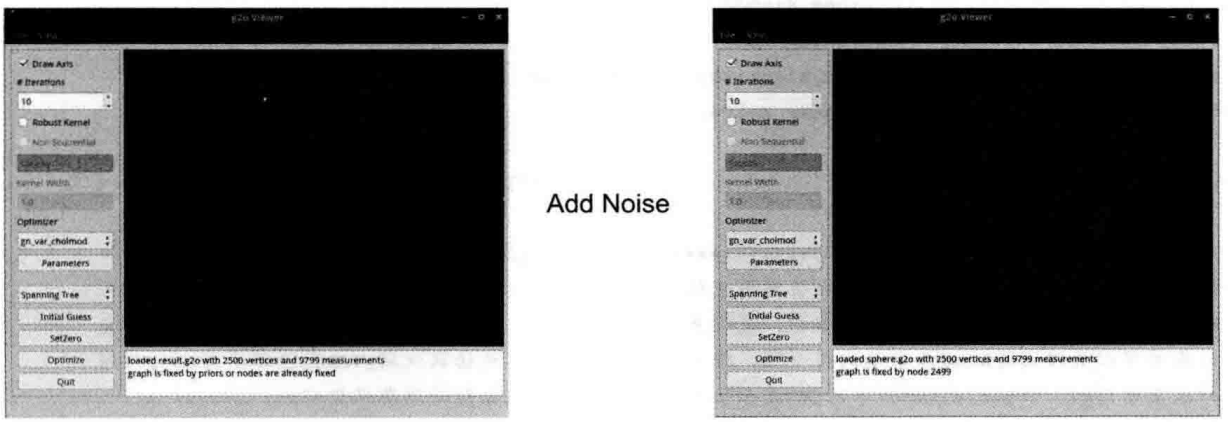


图 10-4　g2o 仿真产生的位姿图。真值是完整的球形，在真值上添加噪声后得到带累计误差的仿真数据

当然，实际中的机器人肯定不会出现这样正球形的运动轨迹，以及如此完整的里程计与回环观测数据。仿真成正球的好处是我们能够直观地看到优化结果是否正确（只要看它各个角度圆不圆就行了）。读者可以单击 g2o_viewer 中的 optimize 函数，看到每步的优化结果和收敛的过程。另外，sphere.g2o 也是一个文本文件，可以用文本编辑器打开，查看它里面的内容。文件前半部

分由节点组成，后半部分则是边：

```
VERTEX_SE3:QUAT 0 -0.125664 -1.53894e-17 99.9999 0.706662 4.32706e-17 0.707551 -4.3325e-17
......
EDGE_SE3:QUAT 1524 1574 -0.210399 -0.0101193 -6.28806 -0.00122939 0.0375067 -2.85291e-05
0.999296 10000 0 0 0 0 0 10000 0 0 0 0 10000 0 0 0 40000 0 0 40000 0 40000
```

可以看到，节点类型是 VERTEX_SE3，表达一个相机位姿。g2o 默认使用四元数和平移向量表达位姿，所以后面的字段意义为：ID，$t_x, t_y, t_z, q_x, q_y, q_z, q_w$。前 3 个为平移向量元素，后 4 个为表示旋转的单位四元数。同样，边的信息为两个节点的 ID，$t_x, t_y, t_z, q_x, q_y, q_z, q_w$，以及信息矩阵的右上角。可以看出，本例中信息矩阵大小为 6×6，且被设成了对角阵。

为了优化该位姿图，我们可以使用 g2o 默认的顶点和边，它们是由四元数表示的。由于仿真数据也是 g2o 生成的，所以用 g2o 本身优化就无须我们多做什么工作了，只需配置优化参数即可。程序 slambook2/ch10/pose_graph_g2o_SE3.cpp 演示了如何使用列文伯格—马夸尔特方法对该位姿图进行优化，并把结果存储至 result.g2o 文件中。

slambook2/ch10/pose_graph_g2o_SE3.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>

#include <g2o/types/slam3d/types_slam3d.h>
#include <g2o/core/block_solver.h>
#include <g2o/core/optimization_algorithm_levenberg.h>
#include <g2o/solvers/eigen/linear_solver_eigen.h>

using namespace std;

/************************************************
* 本程序演示如何用g2o solver进行位姿图优化
* sphere.g2o是人工生成的一个位姿图，我们来优化它。
* 尽管可以直接通过load函数读取整个图，但我们还是自己来实现读取代码，以期获得更深刻的理解
* 这里使用g2o/types/slam3d/中的SE3表示位姿，它实质上是四元数而非李代数
* ************************************************/

int main(int argc, char **argv) {
    if (argc != 2) {
        cout << "Usage: pose_graph_g2o_SE3 sphere.g2o" << endl;
        return 1;
    }
    ifstream fin(argv[1]);
    if (!fin) {
```

```
        cout << "file " << argv[1] << " does not exist." << endl;
        return 1;
    }

    // 设定g2o
    typedef g2o::BlockSolver<g2o::BlockSolverTraits<6, 6>> BlockSolverType;
    typedef g2o::LinearSolverEigen<BlockSolverType::PoseMatrixType> LinearSolverType;
    auto solver = new g2o::OptimizationAlgorithmLevenberg(
    g2o::make_unique<BlockSolverType>(g2o::make_unique<LinearSolverType>()));
    g2o::SparseOptimizer optimizer;     // 图模型
    optimizer.setAlgorithm(solver);   // 设置求解器
    optimizer.setVerbose(true);       // 打开调试输出

    int vertexCnt = 0, edgeCnt = 0; // 顶点和边的数量
    while (!fin.eof()) {
        string name;
        fin >> name;
        if (name == "VERTEX_SE3:QUAT") {
            // SE3顶点
            g2o::VertexSE3 *v = new g2o::VertexSE3();
            int index = 0;
            fin >> index;
            v->setId(index);
            v->read(fin);
            optimizer.addVertex(v);
            vertexCnt++;
            if (index == 0)
            v->setFixed(true);
        } else if (name == "EDGE_SE3:QUAT") {
            // SE3-SE3边
            g2o::EdgeSE3 *e = new g2o::EdgeSE3();
            int idx1, idx2;     // 关联的两个顶点
            fin >> idx1 >> idx2;
            e->setId(edgeCnt++);
            e->setVertex(0, optimizer.vertices()[idx1]);
            e->setVertex(1, optimizer.vertices()[idx2]);
            e->read(fin);
            optimizer.addEdge(e);
        }
        if (!fin.good()) break;
    }

    cout << "read total " << vertexCnt << " vertices, " << edgeCnt << " edges." << endl;
```

```
69
70      cout << "optimizing ..." << endl;
71      optimizer.initializeOptimization();
72      optimizer.optimize(30);
73
74      cout << "saving optimization results ..." << endl;
75      optimizer.save("result.g2o");
76
77      return 0;
78  }
```

我们选择了 6×6 的块求解器，使用列文伯格—马夸尔特下降方式，迭代次数选择 30 次。调用此程序对位姿图进行优化：

终端输入：

```
$ build/pose_graph_g2o_SE3 sphere.g2o
read total 2500 vertices, 9799 edges.
optimizing ...
iteration= 0  chi2= 1023011093.851879 edges= 9799 schur= 0 lambda= 805.622433 levenbergIter=
1
iteration= 1  chi2= 385118688.233188  time= 0.863567 cumTime= 1.71545  edges= 9799 schur= 0
lambda= 537.081622 levenbergIter= 1
iteration= 2  chi2= 166223726.693659  time= 0.861235 cumTime= 2.57668  edges= 9799 schur= 0
lambda= 358.054415 levenbergIter= 1
iteration= 3  chi2= 86610874.269316   time= 0.844105 cumTime= 3.42079  edges= 9799 schur= 0
lambda= 238.702943 levenbergIter= 1
iteration= 4  chi2= 40582782.710190   time= 0.862221 cumTime= 4.28301  edges= 9799 schur= 0
lambda= 159.135295 levenbergIter= 1
......
iteration= 28 chi2= 45095.174398 time= 0.869451 cumTime= 30.0809 edges= 9799 schur= 0 lambda=
 0.003127 levenbergIter= 1
iteration= 29 chi2= 44811.248504 time= 1.76326  cumTime= 31.8442 edges= 9799 schur= 0 lambda=
 0.003785 levenbergIter= 2
saving optimization results ...
```

然后，用 g2o_viewer 打开 result.g2o 查看结果，如图 10-5 所示。

结果从一个不规则的形状优化成了一个看起来完整的球。这个过程实质上和我们单击 g2o_viewer 上的 Optimize 按钮没有区别。下面，我们根据前面的李代数推导来实现李代数上的优化。



图 10-5　使用 g2o 自带的顶点与边求解的结果

10.3.2　李代数上的位姿图优化

还记得我们用 Sophus 表达李代数的事情吗？我们来试试把 Sophus 用到 g2o 中定义自己的顶点和边吧。

slambook2/ch10/pose_graph_g2o_lie_algebra.cpp（片段）

```
typedef Matrix<double, 6, 6> Matrix6d;

// 给定误差求J_R^{-1}的近似
Matrix6d JRInv(const SE3d &e) {
    Matrix6d J;
    J.block(0, 0, 3, 3) = SO3d::hat(e.so3().log());
    J.block(0, 3, 3, 3) = SO3d::hat(e.translation());
    J.block(3, 0, 3, 3) = Matrix3d::Zero(3, 3);
    J.block(3, 3, 3, 3) = SO3d::hat(e.so3().log());
    J = J * 0.5 + Matrix6d::Identity();
    return J;
}

```